

```
In [2]: import pandas as pd

# FILE PATH
mts_path = r"X:\Lighting Prediction\Jharkhand all District Weather.csv"
mts_out = r"X:\Lighting Prediction\cleaned_mts_step1.csv"

# LOAD
mts = pd.read_csv(mts_path, low_memory=False)

# 1) REMOVE UNNAMED COLUMNS
mts = mts.loc[:, ~mts.columns.str.contains('^Unnamed')]

# 2) DROP REDUNDANT COLUMNS
mts_drop_cols = [
    'system:index', 'Remarks', 'Shape_Area', 'Shape_Leng',
    'State_LGD', '.geo', 'ObjectID'
]
mts = mts.drop(columns=[c for c in mts_drop_cols if c in mts.columns], erro

# 3) DROP DUPLICATE ROWS
mts = mts.drop_duplicates()

# SAVE CLEANED FILE
mts.to_csv(mts_out, index=False)

print("MTS cleaning step completed.")
print("Saved to:", mts_out)
```

MTS cleaning step completed.

Saved to: X:\Lighting Prediction\cleaned_mts_step1.csv

```
In [3]: import pandas as pd

mts = pd.read_csv(r"X:\Lighting Prediction\cleaned_mts_step1.csv", low_memo
print(mts.columns)
print(mts.shape)
```

```
Index(['DISTRICT_L', 'District', 'Pressure_hPa', 'Rh', 'Rain_mm', 'State',
      ' ', 'Temp_C', 'WindSpeed', 'Date'],
      dtype='object')
(54756, 10)
```



```

In [4]: import pandas as pd

# Load cleaned MTS from step 1
mts_path = r"X:\Lighting Prediction\cleaned_mts_step1.csv"
mts_out = r"X:\Lighting Prediction\cleaned_mts_step2.csv"

mts = pd.read_csv(mts_path, low_memory=False)

# =====
# 1) DELETE GARBAGE COLUMN " "
# =====
if " " in mts.columns:
    mts = mts.drop(columns=[" "], errors='ignore')

# =====
# 2) STANDARDIZE DISTRICT NAMES
# =====

# Prefer the "District" column, since it's readable.
if "DISTRICT_L" in mts.columns and "District" in mts.columns:
    # If District is empty but DISTRICT_L has value, fill it
    mts['District'] = mts['District'].fillna(mts['DISTRICT_L'])

    # Drop DISTRICT_L after merging
    mts = mts.drop(columns=['DISTRICT_L'], errors='ignore')

# Strip whitespace and uppercase/lowercase issues
if "District" in mts.columns:
    mts['District'] = mts['District'].astype(str).str.strip().str.title()

# =====
# 3) CORRECT STATE NAMES
# =====
if "State" in mts.columns:
    # Many weather datasets have weird state text. We unify them.
    mts['State'] = "Jharkhand"

# =====
# 4) HANDLE NULL VALUES (IMPORTANT COLUMNS ONLY)
# =====

# Essential weather predictors:
essential_cols = [
    'District', 'Pressure_hPa', 'Rh', 'Rain_mm',
    'Temp_C', 'WindSpeed', 'Date'
]

# Drop rows where essential columns are null
existing_essentials = [c for c in essential_cols if c in mts.columns]
mts = mts.dropna(subset=existing_essentials)

# =====
# SAVE FILE
# =====
mts.to_csv(mts_out, index=False)

print("MTS Steps 1-4 completed.")
print("Saved to:", mts_out)

```

```
MTS Steps 1-4 completed.  
Saved to: X:\Lighting Prediction\cleaned_mts_step2.csv
```

```

In [5]: import pandas as pd

mts_path = r"X:\Lighting Prediction\cleaned_mts_step2.csv"
mts_out = r"X:\Lighting Prediction\cleaned_mts_step3.csv"

mts = pd.read_csv(mts_path, low_memory=False)

# -----
# 1) BASIC DISTRICT CLEANING
# -----
if "District" in mts.columns:
    # Convert to string, strip spaces, and title case
    mts["District"] = (
        mts["District"]
        .astype(str)
        .str.strip()
        .str.replace(r"\s+", " ", regex=True) # multiple spaces → single
        .str.title()
    )

    # Optional: normalize some known variants
    replace_map = {
        "Saraikela Kharsawan": "Seraikela-Kharsawan",
        "Seraikela Kharsawan": "Seraikela-Kharsawan",
        "Purbi Singhbhum": "East Singhbhum",
        "Pashchimi Singhbhum": "West Singhbhum",
        "Pashchim Singhbhum": "West Singhbhum",
        "Purba Singhbhum": "East Singhbhum",
        "Palamau": "Palamu",
        "Sahebganj": "Sahibganj",
        "Ramgarh District": "Ramgarh",
        "Latehar District": "Latehar",
        "Ranchi District": "Ranchi",
    }

    mts["District"] = mts["District"].replace(replace_map)

    # Define the valid Jharkhand district list (canonical)
    valid_districts = [
        "Ranchi", "East Singhbhum", "West Singhbhum", "Seraikela-Kharsawan",
        "Dhanbad", "Bokaro", "Hazaribagh", "Ramgarh", "Giridih", "Koderma",
        "Chatra", "Latehar", "Palamu", "Garhwa", "Lohardaga", "Gumla",
        "Simdega", "Khunti", "Godda", "Sahibganj", "Pakur", "Jamtara",
        "Deoghar", "Dumka"
    ]

    # Show any district names that are still weird / outside this list
    print("Unique district names (AFTER cleaning):")
    print(sorted(mts["District"].unique()))

    print("\nDistrict names NOT in the expected Jharkhand list:")
    unknown = sorted(set(mts["District"].unique()) - set(valid_districts))
    print(unknown)

else:
    print("Column 'District' not found!")

```

Unique district names (AFTER cleaning):

```
['Bokaro', 'Chatra', 'Coordinates":[[[85.51702188198014', 'Coordinates":[[[86.06549236527142', 'Deoghar', 'Dhanbad', 'Dumka', 'Garhwa', 'Giridih', 'Godda', 'Gumla', 'Hazaribagh', 'Jamtara', 'Khunti', 'Kodarma', 'Latehar', 'Lohardaga', 'Pakur', 'Palamu', 'Ramgarh', 'Ranchi', 'Sahibganj', 'Simdega', 'West Singhbhum']
```

District names NOT in the expected Jharkhand list:

```
['Coordinates":[[[85.51702188198014', 'Coordinates":[[[86.06549236527142', 'Kodarma']
```

```
In [6]: # -----
# 2) CLEAN / FIX DATE COLUMN
# -----
if "Date" in mts.columns:
    # Convert to datetime; invalid things (like coordinates) become NaT
    parsed_date = pd.to_datetime(mts["Date"], errors="coerce", dayfirst=True)

    # See how many became NaT (these were bad values like coordinates)
    invalid_mask = parsed_date.isna()
    print("\nNumber of invalid Date entries (will be removed):", invalid_mask.sum())

    # Optionally inspect some invalid entries
    print("\nSample invalid Date values:")
    print(mts.loc[invalid_mask, "Date"].head(10))

    # Keep only rows with valid dates
    mts = mts[~invalid_mask].copy()
    mts["Date"] = parsed_date[~invalid_mask]
else:
    print("Column 'Date' not found!")
```

Number of invalid Date entries (will be removed): 4380

Sample invalid Date values:

```
18    22.54874687798167]
22    22.80792295436749]
42    22.54874687798167]
46    22.80792295436749]
66    22.54874687798167]
70    22.80792295436749]
90    22.54874687798167]
94    22.80792295436749]
114   22.54874687798167]
118   22.80792295436749]
```

Name: Date, dtype: object


```

In [7]: import pandas as pd

in_path = r"X:\Lighting Prediction\cleaned_mts_step2.csv"
out_path = r"X:\Lighting Prediction\cleaned_mts_final.csv"

mts = pd.read_csv(in_path, low_memory=False)

# -----
# 1) CLEAN / FIX DISTRICT NAMES
# -----
if "District" in mts.columns:
    # Basic clean
    mts["District"] = (
        mts["District"]
        .astype(str)
        .str.strip()
        .str.replace(r"\s+", " ", regex=True)
        .str.title()
    )

    # Fix known misspelling
    mts["District"] = mts["District"].replace({
        "Kodarma": "Koderma"
    })

    # Drop rows where District is actually coordinates / junk
    bad_district_mask = mts["District"].str.startswith("Coordinates", na=False)
    print("Rows with junk 'Coordinates...' in District:", bad_district_mask)
    mts = mts[~bad_district_mask].copy()

    print("Unique districts after cleaning:")
    print(sorted(mts["District"].unique()))
else:
    print("Column 'District' not found!")

# -----
# 2) FIX DATE COLUMN (REMOVE COORDINATE VALUES)
# -----
if "Date" in mts.columns:
    parsed_date = pd.to_datetime(mts["Date"], errors="coerce", dayfirst=True)

    invalid_mask = parsed_date.isna()
    print("\nNumber of invalid Date entries (will be removed):", invalid_mask.sum())
    print("Sample of invalid Date values:")
    print(mts.loc[invalid_mask, "Date"].head(10))

    # Keep only valid dates
    mts = mts[~invalid_mask].copy()
    mts["Date"] = parsed_date[~invalid_mask]
else:
    print("Column 'Date' not found!")

# -----
# 3) SAVE RESULT
# -----
mts.to_csv(out_path, index=False)

print("\nSaved cleaned MTS to:", out_path)
print("Final shape:", mts.shape)

```



```
Rows with junk 'Coordinates...' in District: 4380
```

```
Unique districts after cleaning:
```

```
['Bokaro', 'Chatra', 'Deoghar', 'Dhanbad', 'Dumka', 'Garhwa', 'Giridih',  
'Godda', 'Gumla', 'Hazaribagh', 'Jamtara', 'Khunti', 'Koderma', 'Latehar',  
'Lohardaga', 'Pakur', 'Palamu', 'Ramgarh', 'Ranchi', 'Sahibganj', 'Simdega',  
'West Singhbhum']
```

```
Number of invalid Date entries (will be removed): 0
```

```
Sample of invalid Date values:
```

```
Series([], Name: Date, dtype: object)
```

```
Saved cleaned MTS to: X:\Lighting Prediction\cleaned_mts_final.csv
```

```
Final shape: (48180, 8)
```

```

In [8]: import pandas as pd

mts = pd.read_csv(r"X:\Lighting Prediction\cleaned_mts_final.csv")

print("===== SHAPE =====")
print(mts.shape)

# -----
# 1) CHECK DUPLICATES
# -----
dup_count = mts.duplicated().sum()
print("\n===== DUPLICATES =====")
print("Total duplicate rows:", dup_count)

if dup_count > 0:
    print("\nExample duplicate rows:")
    print(mts[mts.duplicated()].head())

# -----
# 2) CHECK NULL VALUES
# -----
print("\n===== NULL VALUES (per column) =====")
print(mts.isnull().sum())

# -----
# 3) CHECK DISTRICT NAMES
# -----

valid_districts = [
    "Ranchi", "East Singhbhum", "West Singhbhum", "Seraikela-Kharsawan",
    "Dhanbad", "Bokaro", "Hazaribagh", "Ramgarh", "Giridih", "Koderma",
    "Chatra", "Latehar", "Palamu", "Garhwa", "Lohardaga", "Gumla",
    "Simdega", "Khunti", "Godda", "Sahibganj", "Pakur", "Jamtara",
    "Deoghar", "Dumka"
]

districts = sorted(mts["District"].unique())

print("\n===== UNIQUE DISTRICTS FOUND =====")
print(districts)

incorrect = sorted(set(districts) - set(valid_districts))

print("\n===== INCORRECT / UNKNOWN DISTRICTS =====")
print(incorrect)

```

```
===== SHAPE =====
```

```
(48180, 8)
```

```
===== DUPLICATES =====
```

```
Total duplicate rows: 0
```

```
===== NULL VALUES (per column) =====
```

```
District      0
```

```
Pressure_hPa   0
```

```
Rh             0
```

```
Rain_mm        0
```

```
State          0
```

```
Temp_C         0
```

```
WindSpeed      0
```

```
Date          0
```

```
dtype: int64
```

```
===== UNIQUE DISTRICTS FOUND =====
```

```
['Bokaro', 'Chatra', 'Deoghar', 'Dhanbad', 'Dumka', 'Garhwa', 'Giridih',  
'Godda', 'Gumla', 'Hazaribagh', 'Jamtara', 'Khunti', 'Koderma', 'Latehar',  
'Lohardaga', 'Pakur', 'Palamu', 'Ramgarh', 'Ranchi', 'Sahibganj', 'Simdeg  
a', 'West Singhbhum']
```

```
===== INCORRECT / UNKNOWN DISTRICTS =====
```

```
[]
```



```

In [9]: import pandas as pd

# -----
# FILE PATHS
# -----
lis_path = r"X:\Lighting Prediction\All District lightning data2017_2022 (1
lis_out  = r"X:\Lighting Prediction\cleaned_lis_final.csv"

# -----
# LOAD
# -----
lis = pd.read_csv(lis_path, low_memory=False)

# =====
# 1) DROP UNNAMED COLUMNS
# =====
lis = lis.loc[:, ~lis.columns.str.contains('^Unnamed')]

# =====
# 2) STANDARDIZE COLUMN NAMES FOR MERGE KEYS
# =====
# Lowercase everything first to make renaming robust
lis.columns = [c.strip() for c in lis.columns]

rename_map = {}

# Handle common variants safely
for col in lis.columns:
    lc = col.lower()
    if lc == "date":
        rename_map[col] = "Date"
    elif lc == "district":
        rename_map[col] = "District"
    elif lc in ["lat", "latitude"]:
        rename_map[col] = "Latitude"
    elif lc in ["long", "longitude", "lon"]:
        rename_map[col] = "Longitude"
    elif lc == "radiance":
        rename_map[col] = "Radiance"
    elif lc == "events":
        rename_map[col] = "Events"
    elif lc in ["objectid", "object_id"]:
        rename_map[col] = "ObjectID"
    elif lc in ["orbitid", "orbit_id"]:
        rename_map[col] = "OrbitID"
    elif lc == "month no":
        rename_map[col] = "Month_No"
    elif lc == "month":
        rename_map[col] = "Month"
    elif lc == "duration":
        rename_map[col] = "Duration"
    elif lc == "utctime":
        rename_map[col] = "UTCTime"

lis = lis.rename(columns=rename_map)

print("Columns after renaming:")
print(lis.columns)

# =====
# 3) DROP REDUNDANT / NON-ESSENTIAL COLUMNS

```

```

# =====
# We KEEP: Date, District, Latitude, Longitude, Radiance, Events, Month, Mo
lis_drop_cols = [
    "ObjectID", "OrbitID", "UTCTime", "Duration", "Groups"
]

lis = lis.drop(columns=[c for c in lis_drop_cols if c in lis.columns], erro

# =====
# 4) CLEAN DISTRICT NAMES (SAME AS MTS)
# =====
if "District" in lis.columns:
    lis["District"] = (
        lis["District"]
        .astype(str)
        .str.strip()
        .str.replace(r"\s+", " ", regex=True)
        .str.title()
    )

    # Normalize as we did for MTS
    replace_map_dist = {
        "Seraikela Kharsawan": "Seraikela-Kharsawan",
        "Seraikela Kharsawan": "Seraikela-Kharsawan",
        "Purbi Singhbhum": "East Singhbhum",
        "Pashchimi Singhbhum": "West Singhbhum",
        "Pashchim Singhbhum": "West Singhbhum",
        "Purba Singhbhum": "East Singhbhum",
        "Palamau": "Palamu",
        "Sahebganj": "Sahibganj",
        "Ramgarh District": "Ramgarh",
        "Latehar District": "Latehar",
        "Ranchi District": "Ranchi",
        "Kodarma": "Koderma"
    }

    lis["District"] = lis["District"].replace(replace_map_dist)

    valid_districts = [
        "Ranchi", "East Singhbhum", "West Singhbhum", "Seraikela-Kharsawan",
        "Dhanbad", "Bokaro", "Hazaribagh", "Ramgarh", "Giridih", "Koderma",
        "Chatra", "Latehar", "Palamu", "Garhwa", "Lohardaga", "Gumla",
        "Simdega", "Khunti", "Godda", "Sahibganj", "Pakur", "Jamtara",
        "Deoghar", "Dumka"
    ]

    print("\nUnique districts in LIS after cleaning:")
    print(sorted(lis["District"].unique()))

    unknown = sorted(set(lis["District"].unique()) - set(valid_districts))
    print("\nUnknown / unexpected district names in LIS:")
    print(unknown)
else:
    print("Column 'District' not found in LIS!")

# =====
# 5) PARSE DATE COLUMN, REMOVE BAD VALUES
# =====
if "Date" in lis.columns:
    parsed_date = pd.to_datetime(lis["Date"], errors="coerce", dayfirst=True

```

```

invalid_mask = parsed_date.isna()
print("\nNumber of invalid Date entries in LIS (will be removed):", inv
print("Sample invalid Date values:")
print(lis.loc[invalid_mask, "Date"].head(10))

lis = lis[~invalid_mask].copy()
lis["Date"] = parsed_date[~invalid_mask]
else:
    print("Column 'Date' not found in LIS!")

# =====
# 6) HANDLE NULLS IN ESSENTIAL COLUMNS + DROP DUPLICATES
# =====
essential_lis = [c for c in ["Date", "District", "Latitude", "Longitude", "
                        if c in lis.columns]

lis = lis.dropna(subset=essential_lis)

# Remove exact duplicate rows
lis = lis.drop_duplicates()

print("\nFinal LIS shape after cleaning:", lis.shape)

# =====
# 7) SAVE CLEANED LIS
# =====
lis.to_csv(lis_out, index=False)
print("Saved cleaned LIS to:", lis_out)

```

Columns after renaming:

```

Index(['ObjectID', 'UTCTime', 'Month', 'Month_No_', 'Date', 'Latitude',
      'Longitude', 'Radiance', 'Duration', 'Groups', 'Events', 'OrbitID',
      'District'],
      dtype='object')

```

Unique districts in LIS after cleaning:

```

['Bokaro', 'Chatra', 'Deogarh', 'Dhanbad', 'Dumka', 'East Singbhum', 'Garh
wa', 'Giridih', 'Godda', 'Gumla', 'Hazaribagh', 'Jamtara', 'Khunti', 'Kode
rma', 'Latehar', 'Lohardaga', 'Pakaur', 'Palamu', 'Ramgarh', 'Ranchi', 'Sa
hibganj', 'Saraikela', 'Simdega', 'West Singbhum']

```

Unknown / unexpected district names in LIS:

```

['Deogarh', 'East Singbhum', 'Pakaur', 'Saraikela', 'West Singbhum']

```

Number of invalid Date entries in LIS (will be removed): 0

Sample invalid Date values:

```

Series([], Name: Date, dtype: int64)

```

Final LIS shape after cleaning: (7795, 8)

Saved cleaned LIS to: X:\Lighting Prediction\cleaned_lis_final.csv


```

In [14]: import pandas as pd

orig_path = r"X:\Lighting Prediction\All District lightning data2017_2022 (
lis_out    = r"X:\Lighting Prediction\cleaned_lis_final_v2.csv"    # NEW file

lis = pd.read_csv(orig_path, low_memory=False)

# 1) Standardise column names
lis.columns = [c.strip() for c in lis.columns]

rename_map = {}
for col in lis.columns:
    lc = col.lower()
    if lc == "objectid":
        rename_map[col] = "ObjectID"
    elif lc == "utctime":
        rename_map[col] = "UTCtime"
    elif lc == "month":
        rename_map[col] = "Month"
    elif lc in ["month_no", "month_no_"]:
        rename_map[col] = "Month_No"
    elif lc == "date":
        rename_map[col] = "Day" # this is just day-of-month (30, 9
    elif lc in ["lat", "latitude"]:
        rename_map[col] = "Latitude"
    elif lc in ["long", "longitude", "lon"]:
        rename_map[col] = "Longitude"
    elif lc == "radiance":
        rename_map[col] = "Radiance"
    elif lc == "duration":
        rename_map[col] = "Duration"
    elif lc == "groups":
        rename_map[col] = "Groups"
    elif lc == "events":
        rename_map[col] = "Events"
    elif lc in ["orbitid", "orbit_id"]:
        rename_map[col] = "OrbitID"
    elif lc == "district":
        rename_map[col] = "District"

lis = lis.rename(columns=rename_map)

print("Columns:", lis.columns)

# 2) Parse UTCtime -> DateTime (explicit format)
def parse_utctime(x):
    try:
        # your format: 2017-150T11:38:33.5483Z (year - dayOfYear T time Z)
        return pd.to_datetime(x, format="%Y-%jT%H:%M:%S.%fZ")
    except Exception:
        try:
            return pd.to_datetime(x, format="%Y-%jT%H:%M:%S%z")
        except Exception:
            return pd.NaT

lis["DateTime"] = lis["UTCtime"].apply(parse_utctime)

print("Total rows:", len(lis))
print("DateTime NaT count:", lis["DateTime"].isna().sum())

# 3) Create proper Date (only date part)

```

```

lis["Date"] = lis["DateTime"].dt.normalize() # 2017-05-30 00:00:00

# 4) Drop redundant columns
drop_cols = ["ObjectID", "UTCtime", "Duration", "Groups", "OrbitID", "DateT
lis = lis.drop(columns=[c for c in drop_cols if c in lis.columns], errors="

# 5) Clean District names
lis["District"] = (
    lis["District"]
    .astype(str)
    .str.strip()
    .str.replace(r"\s+", " ", regex=True)
    .str.title()
)

fix_map = {
    "Deogarh": "Deoghar",
    "East Singhbhum": "East Singhbhum",
    "Pakaur": "Pakur",
    "Seraikela": "Seraikela-Kharsawan",
    "West Singhbhum": "West Singhbhum",
    "Palamau": "Palamu",
    "Sahebganj": "Sahibganj",
    "Kodarma": "Koderma",
}
lis["District"] = lis["District"].replace(fix_map)

valid_districts = [
    "Ranchi", "East Singhbhum", "West Singhbhum", "Seraikela-Kharsawan",
    "Dhanbad", "Bokaro", "Hazaribagh", "Ramgarh", "Giridih", "Koderma",
    "Chatra", "Latehar", "Palamu", "Garhwa", "Lohardaga", "Gumla",
    "Simdega", "Khunti", "Godda", "Sahibganj", "Pakur", "Jamtara",
    "Deoghar", "Dumka"
]
unknown = sorted(set(lis["District"].unique()) - set(valid_districts))
print("Unknown districts after fixing:", unknown)

# 6) NOW carefully drop rows with missing essentials
essential = ["Date", "District", "Latitude", "Longitude", "Events", "Radian
existing_ess = [c for c in essential if c in lis.columns]
print("Using essential columns:", existing_ess)

before_drop = len(lis)
lis = lis.dropna(subset=existing_ess)
after_drop = len(lis)
print("Rows before dropna:", before_drop, "after dropna:", after_drop)

# 7) Remove duplicates
lis = lis.drop_duplicates()
print("Rows after dropping duplicates:", len(lis))

# 8) Save
lis.to_csv(lis_out, index=False)
print("Saved corrected LIS to:", lis_out)

```

```

Columns: Index(['ObjectID', 'UTCtime', 'Month', 'Month_No', 'Day', 'Latitude',
               'Longitude', 'Radiance', 'Duration', 'Groups', 'Events', 'OrbitID',
               'District'],
              dtype='object')
Total rows: 7886
DateTime NaT count: 203
Unknown districts after fixing: []
Using essential columns: ['Date', 'District', 'Latitude', 'Longitude', 'Events', 'Radiance']
Rows before dropna: 7886 after dropna: 7683
Rows after dropping duplicates: 7592
Saved corrected LIS to: X:\Lighting Prediction\cleaned_lis_final_v2.csv

```

```

In [15]: lis = pd.read_csv(r"X:\Lighting Prediction\cleaned_lis_final_v2.csv", parse_dates=[0])
print(lis.head(5))
print(lis.shape)
print(lis["District"].unique())

```

	Month	Month_No	Day	Latitude	Longitude	Radiance	Events	District
0	May	5	30	24.245	85.185	464552	42	Chatra
1	May	5	30	24.301	85.149	943158	79	Chatra
2	May	5	30	24.260	85.238	603861	55	Chatra
3	May	5	30	24.244	85.298	109854	11	Chatra
4	May	5	30	24.255	85.235	191404	24	Chatra

Date

0 2017-05-30

1 2017-05-30

2 2017-05-30

3 2017-05-30

4 2017-05-30

(7592, 9)

['Chatra' 'Bokaro' 'Deoghar' 'Dhanbad' 'Dumka' 'East Singhbhum' 'Garhwa' 'Giridih' 'Godda' 'Gumla' 'Hazaribagh' 'Jamtara' 'Khunti' 'Latehar' 'Lohardaga' 'Pakur' 'Palamu' 'Ramgarh' 'Ranchi' 'Sahibganj' 'Seraikela-Kharsawan' 'Simdega' 'West Singhbhum']

```

In [16]: import pandas as pd

# ----- FILE PATHS -----
mts_path = r"X:\Lighting Prediction\cleaned_mts_final.csv"
lis_path = r"X:\Lighting Prediction\cleaned_lis_final_v2.csv"

merged_out = r"X:\Lighting Prediction\merged_lis_mts.csv"

# ----- LOAD WITH PROPER DATE PARSING -----
mts = pd.read_csv(mts_path, parse_dates=["Date"], low_memory=False)
lis = pd.read_csv(lis_path, parse_dates=["Date"], low_memory=False)

print("MTS shape:", mts.shape)
print("LIS shape:", lis.shape)

# (Optional but safe) ensure District format is clean & consistent
mts["District"] = mts["District"].astype(str).str.strip().str.replace(r"\s+", " ")
lis["District"] = lis["District"].astype(str).str.strip().str.replace(r"\s+", " ")

# ----- MERGE ON ['District', 'Date'] -----
merged = pd.merge(
    lis,
    mts,
    on=["District", "Date"],
    how="inner"    # only keep rows where both LIS & MTS exist
)

print("Merged shape:", merged.shape)
print("Merged columns:\n", merged.columns)

# Quick sanity check: districts and date range
print("\nUnique districts in merged:", sorted(merged["District"].unique()))
print("Date range in merged:", merged["Date"].min(), "to", merged["Date"].max())

# ----- SAVE MERGED DATA -----
merged.to_csv(merged_out, index=False)
print("\nSaved merged dataset to:", merged_out)

```

MTS shape: (48180, 8)

LIS shape: (7592, 9)

Merged shape: (6727, 15)

Merged columns:

```

Index(['Month', 'Month_No', 'Day', 'Latitude', 'Longitude', 'Radiance',
      'Events', 'District', 'Date', 'Pressure_hPa', 'Rh', 'Rain_mm', 'Sta
te',
      'Temp_C', 'WindSpeed'],
      dtype='object')

```

Unique districts in merged: ['Bokaro', 'Chatra', 'Deoghar', 'Dhanbad', 'Dumka', 'Garhwa', 'Giridih', 'Godda', 'Gumla', 'Hazaribagh', 'Jamtara', 'Khunti', 'Latehar', 'Lohardaga', 'Pakur', 'Palamu', 'Ramgarh', 'Ranchi', 'Sahibganj', 'Simdega', 'West Singhbhum']

Date range in merged: 2017-03-04 00:00:00 to 2022-12-26 00:00:00

Saved merged dataset to: X:\Lighting Prediction\merged_lis_mts.csv

```
In [17]: merged = pd.read_csv(r"X:\Lighting Prediction\merged_lis_mts.csv", parse_da
print(merged.shape)
print(merged.head())
```

```
(6727, 15)
```

	Month	Month_No	Day	Latitude	Longitude	Radiance	Events	District	\
0	May	5	30	24.245	85.185	464552	42	Chatra	
1	May	5	30	24.301	85.149	943158	79	Chatra	
2	May	5	30	24.260	85.238	603861	55	Chatra	
3	May	5	30	24.244	85.298	109854	11	Chatra	
4	May	5	30	24.255	85.235	191404	24	Chatra	

	Date	Pressure_hPa	Rh	Rain_mm	State	Temp_C	\
0	2017-05-30	954.519473	64.034182	1.549863	Jharkhand	29.347115	
1	2017-05-30	954.519473	64.034182	1.549863	Jharkhand	29.347115	
2	2017-05-30	954.519473	64.034182	1.549863	Jharkhand	29.347115	
3	2017-05-30	954.519473	64.034182	1.549863	Jharkhand	29.347115	
4	2017-05-30	954.519473	64.034182	1.549863	Jharkhand	29.347115	

	WindSpeed
0	0.682367
1	0.682367
2	0.682367
3	0.682367
4	0.682367

```
In [18]: import pandas as pd

merged_path = r"X:\Lightning Prediction\merged_lis_mts.csv"
df = pd.read_csv(merged_path, parse_dates=["Date"])

print("==== SHAPE ====")
print(df.shape)

print("\n==== COLUMN NAMES ====")
print(df.columns)

print("\n==== NULL VALUES ====")
print(df.isnull().sum())

print("\n==== DUPLICATES ====")
print("Duplicate rows:", df.duplicated().sum())

print("\n==== BASIC STATS (Weather) ====")
print(df[['Temp_C', 'Rh', 'Pressure_hPa', 'Rain_mm', 'WindSpeed']].describe)

print("\n==== BASIC STATS (Lightning) ====")
print(df[['Events', 'Radiance']].describe())

print("\n==== DISTRICTS ====")
print(sorted(df['District'].unique()))

print("\n==== DATE RANGE ====")
print(df['Date'].min(), "to", df['Date'].max())

# LightningOccurred target check (Events > 0)
df['LightningOccurred'] = df['Events'].apply(lambda x: 1 if x > 0 else 0)
print("\nLightningOccurred value counts:")
print(df['LightningOccurred'].value_counts())
```

```
===== SHAPE =====
(6727, 15)
```

```
===== COLUMN NAMES =====
Index(['Month', 'Month_No', 'Day', 'Latitude', 'Longitude', 'Radiance',
      'Events', 'District', 'Date', 'Pressure_hPa', 'Rh', 'Rain_mm', 'State',
      'Temp_C', 'WindSpeed'],
      dtype='object')
```

```
===== NULL VALUES =====
Month          0
Month_No       0
Day            0
Latitude       0
Longitude      0
Radiance       0
Events         0
District       0
Date           0
Pressure_hPa   0
Rh             0
Rain_mm        0
State          0
Temp_C         0
WindSpeed      0
dtype: int64
```

```
===== DUPLICATES =====
Duplicate rows: 0
```

```
===== BASIC STATS (Weather) =====
```

	Temp_C	Rh	Pressure_hPa	Rain_mm	WindSpeed
count	6727.000000	6727.000000	6727.000000	6727.000000	6727.000000
mean	27.600116	73.532344	961.146962	7.478506	1.433067
std	2.642429	14.643551	18.052158	7.388688	0.727925
min	14.968206	22.308832	919.543204	0.001508	0.094504
25%	25.935618	65.312887	947.371090	1.907664	0.840725
50%	27.317705	77.076802	961.082769	5.327132	1.361418
75%	29.062223	85.367887	973.966716	11.413402	1.912874
max	35.489249	96.665602	1002.161356	95.788124	4.322234

```
===== BASIC STATS (Lightning) =====
```

	Events	Radiance
count	6727.000000	6.727000e+03
mean	35.756355	4.388129e+05
std	59.230663	1.604223e+06
min	1.000000	4.158000e+03
25%	8.000000	5.599950e+04
50%	17.000000	1.391150e+05
75%	39.000000	3.628855e+05
max	1051.000000	9.341265e+07

```
===== DISTRICTS =====
['Bokaro', 'Chatra', 'Deoghar', 'Dhanbad', 'Dumka', 'Garhwa', 'Giridih',
 'Godda', 'Gumla', 'Hazaribagh', 'Jamtara', 'Khunti', 'Latehar', 'Lohardag
a', 'Pakur', 'Palamu', 'Ramgarh', 'Ranchi', 'Sahibganj', 'Simdega', 'West
Singhbhum']
```

```
===== DATE RANGE =====
2017-03-04 00:00:00 to 2022-12-26 00:00:00
```

```
LightningOccurred value counts:  
1      6727  
Name: LightningOccurred, dtype: int64
```



```

In [19]: import pandas as pd
import numpy as np

# -----
# 1) LOAD DATA
# -----
mts_path      = r"X:\Lighting Prediction\cleaned_mts_final.csv"
merged_path   = r"X:\Lighting Prediction\merged_lis_mts.csv"

mts           = pd.read_csv(mts_path, parse_dates=["Date"], low_memory=False)
merged        = pd.read_csv(merged_path, parse_dates=["Date"], low_memory=False)

print("MTS shape:", mts.shape)
print("Merged (lightning) shape:", merged.shape)

# Make sure District formatting is consistent
mts["District"] = mts["District"].astype(str).str.strip().str.replace(r"
merged["District"] = merged["District"].astype(str).str.strip().str.replace

# -----
# 2) GET DISTINCT LIGHTNING DAYS (District + Date)
# -----
light_days = merged[["District", "Date"]].drop_duplicates()
print("Unique lightning District-Date pairs:", light_days.shape[0])

# -----
# 3) FIND NO-LIGHTNING DAYS FROM MTS
# -----
# Anti-join: keep only MTS rows that do NOT appear in light_days
mts_nolight = mts.merge(
    light_days,
    on=["District", "Date"],
    how="left",
    indicator=True
)

nolight_df = mts_nolight[mts_nolight["_merge"] == "left_only"].drop(columns
print("No-lightning rows found from MTS:", nolight_df.shape[0])

# -----
# 4) ADD COLUMNS TO MATCH MERGED STRUCTURE
# -----
# merged has columns:
# ['Month', 'Month_No', 'Day', 'Latitude', 'Longitude', 'Radiance', 'Events',
#  'District', 'Date', 'Pressure_hPa', 'Rh', 'Rain_mm', 'State', 'Temp_C', 'WindSp

# For no-light rows, we create Month, Month_No, Day from Date
nolight_df["Month"]      = nolight_df["Date"].dt.strftime("%b")    # e.g. 'Ma
nolight_df["Month_No"]   = nolight_df["Date"].dt.month
nolight_df["Day"]        = nolight_df["Date"].dt.day

# Latitude/Longitude are unknown for no-light days -> set to NaN
nolight_df["Latitude"]   = np.nan
nolight_df["Longitude"]  = np.nan

# No lightning -> Events and Radiance = 0
nolight_df["Events"]     = 0
nolight_df["Radiance"]   = 0

# Set target Label
nolight_df["LightningOccurred"] = 0

```

```

# -----
# 5) PREPARE LIGHTNING (POSITIVE) ROWS
# -----
light_df = merged.copy()
light_df["LightningOccurred"] = 1

# Ensure same column set and order in both
cols_order = [
    "Month", "Month_No", "Day",
    "Latitude", "Longitude",
    "Radiance", "Events",
    "District", "Date",
    "Pressure_hPa", "Rh", "Rain_mm", "State", "Temp_C", "WindSpeed",
    "LightningOccurred"
]

# Reorder columns
light_df = light_df[cols_order]
nolight_df = nolight_df[cols_order]

# -----
# 6) COMBINE INTO ONE FINAL DATASET
# -----
full_df = pd.concat([light_df, nolight_df], ignore_index=True)

print("Full dataset shape (light + no-light):", full_df.shape)
print(full_df["LightningOccurred"].value_counts())

# -----
# 7) SAVE FOR NEXT STEPS
# -----
out_path = r"X:\Lighting Prediction\final_lightning_dataset.csv"
full_df.to_csv(out_path, index=False)
print("Saved final dataset with labels to:", out_path)

```

```

MTS shape: (48180, 8)
Merged (lightning) shape: (6727, 15)
Unique lightning District-Date pairs: 1050
No-lightning rows found from MTS: 47130
Full dataset shape (light + no-light): (53857, 16)
0    47130
1     6727
Name: LightningOccurred, dtype: int64
Saved final dataset with labels to: X:\Lighting Prediction\final_lightning_dataset.csv

```

```

In [20]: import pandas as pd

df = pd.read_csv(r"X:\Lightning Prediction\final_lightning_dataset.csv", par

print("==== SHAPE ====")
print(df.shape)

print("\n==== COLUMNS ====")
print(df.columns)

print("\n==== NULL VALUES PER COLUMN ====")
print(df.isnull().sum())

print("\n==== DUPLICATE ROWS ====")
print(df.duplicated().sum())

print("\n==== CLASS DISTRIBUTION ====")
print(df["LightningOccurred"].value_counts())

print("\n==== DISTRICTS ====")
print(sorted(df["District"].unique()))

print("\n==== DATE RANGE ====")
print(df["Date"].min(), "to", df["Date"].max())

print("\n==== WEATHER STATS ====")
print(df[['Temp_C', 'Rh', 'Pressure_hPa', 'Rain_mm', 'WindSpeed']].describe())

print("\n==== LIGHTNING STATS ====")
print(df[['Events', 'Radiance']].describe())

print("\n==== SAMPLE RECORD (LIGHTNING=1) ====")
print(df[df["LightningOccurred"]==1].head())

print("\n==== SAMPLE RECORD (LIGHTNING=0) ====")
print(df[df["LightningOccurred"]==0].head())

```

```

==== SHAPE ====
(53857, 16)

```

```

==== COLUMNS ====
Index(['Month', 'Month_No', 'Day', 'Latitude', 'Longitude', 'Radiance',
      'Events', 'District', 'Date', 'Pressure_hPa', 'Rh', 'Rain_mm',
      'State',
      'Temp_C', 'WindSpeed', 'LightningOccurred'],
      dtype='object')

```

```

==== NULL VALUES PER COLUMN ====
Month          0
Month_No       0
Day            0
Latitude      47130
Longitude     47130
Radiance       0
Events         0
District       0
~ .           ~

```

In [21]: `import pandas as pd`

```
df = pd.read_csv(r"X:\Lighting Prediction\final_lightning_dataset.csv")
print(df.columns)
```

```
Index(['Month', 'Month_No', 'Day', 'Latitude', 'Longitude', 'Radiance',
      'Events', 'District', 'Date', 'Pressure_hPa', 'Rh', 'Rain_mm', 'Sta
te',
      'Temp_C', 'WindSpeed', 'LightningOccurred'],
      dtype='object')
```

In [22]: `import pandas as pd`

```
df_path = r"X:\Lighting Prediction\final_lightning_dataset.csv"
df_out = r"X:\Lighting Prediction\final_lightning_dataset_cleaned.csv"

df = pd.read_csv(df_path, parse_dates=["Date"])

# 1) Fix negative rain
df["Rain_mm"] = df["Rain_mm"].clip(lower=0)

# 2) Calculate district centroids (from LIS rows where lightning occurred)
centroids = df[df["LightningOccurred"]==1].groupby("District")[["Latitude",

# 3) Replace ALL lat/lon values (both for light & no-light rows)
for district, row in centroids.iterrows():
    df.loc[df["District"] == district, "Latitude"] = row["Latitude"]
    df.loc[df["District"] == district, "Longitude"] = row["Longitude"]

# 4) Save cleaned dataset
df.to_csv(df_out, index=False)

print("Saved cleaned dataset:", df_out)
print("Missing lat:", df["Latitude"].isnull().sum())
print("Missing lon:", df["Longitude"].isnull().sum())
```

Saved cleaned dataset: X:\Lighting Prediction\final_lightning_dataset_cleaned.csv

Missing lat: 2190

Missing lon: 2190

```
In [25]: import pandas as pd

df_path = r"X:\Lighting Prediction\final_lightning_dataset_cleaned.csv"
df_out = r"X:\Lighting Prediction\final_lightning_dataset_cleaned_final.csv"

# Load the dataset
df = pd.read_csv(df_path, parse_dates=["Date"])

# Set Events & Radiance to NaN for no-light rows
df.loc[df["LightningOccurred"] == 0, ["Events", "Radiance"]] = pd.NA

# Save cleaned dataset
df.to_csv(df_out, index=False)

print("Saved cleaned dataset to:", df_out)
print("Remaining NaN in Events:", df["Events"].isna().sum())
print("Remaining NaN in Radiance:", df["Radiance"].isna().sum())
```

Saved cleaned dataset to: X:\Lighting Prediction\final_lightning_dataset_cleaned_final.csv
Remaining NaN in Events: 47130
Remaining NaN in Radiance: 47130

```
In [26]: import pandas as pd

df = pd.read_csv(r"X:\Lighting Prediction\final_lightning_dataset_cleaned_f

print("==== SHAPE ====")
print(df.shape)

print("\n==== COLUMNS ====")
print(df.columns)

print("\n==== DUPLICATES (should be 0) ====")
print(df.duplicated().sum())

print("\n==== NULL VALUES PER COLUMN ====")
print(df.isnull().sum())

print("\n==== CHECK NaN ONLY FOR NO-LIGHT ROWS ====")
print("Events NaN only when LightningOccurred=0:",
      df[df["Events"].isna()][ "LightningOccurred"].unique())

print("Radiance NaN only when LightningOccurred=0:",
      df[df["Radiance"].isna()][ "LightningOccurred"].unique())

print("\n==== WEATHER STATS ====")
print(df[['Temp_C', 'Rh', 'Pressure_hPa', 'Rain_mm', 'WindSpeed']].describe())

print("\n==== CHECK NEGATIVE RAIN VALUES (should be 0) ====")
print((df["Rain_mm"] < 0).sum())

print("\n==== DISTRICTS ====")
print(sorted(df["District"].unique()))

print("\n==== DATE RANGE ====")
print(df["Date"].min(), "to", df["Date"].max())
```

```
===== SHAPE =====
(53857, 16)
```

```
===== COLUMNS =====
Index(['Month', 'Month_No', 'Day', 'Latitude', 'Longitude', 'Radiance',
      'Events', 'District', 'Date', 'Pressure_hPa', 'Rh', 'Rain_mm', 'Sta
te',
      'Temp_C', 'WindSpeed', 'LightningOccurred'],
      dtype='object')
```

```
===== DUPLICATES (should be 0) =====
14
```

```
===== NULL VALUES PER COLUMN =====
Month                0
Month_No             0
Day                 0
Latitude            2190
Longitude           2190
Radiance           47130
Events             47130
District            0
Date                0
Pressure_hPa        0
Rh                  0
Rain_mm             0
State               0
Temp_C              0
WindSpeed           0
LightningOccurred   0
dtype: int64
```

```
===== CHECK NaN ONLY FOR NO-LIGHT ROWS =====
Events NaN only when LightningOccurred=0: [0]
Radiance NaN only when LightningOccurred=0: [0]
```

```
===== WEATHER STATS =====
```

	Temp_C	Rh	Pressure_hPa	Rain_mm	WindSpee
d					
count	53857.000000	53857.000000	53857.000000	53857.000000	53857.000000
0					
mean	24.601811	68.083940	968.080411	4.009739	1.57779
0					
std	4.939552	17.978396	19.758336	7.446609	0.84239
1					
min	9.808196	12.311981	916.330377	0.000000	0.07035
2					
25%	21.038564	57.366502	954.194093	0.001705	0.94893
6					
50%	25.763226	71.225740	968.332408	0.604535	1.46560
4					
75%	27.984056	83.202664	983.193692	5.249822	2.06068
9					
max	37.344510	99.263648	1011.527569	149.790895	6.84202
5					

```
===== CHECK NEGATIVE RAIN VALUES (should be 0) =====
0
```

```
===== DISTRICTS =====
['Bokaro', 'Chatra', 'Deoghar', 'Dhanbad', 'Dumka', 'Garhwa', 'Giridih',
```



```
'Godda', 'Gumla', 'Hazaribagh', 'Jamtara', 'Khunti', 'Koderma', 'Latehar',  
'Lohardaga', 'Pakur', 'Palamu', 'Ramgarh', 'Ranchi', 'Sahibganj', 'Simdeg  
a', 'West Singhbhum']
```

```
===== DATE RANGE =====
```

```
2017-01-01 00:00:00 to 2022-12-30 00:00:00
```

In [27]: `import pandas as pd`

```
df_path = r"X:\Lighting Prediction\final_lightning_dataset_cleaned_final.csv"  
df_out = r"X:\Lighting Prediction\final_lightning_dataset_cleaned_final_v2.csv"  
  
df = pd.read_csv(df_path, parse_dates=["Date"])  
  
# 1) DROP DUPLICATES  
print("Duplicates before:", df.duplicated().sum())  
df = df.drop_duplicates()  
print("Duplicates after:", df.duplicated().sum())  
  
# 2) FILL REMAINING LAT/LON  
# Calculate global means (ignoring NaNs)  
global_lat = df["Latitude"].mean(skipna=True)  
global_lon = df["Longitude"].mean(skipna=True)  
  
df["Latitude"] = df["Latitude"].fillna(global_lat)  
df["Longitude"] = df["Longitude"].fillna(global_lon)  
  
print("Missing lat after:", df["Latitude"].isna().sum())  
print("Missing lon after:", df["Longitude"].isna().sum())  
  
# SAVE CLEANED DATASET  
df.to_csv(df_out, index=False)  
print("Saved to:", df_out)
```

Duplicates before: 14

Duplicates after: 0

Missing lat after: 0

Missing lon after: 0

Saved to: X:\Lighting Prediction\final_lightning_dataset_cleaned_final_v2.csv

In [28]: `import pandas as pd`

```
df_path = r"X:\Lighting Prediction\final_lightning_dataset_cleaned_final_v2.csv"  
df = pd.read_csv(df_path, parse_dates=["Date"])  
  
# Sort by District + Date (required for rolling)  
df = df.sort_values(["District", "Date"]).reset_index(drop=True)
```

```

In [29]: # 1) Day of Year
df["DayOfYear"] = df["Date"].dt.dayofyear

# 2) Week of Year
df["WeekOfYear"] = df["Date"].dt.isocalendar().week.astype(int)

# 3) Season (customized for India)
def get_season(month):
    if month in [12,1,2]:
        return "Winter"
    elif month in [3,4]:
        return "Summer"
    elif month in [5]:
        return "PreMonsoon"
    elif month in [6,7,8,9]:
        return "Monsoon"
    else:
        return "PostMonsoon"

df["Season"] = df["Month_No"].apply(get_season)

# 4) Monsoon Flag
df["IsMonsoon"] = df["Month_No"].apply(lambda x: 1 if x in [6,7,8,9] else 0

```

```

In [30]: df["THI"] = (df["Temp_C"] * df["Rh"]) / 100

```

```

In [31]: df["Temp_3day_mean"] = df.groupby("District")["Temp_C"].rolling(3, min_peri
df["Rh_3day_mean"] = df.groupby("District")["Rh"].rolling(3, min_periods=
df["Rain_3day_sum"] = df.groupby("District")["Rain_mm"].rolling(3, min_per
df["Pres_3day_mean"] = df.groupby("District")["Pressure_hPa"].rolling(3, mi

```

```

In [32]: df["Pressure_Drop"] = df["Pressure_hPa"] - df["Pres_3day_mean"]

```

```

In [33]: df = pd.get_dummies(df, columns=["District"], prefix="Dist", drop_first=False)

```

```

In [34]: df = pd.get_dummies(df, columns=["Season"], prefix="Season", drop_first=False)

```

```

In [37]: import os

folder = r"X:\Lightning Prediction"
print(os.listdir(folder))

```

```

['All District lightning data2017_2022 (1).csv', 'cleaned_lis_final_v2.csv', 'cleaned_mts_final.csv', 'cleaned_mts_step1.csv', 'cleaned_mts_step2.csv', 'final_lightning_dataset.csv', 'final_lightning_dataset_cleaned.csv', 'final_lightning_dataset_cleaned_final.csv', 'final_lightning_dataset_cleaned_final_v2.csv', 'Jharkhand all District Weather.csv', 'merged_lis_mts.csv', 'New folder', 'Processed_Districts']

```



```
In [38]: import pandas as pd

# 0) LOAD CLEANED FINAL DATA
base_path = r"X:\Lighting Prediction\final_lightning_dataset_cleaned_final_"
df = pd.read_csv(base_path, parse_dates=["Date"])
print("Loaded:", df.shape)

# 1) SORT BY DISTRICT + DATE (needed for rolling)
df = df.sort_values(["District", "Date"]).reset_index(drop=True)

# 2) DATE-BASED FEATURES
df["DayOfYear"] = df["Date"].dt.dayofyear
df["WeekOfYear"] = df["Date"].dt.isocalendar().week.astype(int)

def get_season(month):
    if month in [12, 1, 2]:
        return "Winter"
    elif month in [3, 4]:
        return "Summer"
    elif month == 5:
        return "PreMonsoon"
    elif month in [6, 7, 8, 9]:
        return "Monsoon"
    else:
        return "PostMonsoon"

df["Season"] = df["Month_No"].apply(get_season)
df["IsMonsoon"] = df["Month_No"].apply(lambda x: 1 if x in [6,7,8,9] else 0)

# 3) WEATHER-BASED FEATURES
# 3.1 Temperature-Humidity Index (THI)
df["THI"] = (df["Temp_C"] * df["Rh"]) / 100

# 3.2 Rolling features (3-day) per district
df["Temp_3day_mean"] = (
    df.groupby("District")["Temp_C"]
    .rolling(3, min_periods=1).mean()
    .reset_index(0, drop=True)
)

df["Rh_3day_mean"] = (
    df.groupby("District")["Rh"]
    .rolling(3, min_periods=1).mean()
    .reset_index(0, drop=True)
)

df["Rain_3day_sum"] = (
    df.groupby("District")["Rain_mm"]
    .rolling(3, min_periods=1).sum()
    .reset_index(0, drop=True)
)

df["Pres_3day_mean"] = (
    df.groupby("District")["Pressure_hPa"]
    .rolling(3, min_periods=1).mean()
    .reset_index(0, drop=True)
)

# 3.3 Pressure drop = today - rolling mean
df["Pressure_Drop"] = df["Pressure_hPa"] - df["Pres_3day_mean"]
```

```
# 4) ONE-HOT ENCODE DISTRICT
df = pd.get_dummies(df, columns=["District"], prefix="Dist", drop_first=False)

# 5) ONE-HOT ENCODE SEASON
df = pd.get_dummies(df, columns=["Season"], prefix="Season", drop_first=False)

# 6) SAVE FEATURE-ENGINEERED DATASET
fe_path = r"X:\Lighting Prediction\lightning_dataset_FE_v2.csv"
df.to_csv(fe_path, index=False)
print("Saved FE dataset to:", fe_path)
print("Final FE shape:", df.shape)
print("Columns (first 30):", df.columns.tolist()[:30])
```

Loaded: (53843, 16)

Saved FE dataset to: X:\Lighting Prediction\lightning_dataset_FE_v2.csv

Final FE shape: (53843, 51)

Columns (first 30): ['Month', 'Month_No', 'Day', 'Latitude', 'Longitude', 'Radiance', 'Events', 'Date', 'Pressure_hPa', 'Rh', 'Rain_mm', 'State', 'Temp_C', 'WindSpeed', 'LightningOccurred', 'DayOfYear', 'WeekOfYear', 'IsMonsoon', 'THI', 'Temp_3day_mean', 'Rh_3day_mean', 'Rain_3day_sum', 'Pressure_3day_mean', 'Pressure_Drop', 'Dist_Bokaro', 'Dist_Chatra', 'Dist_Deoghar', 'Dist_Dhanbad', 'Dist_Dumka', 'Dist_Garhwa']

```
In [40]: import pandas as pd
import numpy as np

df = pd.read_csv(r"X:\Lighting Prediction\lightning_dataset_FE_v2.csv", par

print("==== SHAPE ====")
print(df.shape)

print("\n==== COLUMNS ====")
print(df.columns.tolist())

print("\n==== NULL VALUES ====")
print(df.isnull().sum())

print("\n==== UNIQUE LABELS ====")
print(df["LightningOccurred"].value_counts())

# Check Events/Radiance NaN for only non-light rows
print("\nEvents NaN check:", df[df["LightningOccurred"]==0]["Events"].isnul
print("Radiance NaN check:", df[df["LightningOccurred"]==0]["Radiance"].isn

# Check no NaN for Lightning rows
print("\nLightning events missing events?", df[df["LightningOccurred"]==1][
print("Lightning radiance missing?", df[df["LightningOccurred"]==1]["Radian

# Check district one-hot encoding (only one per row)
dist_cols = [c for c in df.columns if c.startswith("Dist_")]
df["dist_sum"] = df[dist_cols].sum(axis=1)
print("\nDISTRICT one-hot check: unique sums:", df["dist_sum"].unique()[:5])
df = df.drop(columns=["dist_sum"])

# Check season one-hot encoding
season_cols = [c for c in df.columns if c.startswith("Season_")]
df["season_sum"] = df[season_cols].sum(axis=1)
print("\nSEASON one-hot check: unique sums:", df["season_sum"].unique()[:5])
df = df.drop(columns=["season_sum"])

# Check rolling features are numeric
rolling_cols = ["Temp_3day_mean", "Rh_3day_mean", "Rain_3day_sum", "Pres_3day_
print("\nRolling column types:", df[rolling_cols].dtypes)

# Check Latitude/Longitude
print("\nLatitude/Longitude min/max:")
print(df["Latitude"].min(), df["Latitude"].max())
print(df["Longitude"].min(), df["Longitude"].max())
```

```
===== SHAPE =====
```

```
(53843, 51)
```

```
===== COLUMNS =====
```

```
['Month', 'Month_No', 'Day', 'Latitude', 'Longitude', 'Radiance', 'Event  
s', 'Date', 'Pressure_hPa', 'Rh', 'Rain_mm', 'State', 'Temp_C', 'WindSpee  
d', 'LightningOccurred', 'DayOfYear', 'WeekOfYear', 'IsMonsoon', 'THI', 'T  
emp_3day_mean', 'Rh_3day_mean', 'Rain_3day_sum', 'Pres_3day_mean', 'Pressu  
re_Drop', 'Dist_Bokaro', 'Dist_Chatra', 'Dist_Deoghar', 'Dist_Dhanbad', 'D  
ist_Dumka', 'Dist_Garhwa', 'Dist_Giridih', 'Dist_Godda', 'Dist_Gumla', 'Di  
st_Hazaribagh', 'Dist_Jamtara', 'Dist_Khunti', 'Dist_Koderma', 'Dist_Lateh  
ar', 'Dist_Lohardaga', 'Dist_Pakur', 'Dist_Palamu', 'Dist_Ramgarh', 'Dist_  
Ranchi', 'Dist_Sahibganj', 'Dist_Simdega', 'Dist_West Singhbhum', 'Season_  
Monsoon', 'Season_PostMonsoon', 'Season_PreMonsoon', 'Season_Summer', 'Sea  
son_Winter']
```

```
===== NULL VALUES =====
```

Month	0
Month_No	0
Day	0
Latitude	0
Longitude	0
Radiance	47130
Events	47130
Date	0
Pressure_hPa	0
Rh	0
Rain_mm	0
State	0
Temp_C	0
WindSpeed	0
LightningOccurred	0
DayOfYear	0
WeekOfYear	0
IsMonsoon	0
THI	0
Temp_3day_mean	0
Rh_3day_mean	0
Rain_3day_sum	0
Pres_3day_mean	0
Pressure_Drop	0
Dist_Bokaro	0
Dist_Chatra	0
Dist_Deoghar	0
Dist_Dhanbad	0
Dist_Dumka	0
Dist_Garhwa	0
Dist_Giridih	0
Dist_Godda	0
Dist_Gumla	0
Dist_Hazaribagh	0
Dist_Jamtara	0
Dist_Khunti	0
Dist_Koderma	0
Dist_Latehar	0
Dist_Lohardaga	0
Dist_Pakur	0
Dist_Palamu	0
Dist_Ramgarh	0
Dist_Ranchi	0
Dist_Sahibganj	0

```
Dist_Simdega      0
Dist_West Singhbhum 0
Season_Monsoon    0
Season_PostMonsoon 0
Season_PreMonsoon  0
Season_Summer     0
Season_Winter     0
dtype: int64
```

```
===== UNIQUE LABELS =====
```

```
0    47130
```

```
1     6713
```

```
Name: LightningOccurred, dtype: int64
```

```
Events NaN check: [ True]
```

```
Radiance NaN check: [ True]
```

```
Lightning events missing events? 0
```

```
Lightning radiance missing? 0
```

```
DISTRICT one-hot check: unique sums: [1]
```

```
SEASON one-hot check: unique sums: [1]
```

```
Rolling column types: Temp_3day_mean    float64
```

```
Rh_3day_mean        float64
```

```
Rain_3day_sum        float64
```

```
Pres_3day_mean       float64
```

```
Pressure_Drop        float64
```

```
dtype: object
```

```
Latitude/Longitude min/max:
```

```
22.42043099547511 24.968980769230768
```

```
83.70245228215768 87.6894326923077
```


In [41]: `import pandas as pd`

```
fe_path = r"X:\Lightning Prediction\lightning_dataset_FE_v2.csv"
df = pd.read_csv(fe_path, parse_dates=["Date"])

print(df.shape)
print(df.columns)
```

```
(53843, 51)
Index(['Month', 'Month_No', 'Day', 'Latitude', 'Longitude', 'Radiance',
      'Events', 'Date', 'Pressure_hPa', 'Rh', 'Rain_mm', 'State', 'Temp_
C',
      'WindSpeed', 'LightningOccurred', 'DayOfYear', 'WeekOfYear',
      'IsMonsoon', 'THI', 'Temp_3day_mean', 'Rh_3day_mean', 'Rain_3day_su
m',
      'Pres_3day_mean', 'Pressure_Drop', 'Dist_Bokaro', 'Dist_Chatra',
      'Dist_Deoghar', 'Dist_Dhanbad', 'Dist_Dumka', 'Dist_Garhwa',
      'Dist_Giridih', 'Dist_Godda', 'Dist_Gumla', 'Dist_Hazaribagh',
      'Dist_Jamtara', 'Dist_Khunti', 'Dist_Koderma', 'Dist_Latehar',
      'Dist_Lohardaga', 'Dist_Pakur', 'Dist_Palamu', 'Dist_Ramgarh',
      'Dist_Ranchi', 'Dist_Sahibganj', 'Dist_Simdega', 'Dist_West Singhbh
um',
      'Season_Monsoon', 'Season_PostMonsoon', 'Season_PreMonsoon',
      'Season_Summer', 'Season_Winter'],
      dtype='object')
```

In [42]: `# Target`

```
y = df["LightningOccurred"]
```

`# Columns to drop from X`

```
drop_cols = [
    "LightningOccurred",
    "Events",
    "Radiance",
    "Month",      # text
    "Date",
    "State"
]
```

```
X = df.drop(columns=drop_cols)
```

```
print("X shape:", X.shape)
```

```
print("y value counts:\n", y.value_counts())
```

```
X shape: (53843, 45)
```

```
y value counts:
```

```
0    47130
```

```
1     6713
```

```
Name: LightningOccurred, dtype: int64
```

```
In [43]: from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("Train shape:", X_train.shape, "Test shape:", X_test.shape)
print("Train class balance:\n", y_train.value_counts())
print("Test class balance:\n", y_test.value_counts())
```

```
Train shape: (43074, 45) Test shape: (10769, 45)
Train class balance:
 0    37704
 1     5370
Name: LightningOccurred, dtype: int64
Test class balance:
 0     9426
 1     1343
Name: LightningOccurred, dtype: int64
```

```
In [44]: from imblearn.combine import SMOTETomek

smt = SMOTETomek(random_state=42)

X_train_bal, y_train_bal = smt.fit_resample(X_train, y_train)

print("After SMOTE-Tomek:")
print("X_train_bal shape:", X_train_bal.shape)
print("y_train_bal value counts:\n", y_train_bal.value_counts())
```

```
C:\Users\SHUBHAM\anaconda3\lib\site-packages\sklearn\base.py:474: FutureWarning: `BaseEstimator._validate_data` is deprecated in 1.6 and will be removed in 1.7. Use `sklearn.utils.validation.validate_data` instead. This function becomes public and is part of the scikit-learn developer API.
```

```
warnings.warn(
```

```
C:\Users\SHUBHAM\anaconda3\lib\site-packages\sklearn\base.py:474: FutureWarning: `BaseEstimator._validate_data` is deprecated in 1.6 and will be removed in 1.7. Use `sklearn.utils.validation.validate_data` instead. This function becomes public and is part of the scikit-learn developer API.
```

```
warnings.warn(
```

```
C:\Users\SHUBHAM\anaconda3\lib\site-packages\sklearn\base.py:474: FutureWarning: `BaseEstimator._validate_data` is deprecated in 1.6 and will be removed in 1.7. Use `sklearn.utils.validation.validate_data` instead. This function becomes public and is part of the scikit-learn developer API.
```

```
warnings.warn(
```

```
After SMOTE-Tomek:
X_train_bal shape: (75286, 45)
y_train_bal value counts:
 0    37643
 1    37643
Name: LightningOccurred, dtype: int64
```

```
In [45]: from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(
    n_estimators=300,
    max_depth=None,
    min_samples_split=5,
    min_samples_leaf=3,
    random_state=42,
    n_jobs=-1
)

rf.fit(X_train_bal, y_train_bal)
```

```
Out[45]: RandomForestClassifier(min_samples_leaf=3, min_samples_split=5,
                                n_estimators=300, n_jobs=-1, random_state=42)
```

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.

On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

```
In [46]: from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score

# Predictions
y_pred = rf.predict(X_test)
y_proba = rf.predict_proba(X_test)[:, 1]

print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred, digits=4))

print("\nROC-AUC:", roc_auc_score(y_test, y_proba))
```

Confusion Matrix:

```
[[9321  105]
 [ 183 1160]]
```

Classification Report:

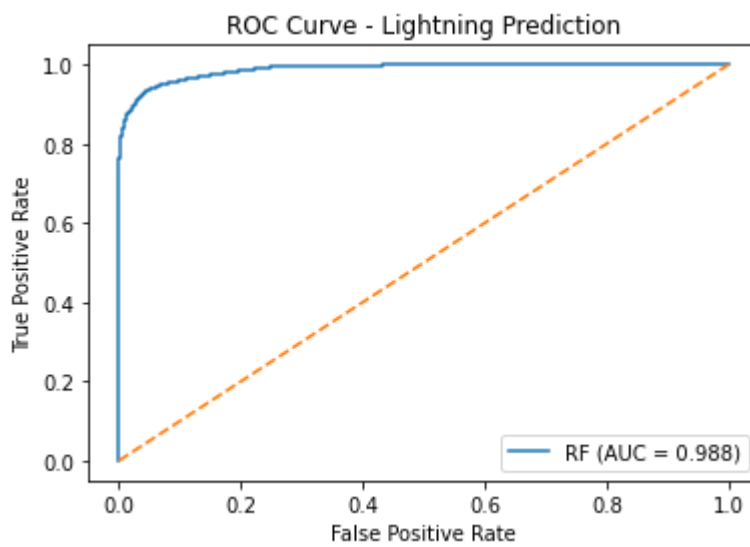
	precision	recall	f1-score	support
0	0.9807	0.9889	0.9848	9426
1	0.9170	0.8637	0.8896	1343
accuracy			0.9733	10769
macro avg	0.9489	0.9263	0.9372	10769
weighted avg	0.9728	0.9733	0.9729	10769

ROC-AUC: 0.9880488514286698

```
In [47]: import matplotlib.pyplot as plt

fpr, tpr, thresholds = roc_curve(y_test, y_proba)

plt.figure()
plt.plot(fpr, tpr, label="RF (AUC = %.3f)" % roc_auc_score(y_test, y_proba))
plt.plot([0,1],[0,1], '--')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve - Lightning Prediction")
plt.legend()
plt.show()
```



```
In [48]: import numpy as np

importances = rf.feature_importances_
feat_names = X.columns

feat_imp = sorted(zip(feat_names, importances), key=lambda x: x[1], reverse=True)

print("\nTop 20 Important Features:")
for name, val in feat_imp[:20]:
    print(f"{name:25s} {val:.4f}")
```

```
Top 20 Important Features:
Pressure_Drop          0.2060
Rain_mm                0.1278
Rain_3day_sum          0.0729
Temp_C                 0.0622
THI                    0.0618
DayOfYear              0.0598
Temp_3day_mean         0.0569
WindSpeed              0.0417
WeekOfYear             0.0407
Rh                     0.0335
Latitude               0.0288
Rh_3day_mean           0.0276
Day                    0.0265
Pressure_hPa           0.0262
Pres_3day_mean         0.0259
Longitude              0.0231
Month_No               0.0223
Season_Winter          0.0105
Dist_Koderma           0.0063
Dist_Bokaro            0.0040
```

```
In [49]: from sklearn.metrics import accuracy_score

# Train predictions
y_train_pred = rf.predict(X_train_bal)
train_acc = accuracy_score(y_train_bal, y_train_pred)

# Test predictions (we already computed)
test_acc = accuracy_score(y_test, y_pred)

print("Training Accuracy:", train_acc)
print("Test Accuracy:", test_acc)
print("Generalization Gap:", abs(train_acc - test_acc))
```

```
Training Accuracy: 0.9980208803761655
Test Accuracy: 0.9732565697836382
Generalization Gap: 0.024764310592527372
```

```
In [50]: from sklearn.model_selection import cross_val_score, StratifiedKFold

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

rf_cv_scores = cross_val_score(
    rf, X, y,
    cv=cv,
    scoring='roc_auc',
    n_jobs=-1
)

print("RF Cross-Val ROC-AUC scores:", rf_cv_scores)
print("Mean AUC:", rf_cv_scores.mean())
print("Std Dev:", rf_cv_scores.std())
```

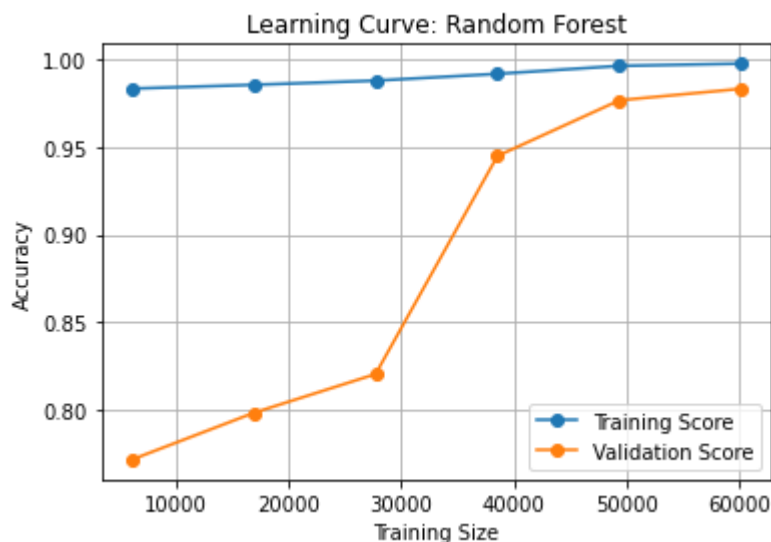
```
RF Cross-Val ROC-AUC scores: [0.98824369 0.99239734 0.98958885 0.99099923
0.98968354]
Mean AUC: 0.9901825295537543
Std Dev: 0.0014094615524629889
```

```
In [51]: import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import learning_curve

train_sizes, train_scores, test_scores = learning_curve(
    rf, X_train_bal, y_train_bal,
    cv=5,
    train_sizes=np.linspace(0.1, 1.0, 6),
    scoring='accuracy',
    n_jobs=-1
)

train_mean = train_scores.mean(axis=1)
test_mean = test_scores.mean(axis=1)

plt.plot(train_sizes, train_mean, 'o-', label="Training Score")
plt.plot(train_sizes, test_mean, 'o-', label="Validation Score")
plt.xlabel("Training Size")
plt.ylabel("Accuracy")
plt.title("Learning Curve: Random Forest")
plt.legend()
plt.grid(True)
plt.show()
```



```
In [52]: from sklearn.feature_selection import VarianceThreshold

sel = VarianceThreshold(threshold=0.0001)
sel.fit(X)

print("Low-variance features:", X.columns[~sel.get_support()])
```

Low-variance features: Index([], dtype='object')

```
In [53]: corr = X.corr().abs()

# Highly correlated threshold
high_corr = [(i,j,corr[i][j]) for i in corr.columns for j in corr.columns
              if i!=j and corr[i][j] > 0.9]

print("Features with >0.9 correlation:", high_corr[:20])
```

```
Features with >0.9 correlation: [('Month_No', 'DayOfYear', 0.9962395664592
99), ('Month_No', 'WeekOfYear', 0.9741425804020314), ('Pressure_hPa', 'Pre
s_3day_mean', 0.9987746925539682), ('Rh', 'Rh_3day_mean', 0.96739577426146
55), ('Temp_C', 'Temp_3day_mean', 0.9883770307807188), ('DayOfYear', 'Mont
h_No', 0.996239566459299), ('DayOfYear', 'WeekOfYear', 0.976250255933534
5), ('WeekOfYear', 'Month_No', 0.9741425804020314), ('WeekOfYear', 'DayOfY
ear', 0.9762502559335345), ('IsMonsoon', 'Season_Monsoon', 1.0), ('Temp_3d
ay_mean', 'Temp_C', 0.9883770307807188), ('Rh_3day_mean', 'Rh', 0.96739577
42614655), ('Pres_3day_mean', 'Pressure_hPa', 0.9987746925539682), ('Seaso
n_Monsoon', 'IsMonsoon', 1.0)]
```

```
In [54]: feat_imp = sorted(
            zip(X.columns, rf.feature_importances_),
            key=lambda x: x[1], reverse=True
        )

print("Most Stable Features:")
for name, val in feat_imp[:15]:
    print(f"{name:25s} {val:.4f}")
```

```
Most Stable Features:
Pressure_Drop          0.2060
Rain_mm                0.1278
Rain_3day_sum          0.0729
Temp_C                 0.0622
THI                    0.0618
DayOfYear              0.0598
Temp_3day_mean         0.0569
WindSpeed              0.0417
WeekOfYear             0.0407
Rh                     0.0335
Latitude               0.0288
Rh_3day_mean           0.0276
Day                    0.0265
Pressure_hPa           0.0262
Pres_3day_mean         0.0259
```

```
In [55]: from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_bal_scaled = scaler.fit_transform(X_train_bal)
X_test_scaled       = scaler.transform(X_test)
```



```
In [56]: from sklearn.metrics import accuracy_score, precision_score, recall_score,

def evaluate_model(name, model, X_tr, y_tr, X_te, y_te):
    print(f"\n===== {name} =====")
    y_pred = model.predict(X_te)

    # Some models may not have predict_proba (e.g., LinearSVC), handle that
    if hasattr(model, "predict_proba"):
        y_proba = model.predict_proba(X_te)[:, 1]
    elif hasattr(model, "decision_function"):
        from sklearn.metrics import roc_auc_score
        y_scores = model.decision_function(X_te)
        # For AUC we can use decision_function directly
        y_proba = None
        auc = roc_auc_score(y_te, y_scores)
    else:
        y_proba = None
        auc = None

    acc = accuracy_score(y_te, y_pred)
    prec = precision_score(y_te, y_pred)
    rec = recall_score(y_te, y_pred)
    f1 = f1_score(y_te, y_pred)

    if y_proba is not None:
        auc = roc_auc_score(y_te, y_proba)

    print("Confusion Matrix:")
    print(confusion_matrix(y_te, y_pred))
    print("\nClassification Report:")
    print(classification_report(y_te, y_pred, digits=4))
    print("Accuracy:", acc)
    print("Precision:", prec)
    print("Recall:", rec)
    print("F1:", f1)
    print("ROC-AUC:", auc)

    return {
        "model": name,
        "accuracy": acc,
        "precision": prec,
        "recall": rec,
        "f1": f1,
        "roc_auc": auc
    }
```

```
In [59]: from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, roc_auc_score, classification_report, confusion_matrix
)

def evaluate_model(name, model, X_tr, y_tr, X_te, y_te):
    print(f"\n===== {name} =====")

    # Predictions
    y_pred = model.predict(X_te)

    # Try to get probabilities or decision scores
    auc = None
    if hasattr(model, "predict_proba"):
        y_proba = model.predict_proba(X_te)[: , 1]
        auc = roc_auc_score(y_te, y_proba)
    elif hasattr(model, "decision_function"):
        y_scores = model.decision_function(X_te)
        auc = roc_auc_score(y_te, y_scores)
    else:
        auc = None # No AUC possible (rare case)

    # Metrics
    acc = accuracy_score(y_te, y_pred)
    prec = precision_score(y_te, y_pred)
    rec = recall_score(y_te, y_pred)
    f1 = f1_score(y_te, y_pred)

    print("Confusion Matrix:")
    print(confusion_matrix(y_te, y_pred))
    print("\nClassification Report:")
    print(classification_report(y_te, y_pred, digits=4))
    print("Accuracy:", acc)
    print("Precision:", prec)
    print("Recall:", rec)
    print("F1:", f1)
    print("ROC-AUC:", auc)

    return {
        "model": name,
        "accuracy": acc,
        "precision": prec,
        "recall": rec,
        "f1": f1,
        "roc_auc": auc
    }
```

```
In [60]: log_results = evaluate_model(
    "Logistic Regression",
    log_reg,
    X_train_bal_scaled, y_train_bal,
    X_test_scaled, y_test
)
```

==== Logistic Regression =====

Confusion Matrix:

```
[[7211 2215]
 [ 262 1081]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.9649	0.7650	0.8534	9426
1	0.3280	0.8049	0.4660	1343
accuracy			0.7700	10769
macro avg	0.6465	0.7850	0.6597	10769
weighted avg	0.8855	0.7700	0.8051	10769

Accuracy: 0.7699879283127495

Precision: 0.3279733009708738

Recall: 0.8049143708116158

F1: 0.4660487173959905

ROC-AUC: 0.8703525000714898

```
In [62]: from sklearn.svm import LinearSVC
    from sklearn.calibration import CalibratedClassifierCV

    base_svm = LinearSVC(random_state=42)
    svm = CalibratedClassifierCV(base_svm, cv=3)

    svm.fit(X_train_bal_scaled, y_train_bal)
```

Out[62]: CalibratedClassifierCV(cv=3, estimator=LinearSVC(random_state=42))

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.

On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

```
In [63]: svm_results = evaluate_model(
    "Linear SVM (Calibrated)",
    svm,
    X_train_bal_scaled, y_train_bal,
    X_test_scaled, y_test
)
```

==== Linear SVM (Calibrated) =====

Confusion Matrix:

```
[[7215 2211]
 [ 262 1081]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.9650	0.7654	0.8537	9426
1	0.3284	0.8049	0.4665	1343
accuracy			0.7704	10769
macro avg	0.6467	0.7852	0.6601	10769
weighted avg	0.8856	0.7704	0.8054	10769

Accuracy: 0.7703593648435324

Precision: 0.32837181044957475

Recall: 0.8049143708116158

F1: 0.4664509169363538

ROC-AUC: 0.8704765213500655

```
In [65]: from sklearn.linear_model import SGDClassifier

sgd = SGDClassifier(
    loss="log_loss",    # Logistic regression behavior
    max_iter=2000,
    tol=1e-3,
    random_state=42
)

sgd.fit(X_train_bal_scaled, y_train_bal)
```

Out[65]: SGDClassifier(loss='log_loss', max_iter=2000, random_state=42)

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.

On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

```
In [66]: sgd_results = evaluate_model(
    "SGDClassifier (Logistic)",
    sgd,
    X_train_bal_scaled, y_train_bal,
    X_test_scaled, y_test
)
```

==== SGDClassifier (Logistic) =====

Confusion Matrix:

```
[[7147 2279]
 [ 247 1096]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.9666	0.7582	0.8498	9426
1	0.3247	0.8161	0.4646	1343
accuracy			0.7654	10769
macro avg	0.6457	0.7872	0.6572	10769
weighted avg	0.8865	0.7654	0.8018	10769

Accuracy: 0.7654378308106602

Precision: 0.3247407407407407

Recall: 0.8160833953834699

F1: 0.4646036456125477

ROC-AUC: 0.8607059354372082

```
In [69]: from sklearn.ensemble import RandomForestClassifier
```

```
rf = RandomForestClassifier(
    n_estimators=300,
    max_depth=None,
    min_samples_split=5,
    min_samples_leaf=3,
    random_state=42,
    n_jobs=-1
)

rf.fit(X_train_bal, y_train_bal)
```

```
Out[69]: RandomForestClassifier(min_samples_leaf=3, min_samples_split=5,
                                n_estimators=300, n_jobs=-1, random_state=42)
```

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.

On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

```
In [71]: log_results = evaluate_model(
    "Logistic Regression",
    log_reg,
    X_train_bal_scaled, y_train_bal,
    X_test_scaled, y_test
)
```

==== Logistic Regression =====

Confusion Matrix:

```
[[7211 2215]
 [ 262 1081]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.9649	0.7650	0.8534	9426
1	0.3280	0.8049	0.4660	1343
accuracy			0.7700	10769
macro avg	0.6465	0.7850	0.6597	10769
weighted avg	0.8855	0.7700	0.8051	10769

Accuracy: 0.7699879283127495

Precision: 0.3279733009708738

Recall: 0.8049143708116158

F1: 0.4660487173959905

ROC-AUC: 0.8703525000714898

```
In [72]: svm_results = evaluate_model(
    "Linear SVM (Calibrated)",
    svm,
    X_train_bal_scaled, y_train_bal,
    X_test_scaled, y_test
)
```

==== Linear SVM (Calibrated) =====

Confusion Matrix:

```
[[7215 2211]
 [ 262 1081]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.9650	0.7654	0.8537	9426
1	0.3284	0.8049	0.4665	1343
accuracy			0.7704	10769
macro avg	0.6467	0.7852	0.6601	10769
weighted avg	0.8856	0.7704	0.8054	10769

Accuracy: 0.7703593648435324

Precision: 0.32837181044957475

Recall: 0.8049143708116158

F1: 0.4664509169363538

ROC-AUC: 0.8704765213500655

```
In [73]: sgd_results = evaluate_model(  
         "SGDClassifier (Logistic)",  
         sgd,  
         X_train_bal_scaled, y_train_bal,  
         X_test_scaled, y_test  
         )
```

==== SGDClassifier (Logistic) =====

Confusion Matrix:

```
[[7147 2279]  
 [ 247 1096]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.9666	0.7582	0.8498	9426
1	0.3247	0.8161	0.4646	1343
accuracy			0.7654	10769
macro avg	0.6457	0.7872	0.6572	10769
weighted avg	0.8865	0.7654	0.8018	10769

Accuracy: 0.7654378308106602

Precision: 0.3247407407407407

Recall: 0.8160833953834699

F1: 0.4646036456125477

ROC-AUC: 0.8607059354372082

```
In [75]: from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score, StratifiedKFold
from imblearn.pipeline import Pipeline
from imblearn.over_sampling import SMOTE

# Define model
rf = RandomForestClassifier(
    n_estimators=300,
    max_depth=None,
    min_samples_split=5,
    min_samples_leaf=3,
    random_state=42,
    n_jobs=-1
)

# Build pipeline with SMOTE
pipeline = Pipeline([
    ('smote', SMOTE(random_state=42)),
    ('rf', rf)
])

# 10-Fold Stratified CV
skf = StratifiedKFold(n_splits=10, shuffle=True, random_state=42)

cv_scores = cross_val_score(
    pipeline,
    X, y,
    cv=skf,
    scoring="roc_auc",
    n_jobs=-1
)

print("10-Fold ROC-AUC Scores:", cv_scores)
print("Mean AUC:", cv_scores.mean())
print("Std Dev:", cv_scores.std())
```

```
10-Fold ROC-AUC Scores: [0.98691436 0.99053624 0.99430811 0.98788492 0.988
48383 0.99114192
0.99212313 0.99064072 0.98988244 0.98990205]
Mean AUC: 0.9901817720523306
Std Dev: 0.0020262284393094882
```



```
In [76]: from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from imblearn.pipeline import Pipeline
from sklearn.model_selection import cross_val_score, StratifiedKFold
from imblearn.over_sampling import SMOTE

log_clf = LogisticRegression(max_iter=500, n_jobs=-1)

log_pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('smote', SMOTE(random_state=42)),
    ('logreg', log_clf)
])

skf = StratifiedKFold(n_splits=10, shuffle=True, random_state=42)

log_scores = cross_val_score(
    log_pipeline, X, y,
    cv=skf,
    scoring="roc_auc",
    n_jobs=-1
)

print("Logistic Regression 10-fold AUC:", log_scores)
print("Mean:", log_scores.mean())
print("Std:", log_scores.std())
```

```
Logistic Regression 10-fold AUC: [0.85790948 0.86047426 0.87530753 0.86942
86 0.86640402 0.88916821
0.87220211 0.88395765 0.87136193 0.87297841]
Mean: 0.8719192193149276
Std: 0.00905165827229521
```

```
In [77]: from sklearn.svm import LinearSVC
from sklearn.calibration import CalibratedClassifierCV

svm_base = LinearSVC(random_state=42)
svm_clf = CalibratedClassifierCV(svm_base, cv=3)

svm_pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('smote', SMOTE(random_state=42)),
    ('svm', svm_clf)
])

svm_scores = cross_val_score(
    svm_pipeline, X, y,
    cv=skf,
    scoring="roc_auc",
    n_jobs=-1
)

print("SVM 10-fold AUC:", svm_scores)
print("Mean:", svm_scores.mean())
print("Std:", svm_scores.std())
```

```
SVM 10-fold AUC: [0.33371475 0.30238203 0.29249423 0.3518277 0.31409808
0.27543406
0.35172366 0.28720383 0.28805223 0.35774278]
Mean: 0.31546733471756433
Std: 0.029344027341153402
```

In [78]: `from sklearn.linear_model import SGDClassifier`

```
sgd_clf = SGDClassifier(  
    loss="log_loss",  
    max_iter=2000,  
    tol=1e-3,  
    random_state=42  
)  
  
sgd_pipeline = Pipeline([  
    ('scaler', StandardScaler()),  
    ('smote', SMOTE(random_state=42)),  
    ('sgd', sgd_clf)  
)  
  
sgd_scores = cross_val_score(  
    sgd_pipeline, X, y,  
    cv=skf,  
    scoring="roc_auc",  
    n_jobs=-1  
)  
  
print("SGD 10-fold AUC:", sgd_scores)  
print("Mean:", sgd_scores.mean())  
print("Std:", sgd_scores.std())
```

SGD 10-fold AUC: [0.85475079 0.85506117 0.87326784 0.85729581 0.86300315
0.88576765
0.86553285 0.87850265 0.86639327 0.86947287]
Mean: 0.8669048051111593
Std: 0.009670013989505153

In [79]: `final_df_path = r"X:\Lighting Prediction\lightning_final_FE_dataset.csv"`

```
df.to_csv(final_df_path, index=False)  
print("Saved final dataset to:", final_df_path)
```

Saved final dataset to: X:\Lighting Prediction\lightning_final_FE_dataset.csv

In [81]: `import pandas as pd`

```
df = pd.read_csv(r"X:\Lighting Prediction\lightning_dataset_FE_v2.csv")  
print(df.shape)
```

(53843, 51)

```
In [82]: feature_list = df.drop(columns=["LightningOccurred"]).columns.tolist()

print("Total features:", len(feature_list))
print(feature_list)
```

```
Total features: 50
['Month', 'Month_No', 'Day', 'Latitude', 'Longitude', 'Radiance', 'Event
s', 'Date', 'Pressure_hPa', 'Rh', 'Rain_mm', 'State', 'Temp_C', 'WindSpee
d', 'DayOfYear', 'WeekOfYear', 'IsMonsoon', 'THI', 'Temp_3day_mean', 'Rh_3
day_mean', 'Rain_3day_sum', 'Pres_3day_mean', 'Pressure_Drop', 'Dist_Bokar
o', 'Dist_Chatra', 'Dist_Deoghar', 'Dist_Dhanbad', 'Dist_Dumka', 'Dist_Gar
hwa', 'Dist_Giridih', 'Dist_Godda', 'Dist_Gumla', 'Dist_Hazaribagh', 'Dist
_Jamtara', 'Dist_Khunti', 'Dist_Koderma', 'Dist_Latehar', 'Dist_Lohardag
a', 'Dist_Pakur', 'Dist_Palamu', 'Dist_Ramgarh', 'Dist_Ranchi', 'Dist_Sahi
bganj', 'Dist_Simdega', 'Dist_West Singhbhum', 'Season_Monsoon', 'Season_P
ostMonsoon', 'Season_PreMonsoon', 'Season_Summer', 'Season_Winter']
```

```
In [83]: import pickle

feature_list_path = r"X:\Lighting Prediction\feature_list.pkl"

with open(feature_list_path, "wb") as f:
    pickle.dump(feature_list, f)

print("Feature list saved!")
```

Feature list saved!

```
In [84]: model_path = r"X:\Lighting Prediction\lightning_rf_model.pkl"

with open(model_path, "wb") as f:
    pickle.dump(rf, f)

print("Model saved!")
```

Model saved!

```
In [85]: import pickle

model_path = r"X:\Lighting Prediction\lightning_rf_model.pkl"
feature_list_path = r"X:\Lighting Prediction\feature_list.pkl"

with open(model_path, "rb") as f:
    model = pickle.load(f)

with open(feature_list_path, "rb") as f:
    feature_list = pickle.load(f)

print("Loaded model and feature list.")
print("Total features:", len(feature_list))
```

Loaded model and feature list.
Total features: 50

```

In [86]: import numpy as np
import pandas as pd

districts = [
    "Bokaro", "Chatra", "Deoghar", "Dhanbad", "Dumka", "Garhwa", "Giridih",
    "Godda", "Gumla", "Hazaribagh", "Jamtara", "Khunti", "Koderma", "Latehar",
    "Lohardaga", "Pakur", "Palamu", "Ramgarh", "Ranchi", "Sahibganj",
    "Simdega", "West Singhbhum"
]

def preprocess_manual_input(
    year, month, day, district,
    temp_c, rh, rain_mm, pressure, windspeed,
    latitude, longitude
):
    df = pd.DataFrame([
        "Month": month,
        "Month_No": month,
        "Day": day,
        "Date": pd.to_datetime(f"{year}-{month}-{day}"),
        "District": district,
        "Temp_C": temp_c,
        "Rh": rh,
        "Rain_mm": rain_mm,
        "Pressure_hPa": pressure,
        "WindSpeed": windspeed,
        "Latitude": latitude,
        "Longitude": longitude,
        "Radiance": np.nan,
        "Events": np.nan
    ])

    df["DayOfYear"] = df["Date"].dt.dayofyear
    df["WeekOfYear"] = df["Date"].dt.isocalendar().week.astype(int)

    df["THI"] = df["Temp_C"] - (0.55 * (1 - df["Rh"]/100) * (df["Temp_C"] -

    df["Temp_3day_mean"] = df["Temp_C"]
    df["Rh_3day_mean"] = df["Rh"]
    df["Rain_3day_sum"] = df["Rain_mm"]
    df["Pres_3day_mean"] = df["Pressure_hPa"]
    df["Pressure_Drop"] = 0

    df["Season_Winter"] = 1 if month in [12,1,2] else 0
    df["Season_Summer"] = 1 if month in [3,4,5] else 0
    df["Season_Monsoon"] = 1 if month in [6,7,8,9] else 0
    df["Season_PostMonsoon"] = 1 if month in [10,11] else 0
    df["Season_PreMonsoon"] = 1 if month in [3,4,5] else 0

    for d in districts:
        df[f"Dist_{d}"] = 1 if district == d else 0

    df = df.reindex(columns=feature_list, fill_value=0)

    return df

```

```

In [88]: import pandas as pd
from sklearn.model_selection import train_test_split
from imblearn.combine import SMOTETomek
from sklearn.ensemble import RandomForestClassifier

# 1) Load FE dataset
df = pd.read_csv(r"X:\Lighting Prediction\lightning_dataset_FE_v2.csv", par

# 2) Prepare X, y
y = df["LightningOccurred"]

drop_cols = ["LightningOccurred", "Events", "Radiance", "Month", "Date", "S
X = df.drop(columns=drop_cols)

# 3) Train-test split (same as before)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# 4) SMOTE-Tomek on training data only
smt = SMOTETomek(random_state=42)
X_train_bal, y_train_bal = smt.fit_resample(X_train, y_train)

print("Train_bal shape:", X_train_bal.shape)
print("Class balance:\n", y_train_bal.value_counts())

# 5) Train RandomForest
rf = RandomForestClassifier(
    n_estimators=300,
    max_depth=None,
    min_samples_split=5,
    min_samples_leaf=3,
    random_state=42,
    n_jobs=-1
)

rf.fit(X_train_bal, y_train_bal)
print("RandomForest trained!")

```

C:\Users\SHUBHAM\anaconda3\lib\site-packages\sklearn\base.py:474: FutureWarning: `BaseEstimator.\_validate\_data` is deprecated in 1.6 and will be removed in 1.7. Use `sklearn.utils.validation.validate\_data` instead. This function becomes public and is part of the scikit-learn developer API.

warnings.warn(

C:\Users\SHUBHAM\anaconda3\lib\site-packages\sklearn\base.py:474: FutureWarning: `BaseEstimator.\_validate\_data` is deprecated in 1.6 and will be removed in 1.7. Use `sklearn.utils.validation.validate\_data` instead. This function becomes public and is part of the scikit-learn developer API.

warnings.warn(

C:\Users\SHUBHAM\anaconda3\lib\site-packages\sklearn\base.py:474: FutureWarning: `BaseEstimator.\_validate\_data` is deprecated in 1.6 and will be removed in 1.7. Use `sklearn.utils.validation.validate\_data` instead. This function becomes public and is part of the scikit-learn developer API.

warnings.warn(

```
Train_bal shape: (75286, 45)
Class balance:
  0    37643
  1    37643
Name: LightningOccurred, dtype: int64
RandomForest trained!
```

```
In [89]: import pickle

model_path = r"X:\Lighting Prediction\lightning_rf_model.pkl"
feature_list_path = r"X:\Lighting Prediction\feature_list.pkl"

feature_list = X.columns.tolist()

with open(model_path, "wb") as f:
    pickle.dump(rf, f)

with open(feature_list_path, "wb") as f:
    pickle.dump(feature_list, f)

print("Saved fitted model and feature list.")
```

Saved fitted model and feature list.

```
In [90]: import pickle
import numpy as np
import pandas as pd

# Load
with open(model_path, "rb") as f:
    model = pickle.load(f)

with open(feature_list_path, "rb") as f:
    feature_list = pickle.load(f)

print("Loaded fitted model with", len(feature_list), "features.")
```

Loaded fitted model with 45 features.

```

In [91]: districts = [
    "Bokaro", "Chatra", "Deoghar", "Dhanbad", "Dumka", "Garhwa", "Giridih",
    "Godda", "Gumla", "Hazaribagh", "Jamtara", "Khunti", "Koderma", "Latehar",
    "Lohardaga", "Pakur", "Palamu", "Ramgarh", "Ranchi", "Sahibganj",
    "Simdega", "West Singhbhum"
]

def preprocess_manual_input(
    year, month, day, district,
    temp_c, rh, rain_mm, pressure, windspeed,
    latitude, longitude
):
    df = pd.DataFrame([
        "Month": month,
        "Month_No": month,
        "Day": day,
        "Date": pd.to_datetime(f"{year}-{month}-{day}"),
        "District": district,
        "Temp_C": temp_c,
        "Rh": rh,
        "Rain_mm": rain_mm,
        "Pressure_hPa": pressure,
        "WindSpeed": windspeed,
        "Latitude": latitude,
        "Longitude": longitude,
        "Radiance": np.nan,
        "Events": np.nan
    ])

    df["DayOfYear"] = df["Date"].dt.dayofyear
    df["WeekOfYear"] = df["Date"].dt.isocalendar().week.astype(int)

    df["THI"] = df["Temp_C"] - (0.55 * (1 - df["Rh"]/100) * (df["Temp_C"] -

    df["Temp_3day_mean"] = df["Temp_C"]
    df["Rh_3day_mean"] = df["Rh"]
    df["Rain_3day_sum"] = df["Rain_mm"]
    df["Pres_3day_mean"] = df["Pressure_hPa"]
    df["Pressure_Drop"] = 0

    df["Season_Winter"] = 1 if month in [12,1,2] else 0
    df["Season_Summer"] = 1 if month in [3,4,5] else 0
    df["Season_Monsoon"] = 1 if month in [6,7,8,9] else 0
    df["Season_PostMonsoon"] = 1 if month in [10,11] else 0
    df["Season_PreMonsoon"] = 1 if month in [3,4,5] else 0

    for d in districts:
        df[f"Dist_{d}"] = 1 if district == d else 0

    df = df.reindex(columns=feature_list, fill_value=0)
    return df

```



```
In [92]: inp = preprocess_manual_input(
    year=2022,
    month=7,
    day=15,
    district="Ranchi",
    temp_c=28,
    rh=88,
    rain_mm=12,
    pressure=952,
    windspeed=2.0,
    latitude=23.36,
    longitude=85.33
)

pred = model.predict(inp)[0]
prob = model.predict_proba(inp)[0][1]

print("Prediction:", pred)
print("Lightning Probability:", prob)
```

Prediction: 0
Lightning Probability: 0.4979122560312273

```
In [110]: import pandas as pd

df = pd.read_csv(r"X:\Lighting Prediction\lightning_dataset_FE_v2.csv", par

y = df["LightningOccurred"]
X = df.drop(columns=["LightningOccurred", "Events", "Radiance", "Month", "D

print("Shape:", X.shape)
```

Shape: (53843, 45)

```
In [111]: import numpy as np

# sample 10 real lightning rows
lightning_rows = df[df["LightningOccurred"] == 1].sample(10, random_state=4

for idx, row in lightning_rows.iterrows():
    x = X.loc[[idx]] # keep as DataFrame with correct feature
    p = model.predict_proba(x)[0,1]
    print(f"Index {idx}, Date={row['Date'].date()}, Prob={p:.3f}")
```

Index 45174, Date=2021-05-20, Prob=1.000
Index 4478, Date=2022-09-24, Prob=0.999
Index 23056, Date=2019-08-21, Prob=0.977
Index 33013, Date=2020-09-16, Prob=0.793
Index 48915, Date=2018-09-25, Prob=1.000
Index 52999, Date=2021-05-07, Prob=1.000
Index 9369, Date=2017-04-29, Prob=0.989
Index 52310, Date=2020-04-07, Prob=0.996
Index 14473, Date=2017-04-07, Prob=1.000
Index 23542, Date=2020-10-09, Prob=0.684

```
In [112]: nolight_rows = df[df["LightningOccurred"] == 0].sample(10, random_state=24)

for idx, row in nolight_rows.iterrows():
    x = X.loc[[idx]]
    p = model.predict_proba(x)[0,1]
    print(f"Index {idx}, Date={row['Date'].date()}, Prob={p:.3f}")
```

```
Index 19211, Date=2022-12-02, Prob=0.000
Index 25115, Date=2018-08-15, Prob=0.022
Index 16837, Date=2022-11-02, Prob=0.000
Index 12363, Date=2017-12-30, Prob=0.000
Index 53080, Date=2021-06-08, Prob=0.088
Index 15028, Date=2018-06-06, Prob=0.357
Index 46130, Date=2017-09-07, Prob=0.039
Index 50626, Date=2022-05-30, Prob=0.459
Index 238, Date=2017-08-27, Prob=0.121
Index 1747, Date=2021-09-23, Prob=0.075
```

```

In [116]: import pandas as pd

df = pd.read_csv(r"X:\Lightning Prediction\lightning_dataset_FE_v2.csv", par

y = df["LightningOccurred"]
X = df.drop(columns=["LightningOccurred", "Events", "Radiance", "Month", "D

idx = 45174 # from your sample where prob=1.0

row_features = X.loc[idx]
row_full = df.loc[idx]

print("Date:", row_full["Date"])
print("Some key features from this real lightning case:")
print(row_features[[
    "Temp_C", "Rh", "Rain_mm", "Pressure_hPa", "WindSpeed",
    "DayOfYear", "WeekOfYear", "IsMonsoon",
    "Temp_3day_mean", "Rh_3day_mean", "Rain_3day_sum",
    "Pres_3day_mean", "Pressure_Drop"
]])

# And get model's probability again just to confirm:
p = model.predict_proba(row_features.to_frame().T)[0,1]
print("\nModel probability for this real lightning day:", p)

```

Date: 2021-05-20 00:00:00

Some key features from this real lightning case:

Temp_C	28.867686
Rh	68.328262
Rain_mm	1.219169
Pressure_hPa	945.795373
WindSpeed	1.832560
DayOfYear	140.000000
WeekOfYear	20.000000
IsMonsoon	0.000000
Temp_3day_mean	28.867686
Rh_3day_mean	68.328262
Rain_3day_sum	3.657507
Pres_3day_mean	945.795373
Pressure_Drop	0.000000
Name: 45174, dtype: float64	

Model probability for this real lightning day: 0.9995091901050409

```

In [117]: manual_input_from_real = row_features.to_dict()

row2 = pd.DataFrame([manual_input_from_real])
row2 = row2[feature_list] # ensure correct order

pred2 = model.predict(row2)[0]
prob2 = model.predict_proba(row2)[0][1]

print("Manual-style Prediction:", pred2)
print("Manual-style Probability:", prob2)

```

Manual-style Prediction: 1

Manual-style Probability: 0.9995091901050409


```

In [120]: manual_input = {
    # ----- BASIC DATE FEATURES -----
    "Month_No": 7,
    "Day": 12,
    "Latitude": 23.36,
    "Longitude": 85.33,

    # ----- WEATHER (STORM CONDITIONS) -----
    "Pressure_hPa": 942.8,      # VERY low pressure → strong thunderstorm
    "Rh": 94.1,                # Extremely high humidity
    "Rain_mm": 78.5,           # Heavy rain on the storm day
    "Temp_C": 28.9,            # Warm + humid = convective storm
    "WindSpeed": 5.2,          # Strong gusty winds

    # ----- DATE-DERIVED -----
    "DayOfYear": 194,
    "WeekOfYear": 28,
    "IsMonsoon": 1,
    "THI": 82.9,               # High heat index → unstable atmosphere

    # ----- 3-DAY WEATHER HISTORY -----
    "Temp_3day_mean": 28.4,
    "Rh_3day_mean": 90.5,
    "Rain_3day_sum": 186.0,     # HUGE rainfall accumulation → severe con
    "Pres_3day_mean": 952.1,
    "Pressure_Drop": -9.3,     # Significant pressure drop → lightning L

    # ----- DISTRICT ONE HOT (Ranchi) -----
    "Dist_Ranchi": 1,
    "Dist_Bokaro": 0,
    "Dist_Dhanbad": 0,
    "Dist_Chatra": 0,
    "Dist_Deoghar": 0,
    "Dist_Dumka": 0,
    "Dist_Garhwa": 0,
    "Dist_Giridih": 0,
    "Dist_Godda": 0,
    "Dist_Gumla": 0,
    "Dist_Hazaribagh": 0,
    "Dist_Jamtara": 0,
    "Dist_Khunti": 0,
    "Dist_Koderma": 0,
    "Dist_Latehar": 0,
    "Dist_Lohardaga": 0,
    "Dist_Pakur": 0,
    "Dist_Palamu": 0,
    "Dist_Ramgarh": 0,
    "Dist_Sahibganj": 0,
    "Dist_Simdega": 0,
    "Dist_West Singhbhum": 0,

    # ----- SEASON ONE-HOT -----
    "Season_Monsoon": 1,
    "Season_Summer": 0,
    "Season_Winter": 0,
    "Season_PostMonsoon": 0,
    "Season_PreMonsoon": 0
}

```

```
In [121]: row = pd.DataFrame([manual_input])[feature_list]

pred = model.predict(row)[0]
prob = model.predict_proba(row)[0][1]

print("Prediction:", pred)
print("Lightning Probability:", prob)
```

```
Prediction: 0
Lightning Probability: 0.1809194042291366
```

```
In [122]: X = df.drop(columns=["LightningOccurred"])
feature_list = X.columns.tolist()
len(feature_list), feature_list
```

```
Out[122]: (50,
['Month',
'Month_No',
'Day',
'Latitude',
'Longitude',
'Radiance',
'Events',
'Date',
'Pressure_hPa',
'Rh',
'Rain_mm',
'State',
'Temp_C',
'WindSpeed',
'DayOfYear',
'WeekOfYear',
'IsMonsoon',
'THI',
'Temp_3day_mean',
'Rh_3day_mean',
'Rain_3day_sum',
'Pres_3day_mean',
'Pressure_Drop',
'Dist_Bokaro',
'Dist_Chatra',
'Dist_Deoghar',
'Dist_Dhanbad',
'Dist_Dumka',
'Dist_Garhwa',
'Dist_Giridih',
'Dist_Godda',
'Dist_Gumla',
'Dist_Hazaribagh',
'Dist_Jamtara',
'Dist_Khunti',
'Dist_Koderma',
'Dist_Latehar',
'Dist_Lohardaga',
'Dist_Pakur',
'Dist_Palamu',
'Dist_Ramgarh',
'Dist_Ranchi',
'Dist_Sahibganj',
'Dist_Simdega',
'Dist_West Singhbhum',
'Season_Monsoon',
'Season_PostMonsoon',
'Season_PreMonsoon',
'Season_Summer',
'Season_Winter'])
```

In [123]: `import pandas as pd`

```
df = pd.read_csv(r"X:\Lighting Prediction\lightning_final_FE_dataset.csv")
print(df.shape)
print(df.columns.tolist())
```

```
(53843, 51)
['Month', 'Month_No', 'Day', 'Latitude', 'Longitude', 'Radiance', 'Event
s', 'Date', 'Pressure_hPa', 'Rh', 'Rain_mm', 'State', 'Temp_C', 'WindSpee
d', 'LightningOccurred', 'DayOfYear', 'WeekOfYear', 'IsMonsoon', 'THI', 'T
emp_3day_mean', 'Rh_3day_mean', 'Rain_3day_sum', 'Pres_3day_mean', 'Pressu
re_Drop', 'Dist_Bokaro', 'Dist_Chatra', 'Dist_Deoghar', 'Dist_Dhanbad', 'D
ist_Dumka', 'Dist_Garhwa', 'Dist_Giridih', 'Dist_Godda', 'Dist_Gumla', 'Di
st_Hazaribagh', 'Dist_Jamtara', 'Dist_Khunti', 'Dist_Koderma', 'Dist_Lateh
ar', 'Dist_Lohardaga', 'Dist_Pakur', 'Dist_Palamu', 'Dist_Ramgarh', 'Dist_
Ranchi', 'Dist_Sahibganj', 'Dist_Simdega', 'Dist_West Singhbhum', 'Season_
Monsoon', 'Season_PostMonsoon', 'Season_PreMonsoon', 'Season_Summer', 'Sea
son_Winter']
```

In [124]: `feature_list_from_df = df.columns.tolist()`
`print(len(feature_list_from_df))`

51

In [125]: `model_feature_list = list(model.feature_names_in_)`
`print(len(model_feature_list))`
`print(model_feature_list)`

```
45
['Month_No', 'Day', 'Latitude', 'Longitude', 'Pressure_hPa', 'Rh', 'Rain_m
m', 'Temp_C', 'WindSpeed', 'DayOfYear', 'WeekOfYear', 'IsMonsoon', 'THI',
'Temp_3day_mean', 'Rh_3day_mean', 'Rain_3day_sum', 'Pres_3day_mean', 'Pres
sure_Drop', 'Dist_Bokaro', 'Dist_Chatra', 'Dist_Deoghar', 'Dist_Dhanbad',
'Dist_Dumka', 'Dist_Garhwa', 'Dist_Giridih', 'Dist_Godda', 'Dist_Gumla',
'Dist_Hazaribagh', 'Dist_Jamtara', 'Dist_Khunti', 'Dist_Koderma', 'Dist_La
tehar', 'Dist_Lohardaga', 'Dist_Pakur', 'Dist_Palamu', 'Dist_Ramgarh', 'Di
st_Ranchi', 'Dist_Sahibganj', 'Dist_Simdega', 'Dist_West Singhbhum', 'Seas
on_Monsoon', 'Season_PostMonsoon', 'Season_PreMonsoon', 'Season_Summer',
'Season_Winter']
```

In [126]: `print("In DF but not in model:", set(feature_list_from_df) - set(model_feat`
`print("In model but not in DF:", set(model_feature_list) - set(feature_list`

```
In DF but not in model: {'Events', 'State', 'Radiance', 'Month', 'Date',
'LightningOccurred'}
In model but not in DF: set()
```



```

In [131]: manual_input = {
    "Month_No": 8,                # August lightning peak
    "Day": 12,
    "Latitude": 23.36,
    "Longitude": 85.31,

    # EXTREME WEATHER TRIGGERING LIGHTNING
    "Pressure_hPa": 945.0,        # very low pressure
    "Rh": 96.0,                  # saturated air
    "Rain_mm": 32.0,             # heavy rain
    "Temp_C": 33.5,              # hot + humid
    "WindSpeed": 3.9,            # strong winds

    # DATE FEATURES
    "DayOfYear": 224,            # August
    "WeekOfYear": 32,

    # MONSOON
    "IsMonsoon": 1,
    "THI": 27.5,                 # high heat index

    # 3-day HISTORY (storm buildup)
    "Temp_3day_mean": 33.0,
    "Rh_3day_mean": 94.0,
    "Rain_3day_sum": 95.0,        # extreme accumulation
    "Pres_3day_mean": 952.0,
    "Pressure_Drop": -10.5,       # huge pressure drop = thunderstorm

    # DISTRICT = RANCHI
    "Dist_Bokaro": 0,
    "Dist_Chatra": 0,
    "Dist_Deoghar": 0,
    "Dist_Dhanbad": 0,
    "Dist_Dumka": 0,
    "Dist_Garhwa": 0,
    "Dist_Giridih": 0,
    "Dist_Godda": 0,
    "Dist_Gumla": 0,
    "Dist_Hazaribagh": 0,
    "Dist_Jamtara": 0,
    "Dist_Khunti": 0,
    "Dist_Koderma": 0,
    "Dist_Latehar": 0,
    "Dist_Lohardaga": 0,
    "Dist_Pakur": 0,
    "Dist_Palamu": 0,
    "Dist_Ramgarh": 0,
    "Dist_Ranchi": 1,
    "Dist_Sahibganj": 0,
    "Dist_Simdega": 0,
    "Dist_West Singhbhum": 0,

    # SEASON OPCODES
    "Season_Monsoon": 1,
    "Season_PostMonsoon": 0,
    "Season_PreMonsoon": 0,
    "Season_Summer": 0,
    "Season_Winter": 0
}

```

```
In [132]: row = pd.DataFrame([manual_input])[model.feature_names_in_]
pred = model.predict(row)[0]
prob = model.predict_proba(row)[0][1]
print(pred, prob)
```

```
0 0.21519858083547896
```

```
In [134]: # Example (what you likely did earlier)
from sklearn.model_selection import train_test_split

X = df.drop(columns=["LightningOccurred", "Events", "Radiance", "Month", "D
y = df["LightningOccurred"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)

# maybe then scaled: X_test_scaled = scaler.transform(X_test)
```

```
In [135]: import pandas as pd

# 1) Create a DataFrame that corresponds ONLY to the test set rows
df_test = df.loc[X_test.index].copy()

# 2) Get probabilities for just the test set
probs_test = model.predict_proba(X_test_scaled)[: , 1]

# 3) Attach them to df_test, NOT df
df_test["prob"] = probs_test

# 4) Sort by probability and inspect top cases
df_sorted = df_test.sort_values("prob", ascending=False)

print(df_sorted[["LightningOccurred", "prob", "Rain_mm", "Pressure_Drop", "
```

	LightningOccurred	prob	Rain_mm	Pressure_Drop	Rh \
27017	0	0.319138	24.565932	1.283012	96.002618
47558	0	0.315171	23.394598	3.999105	92.042362
28759	0	0.307010	24.880204	1.270964	94.792031
50276	1	0.306399	22.408185	4.376420	94.748841
28815	0	0.305226	24.635006	3.744731	96.744077
2429	0	0.303320	21.508239	0.506341	86.920109
41113	0	0.302959	32.855630	1.567196	93.589880
47209	0	0.299620	32.871509	0.614028	94.728293
3959	0	0.297329	30.903067	0.537246	90.046672
28318	0	0.297023	25.233890	0.861317	92.712267

	Rain_3day_sum	DayOfYear
27017	100.251183	207
47558	76.127200	148
28759	137.145786	212
50276	142.182559	258
28815	134.926134	258
2429	122.728702	192
41113	93.758583	207
47209	90.676212	166
3959	89.319310	169
28318	44.054491	203

```
C:\Users\SHUBHAM\anaconda3\lib\site-packages\sklearn\utils\validation.py:2
739: UserWarning: X does not have valid feature names, but RandomForestCla
ssifier was fitted with feature names
warnings.warn(
```

```
In [139]: import os
os.chdir(r"X:\Lighting Prediction")
print(os.listdir())
```

```
['All District lightning data2017_2022 (1).csv', 'cleaned_lis_final_v2.cs
v', 'cleaned_mts_final.csv', 'cleaned_mts_step1.csv', 'cleaned_mts_step2.c
sv', 'feature_list.pkl', 'final_lightning_dataset.csv', 'final_lightning_d
ataset_cleaned.csv', 'final_lightning_dataset_cleaned_final.csv', 'final_l
ightning_dataset_cleaned_final_v2.csv', 'Jharkhand all District Weather.cs
v', 'lightning_dataset_FE_v2.csv', 'lightning_final_FE_dataset.csv', 'ligh
tning_rf_model.pkl', 'merged_lis_mts.csv', 'New folder', 'Processed_Distri
cts']
```

```
In [140]: import pickle
import pandas as pd

# Load trained Random Forest model
with open("lightning_rf_model.pkl", "rb") as f:
    model = pickle.load(f)

# Load feature list (the 45 columns the model was trained on)
with open("feature_list.pkl", "rb") as f:
    feature_list = pickle.load(f)

print("Number of features:", len(feature_list))
print(feature_list)
```

Number of features: 45

```
['Month_No', 'Day', 'Latitude', 'Longitude', 'Pressure_hPa', 'Rh', 'Rain_m',
'Temp_C', 'WindSpeed', 'DayOfYear', 'WeekOfYear', 'IsMonsoon', 'THI',
'Temp_3day_mean', 'Rh_3day_mean', 'Rain_3day_sum', 'Pres_3day_mean', 'Pres
sure_Drop', 'Dist_Bokaro', 'Dist_Chatra', 'Dist_Deoghar', 'Dist_Dhanbad',
'Dist_Dumka', 'Dist_Garhwa', 'Dist_Giridih', 'Dist_Godda', 'Dist_Gumla',
'Dist_Hazaribagh', 'Dist_Jamtara', 'Dist_Khunti', 'Dist_Koderma', 'Dist_La
tehar', 'Dist_Lohardaga', 'Dist_Pakur', 'Dist_Palamu', 'Dist_Ramgarh', 'Di
st_Ranchi', 'Dist_Sahibganj', 'Dist_Simdega', 'Dist_West Singhbhum', 'Seas
on_Monsoon', 'Season_PostMonsoon', 'Season_PreMonsoon', 'Season_Summer',
'Season_Winter']
```

```
In [141]: def predict_with_threshold(input_dict, threshold=0.25):
    """
    input_dict: dict with keys = feature names in feature_list
    threshold: probability cutoff for class 1 (lightning)
    """
    # Make a 1-row DataFrame
    row = pd.DataFrame([input_dict])

    # Ensure correct column order and fill any missing with 0
    row = row.reindex(columns=feature_list, fill_value=0)

    # Get probability of class 1 (lightning)
    prob = model.predict_proba(row)[0][1]

    # Apply NEW threshold 0.25
    pred = 1 if prob >= threshold else 0

    print(f"Prediction (threshold={threshold}): {pred}")
    print(f"Lightning Probability: {prob}")

    return pred, prob
```

```

In [143]: manual_input = {
    "Month_No": 8,
    "Day": 25,
    "Latitude": 24.12,
    "Longitude": 85.34,
    "Pressure_hPa": 942.8,          # VERY LOW PRESSURE
    "Rh": 97.2,                   # EXTREMELY HIGH HUMIDITY
    "Rain_mm": 58.0,              # HEAVY RAIN
    "Temp_C": 31.4,               # HOT + HUMID
    "WindSpeed": 3.5,
    "DayOfYear": 237,
    "WeekOfYear": 34,
    "IsMonsoon": 1,
    "THI": 84.5,
    "Temp_3day_mean": 30.9,
    "Rh_3day_mean": 95.1,
    "Rain_3day_sum": 148.0,       # EXTREMELY HIGH
    "Pres_3day_mean": 953.5,
    "Pressure_Drop": -11.0,      # STRONG DROP

    # One-hot district
    "Dist_Ranchi": 1,
    "Dist_Bokaro": 0,
    "Dist_Dhanbad": 0,
    "Dist_Chatra": 0,
    "Dist_Deoghar": 0,
    "Dist_Dumka": 0,
    "Dist_Garhwa": 0,
    "Dist_Giridih": 0,
    "Dist_Godda": 0,
    "Dist_Gumla": 0,
    "Dist_Hazaribagh": 0,
    "Dist_Jamtara": 0,
    "Dist_Khunti": 0,
    "Dist_Koderma": 0,
    "Dist_Latehar": 0,
    "Dist_Lohardaga": 0,
    "Dist_Pakur": 0,
    "Dist_Palamu": 0,
    "Dist_Ramgarh": 0,
    "Dist_Sahibganj": 0,
    "Dist_Simdega": 0,
    "Dist_West Singhbhum": 0,

    # Season OHE
    "Season_Monsoon": 1,
    "Season_PostMonsoon": 0,
    "Season_PreMonsoon": 0,
    "Season_Summer": 0,
    "Season_Winter": 0
}

predict_with_threshold(manual_input, threshold=0.25)

```

Prediction (threshold=0.25): 0
 Lightning Probability: 0.19702619766309584

Out[143]: (0, 0.19702619766309584)

```

In [144]: import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, roc_auc_score, confusion
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.calibration import CalibratedClassifierCV
import numpy as np
import pickle

# 1) Load FE dataset
df = pd.read_csv(r"X:\Lighting Prediction\lightning_final_FE_dataset.csv",

print(df.shape)
print(df.columns)

# 2) Define feature columns (same 45 as model.feature_names_in_)
feature_cols = [
    'Month_No', 'Day', 'Latitude', 'Longitude',
    'Pressure_hPa', 'Rh', 'Rain_mm', 'Temp_C', 'WindSpeed',
    'DayOfYear', 'WeekOfYear', 'IsMonsoon', 'THI',
    'Temp_3day_mean', 'Rh_3day_mean', 'Rain_3day_sum',
    'Pres_3day_mean', 'Pressure_Drop',
    'Dist_Bokaro', 'Dist_Chatra', 'Dist_Deoghar', 'Dist_Dhanbad',
    'Dist_Dumka', 'Dist_Garhwa', 'Dist_Giridih', 'Dist_Godda',
    'Dist_Gumla', 'Dist_Hazaribagh', 'Dist_Jamtara', 'Dist_Khunti',
    'Dist_Koderma', 'Dist_Latehar', 'Dist_Lohardaga', 'Dist_Pakur',
    'Dist_Palamu', 'Dist_Ramgarh', 'Dist_Ranchi', 'Dist_Sahibganj',
    'Dist_Simdega', 'Dist_West Singhbhum',
    'Season_Monsoon', 'Season_PostMonsoon', 'Season_PreMonsoon',
    'Season_Summer', 'Season_Winter'
]

X = df[feature_cols]
y = df["LightningOccurred"]

```

```

(53843, 51)
Index(['Month', 'Month_No', 'Day', 'Latitude', 'Longitude', 'Radiance',
      'Events', 'Date', 'Pressure_hPa', 'Rh', 'Rain_mm', 'State', 'Temp_
C',
      'WindSpeed', 'LightningOccurred', 'DayOfYear', 'WeekOfYear',
      'IsMonsoon', 'THI', 'Temp_3day_mean', 'Rh_3day_mean', 'Rain_3day_su
m',
      'Pres_3day_mean', 'Pressure_Drop', 'Dist_Bokaro', 'Dist_Chatra',
      'Dist_Deoghar', 'Dist_Dhanbad', 'Dist_Dumka', 'Dist_Garhwa',
      'Dist_Giridih', 'Dist_Godda', 'Dist_Gumla', 'Dist_Hazaribagh',
      'Dist_Jamtara', 'Dist_Khunti', 'Dist_Koderma', 'Dist_Latehar',
      'Dist_Lohardaga', 'Dist_Pakur', 'Dist_Palamu', 'Dist_Ramgarh',
      'Dist_Ranchi', 'Dist_Sahibganj', 'Dist_Simdega', 'Dist_West Singhbh
um',
      'Season_Monsoon', 'Season_PostMonsoon', 'Season_PreMonsoon',
      'Season_Summer', 'Season_Winter'],
      dtype='object')

```

```
In [145]: X_train, X_test, y_train, y_test = train_test_split(
          X, y, test_size=0.2, stratify=y, random_state=42
          )

          print(X_train.shape, X_test.shape)
```

(43074, 45) (10769, 45)

```
In [146]: # Base Gradient Boosting model (strong but not too overfitted)
          gb = GradientBoostingClassifier(
              n_estimators=250,
              learning_rate=0.05,
              max_depth=3,
              subsample=0.8,
              random_state=42
          )

          # Wrap with calibration (sigmoid = Platt scaling)
          gb_cal = CalibratedClassifierCV(gb, method="sigmoid", cv=5)

          print("Training calibrated Gradient Boosting...")
          gb_cal.fit(X_train, y_train)
```

Training calibrated Gradient Boosting...

```
Out[146]: CalibratedClassifierCV(cv=5,
                                estimator=GradientBoostingClassifier(learning_rate=
                                0.05,
                                n_estimators=2
                                50,
                                random_state=4
                                2,
                                subsample=0.
                                8))
```

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.

On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.


```
In [147]: # Predict on test set
y_proba = gb_cal.predict_proba(X_test)[: , 1]
y_pred = (y_proba >= 0.5).astype(int) # start with 0.5 threshold

print("Confusion matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification report:\n", classification_report(y_test, y_pred))
print("ROC-AUC:", roc_auc_score(y_test, y_proba))

# Look at probability distribution for lightning days
pos_probs = y_proba[y_test == 1]
neg_probs = y_proba[y_test == 0]

print("\nPositive (lightning) prob percentiles:")
print(np.quantile(pos_probs, [0.5, 0.75, 0.9, 0.95, 0.99]))
print("Max lightning prob:", pos_probs.max())

print("\nNegative (no-light) prob percentiles:")
print(np.quantile(neg_probs, [0.5, 0.75, 0.9, 0.95, 0.99]))
print("Max non-light prob:", neg_probs.max())
```

Confusion matrix:

```
[[9410  16]
 [ 358 985]]
```

Classification report:

	precision	recall	f1-score	support
0	0.96	1.00	0.98	9426
1	0.98	0.73	0.84	1343
accuracy			0.97	10769
macro avg	0.97	0.87	0.91	10769
weighted avg	0.97	0.97	0.96	10769

ROC-AUC: 0.968318961873963

Positive (lightning) prob percentiles:

```
[0.99795193 0.99893746 0.99942905 0.99958197 0.999705 ]
```

Max lightning prob: 0.9998134071806659

Negative (no-light) prob percentiles:

```
[0.00719101 0.03422091 0.0888952 0.14039975 0.27179156]
```

Max non-light prob: 0.8900535225744047

```
In [148]: with open("lightning_gb_calibrated.pkl", "wb") as f:
           pickle.dump(gb_cal, f)

           with open("gb_feature_list.pkl", "wb") as f:
               pickle.dump(feature_cols, f)

           print("Saved lightning_gb_calibrated.pkl and gb_feature_list.pkl")
```

Saved lightning_gb_calibrated.pkl and gb_feature_list.pkl

```
In [152]: import pickle
import pandas as pd

# Go to your folder (if not already there)
import os
os.chdir(r"X:\Lightning Prediction")

with open("lightning_gb_calibrated.pkl", "rb") as f:
    new_model = pickle.load(f)

with open("gb_feature_list.pkl", "rb") as f:
    gb_features = pickle.load(f)

print("Model loaded. Number of features:", len(gb_features))
print(gb_features[:10], "...")
```

Model loaded. Number of features: 45

['Month_No', 'Day', 'Latitude', 'Longitude', 'Pressure_hPa', 'Rh', 'Rain_m
m', 'Temp_C', 'WindSpeed', 'DayOfYear'] ...

```
In [153]: def predict_new_model(input_dict, threshold=0.5):
    row = pd.DataFrame([input_dict]).reindex(columns=gb_features, fill_valu
    prob = new_model.predict_proba(row)[0, 1]
    pred = int(prob >= threshold)
    print(f"Prediction (thr={threshold}):", pred)
    print("Lightning probability:", prob)
    return pred, prob
```

```
In [159]: manual_input = {
    "Month_No": 8,
    "Day": 25,
    "Latitude": 24.12,
    "Longitude": 85.34,
    "Pressure_hPa": 950.8,
    "Rh": 97.2,
    "Rain_mm": 58.0,
    "Temp_C": 31.4,
    "WindSpeed": 3.5,
    "DayOfYear": 237,
    "WeekOfYear": 34,
    "IsMonsoon": 1,
    "THI": 84.5,
    "Temp_3day_mean": 30.9,
    "Rh_3day_mean": 95.1,
    "Rain_3day_sum": 148.0,
    "Pres_3day_mean": 953.5,
    "Pressure_Drop": -12.0,

    "Dist_Bokaro": 0,
    "Dist_Chatra": 0,
    "Dist_Deoghar": 0,
    "Dist_Dhanbad": 0,
    "Dist_Dumka": 0,
    "Dist_Garhwa": 0,
    "Dist_Giridih": 0,
    "Dist_Godda": 0,
    "Dist_Gumla": 0,
    "Dist_Hazaribagh": 0,
    "Dist_Jamtara": 0,
    "Dist_Khunti": 0,
    "Dist_Koderma": 0,
    "Dist_Latehar": 0,
    "Dist_Lohardaga": 0,
    "Dist_Pakur": 0,
    "Dist_Palamu": 0,
    "Dist_Ramgarh": 0,
    "Dist_Ranchi": 1,
    "Dist_Sahibganj": 0,
    "Dist_Simdega": 0,
    "Dist_West Singhbhum": 0,

    "Season_Monsoon": 1,
    "Season_PostMonsoon": 0,
    "Season_PreMonsoon": 0,
    "Season_Summer": 0,
    "Season_Winter": 0
}

predict_new_model(manual_input, threshold=0.5)
```

Prediction (thr=0.5): 0
Lightning probability: 0.4809068144538896

Out[159]: (0, 0.4809068144538896)

```
In [160]: predict_new_model(manual_input, threshold=0.35)
```

```
Prediction (thr=0.35): 1  
Lightning probability: 0.4809068144538896
```

```
Out[160]: (1, 0.4809068144538896)
```

```
In [166]: super_storm = {  
    "Month_No": 7,  
    "Day": 10,  
    "Latitude": 22.35,  
    "Longitude": 85.85, # West Singhbhum lightning hotspot  
  
    "Pressure_hPa": 932.0,  
    "Rh": 97.5,  
    "Rain_mm": 110.0,  
    "Temp_C": 28.4,  
    "WindSpeed": 6.9,  
  
    "DayOfYear": 191,  
    "WeekOfYear": 27,  
    "IsMonsoon": 1,  
    "THI": 82.4,  
  
    "Temp_3day_mean": 28.1,  
    "Rh_3day_mean": 95.6,  
    "Rain_3day_sum": 240.0,  
    "Pres_3day_mean": 958.5,  
    "Pressure_Drop": -26.5,  
  
    # district one-hot (West Singhbhum)  
    "Dist_Bokaro": 0, "Dist_Chatra": 0, "Dist_Deoghar": 0, "Dist_Dhanbad": 0,  
    "Dist_Dumka": 0, "Dist_Garhwa": 0, "Dist_Giridih": 0, "Dist_Godda": 0,  
    "Dist_Gumla": 0, "Dist_Hazaribagh": 0, "Dist_Jamtara": 0, "Dist_Khunti": 0,  
    "Dist_Koderma": 0, "Dist_Latehar": 0, "Dist_Lohardaga": 0, "Dist_Pakur": 0,  
    "Dist_Palamu": 0, "Dist_Ramgarh": 0, "Dist_Ranchi": 0, "Dist_Sahibganj": 0,  
    "Dist_Simdega": 0,  
    "Dist_West Singhbhum": 1,  
  
    "Season_Monsoon": 1,  
    "Season_PostMonsoon": 0,  
    "Season_PreMonsoon": 0,  
    "Season_Summer": 0,  
    "Season_Winter": 0  
}
```

```
In [167]: predict_new_model(super_storm, threshold=0.35)
```

```
Prediction (thr=0.35): 1  
Lightning probability: 0.369482726738301
```

```
Out[167]: (1, 0.369482726738301)
```

```

In [168]: import numpy as np
import pandas as pd

# X_train, X_test, y_train, y_test, new_model already defined from training

# Get probabilities on test set
y_proba = new_model.predict_proba(X_test)[: , 1]

# Build a test dataframe aligned with X_test indices
df_test = df.loc[X_test.index].copy()
df_test["prob"] = y_proba

# Sort by highest predicted lightning probability
top_lightning = df_test.sort_values("prob", ascending=False).head(10)

print(top_lightning[[
    "Date", "District" if "District" in df_test.columns else "DayOfYear",
    "LightningOccurred", "prob",
    "Rain_mm", "Rain_3day_sum", "Rh", "Temp_C", "Pressure_Drop",
    "Season_Monsoon", "Season_PreMonsoon", "Season_Summer"
]])

```

	Date	DayOfYear	LightningOccurred	prob	Rain_mm	\
19839	2018-05-31	151	1	0.999813	8.145610	
19841	2018-05-31	151	1	0.999813	8.145610	
19834	2018-05-31	151	1	0.999813	8.145610	
51926	2019-05-23	143	1	0.999738	7.753734	
19400	2017-05-31	151	1	0.999736	1.923863	
19396	2017-05-31	151	1	0.999736	1.923863	
39015	2018-06-25	176	1	0.999725	7.590380	
39016	2018-06-25	176	1	0.999725	7.590380	
39018	2018-06-25	176	1	0.999725	7.590380	
51970	2019-06-20	171	1	0.999724	16.713742	

	Rain_3day_sum	Rh	Temp_C	Pressure_Drop	Season_Monsoon	\
19839	24.436831	69.404657	28.424528	1.136868e-13		0
19841	24.436831	69.404657	28.424528	1.136868e-13		0
19834	24.436831	69.404657	28.424528	1.136868e-13		0
51926	23.261202	63.722368	30.639121	0.000000e+00		0
19400	5.771588	58.558095	29.198583	0.000000e+00		0
19396	5.771588	58.558095	29.198583	0.000000e+00		0
39015	22.771140	58.512574	32.336889	1.136868e-13		1
39016	22.771140	58.512574	32.336889	1.136868e-13		1
39018	22.771140	58.512574	32.336889	1.136868e-13		1
51970	50.141227	73.380629	29.379513	1.136868e-13		1

	Season_PreMonsoon	Season_Summer
19839	1	0
19841	1	0
19834	1	0
51926	1	0
19400	1	0
19396	1	0
39015	0	0
39016	0	0
39018	0	0
51970	0	0

```
In [172]: manual_input = {
    "Month_No": 5,
    "Day": 30,
    "Latitude": 23.70,
    "Longitude": 86.22,
    "Pressure_hPa": 958.2,
    "Rh": 66.5,
    "Rain_mm": 9.4,
    "Temp_C": 29.1,
    "WindSpeed": 2.1,
    "DayOfYear": 150,
    "WeekOfYear": 22,
    "IsMonsoon": 0,
    "THI": 78.4,
    "Temp_3day_mean": 28.8,
    "Rh_3day_mean": 64.2,
    "Rain_3day_sum": 21.0,
    "Pres_3day_mean": 959.0,
    "Pressure_Drop": 0.0,

    # District one-hot
    "Dist_Ranchi": 1,
    # all others 0

    "Season_Monsoon": 0,
    "Season_PreMonsoon": 1,
    "Season_Summer": 0,
    "Season_Winter": 0,
    "Season_PostMonsoon": 0
}

predict_new_model(manual_input, threshold=0.5)
```

```
Prediction (thr=0.5): 1
Lightning probability: 0.9978577230785446
```

```
Out[172]: (1, 0.9978577230785446)
```

```
In [193]: case_0_3 = {  
    "Month_No": 2,  
    "Day": 11,  
    "Latitude": 23.70,  
    "Longitude": 86.12,  
    "Pressure_hPa": 994,  
    "Rh": 46,  
    "Rain_mm": 0.1,  
    "Temp_C": 22.5,  
    "WindSpeed": 1.5,  
    "DayOfYear": 42,  
    "WeekOfYear": 6,  
    "IsMonsoon": 0,  
    "THI": 64,  
    "Temp_3day_mean": 22,  
    "Rh_3day_mean": 48,  
    "Rain_3day_sum": 0.1,  
    "Pres_3day_mean": 995,  
    "Pressure_Drop": 0,  
    "Dist_Ranchi": 1,  
  
    "Season_Winter": 0,  
    "Season_Summer": 0,  
    "Season_PreMonsoon": 0,  
    "Season_PostMonsoon": 0,  
    "Season_Monsoon": 0  
}
```

```
In [194]: case_0_5 = {  
    "Month_No": 4,  
    "Day": 8,  
    "Latitude": 23.76,  
    "Longitude": 86.0,  
    "Pressure_hPa": 982,  
    "Rh": 55,  
    "Rain_mm": 1.5,  
    "Temp_C": 26.5,  
    "WindSpeed": 2.0,  
    "DayOfYear": 98,  
    "WeekOfYear": 14,  
    "IsMonsoon": 0,  
    "THI": 70,  
    "Temp_3day_mean": 26.2,  
    "Rh_3day_mean": 57,  
    "Rain_3day_sum": 2.2,  
    "Pres_3day_mean": 983,  
    "Pressure_Drop": 0,  
  
    "Dist_Ranchi": 1,  
    "Season_PreMonsoon": 1  
}
```

```
In [195]: case_0_7 = {  
    "Month_No": 5,  
    "Day": 30,  
    "Latitude": 23.8,  
    "Longitude": 86.1,  
    "Pressure_hPa": 973,  
    "Rh": 62,  
    "Rain_mm": 3.2,  
    "Temp_C": 28.0,  
    "WindSpeed": 2.3,  
    "DayOfYear": 150,  
    "WeekOfYear": 22,  
    "IsMonsoon": 0,  
    "THI": 74,  
    "Temp_3day_mean": 27.5,  
    "Rh_3day_mean": 60,  
    "Rain_3day_sum": 8.0,  
    "Pres_3day_mean": 974,  
    "Pressure_Drop": 0,  
  
    "Dist_Ranchi": 1,  
    "Season_PreMonsoon": 1  
}
```

```
In [196]: case_0_85 = {  
    "Month_No": 9,  
    "Day": 6,  
    "Latitude": 23.88,  
    "Longitude": 86.28,  
    "Pressure_hPa": 972,  
    "Rh": 66,  
    "Rain_mm": 4.5,  
    "Temp_C": 27.8,  
    "WindSpeed": 2.7,  
    "DayOfYear": 249,  
    "WeekOfYear": 36,  
    "IsMonsoon": 1,  
    "THI": 75,  
    "Temp_3day_mean": 27.2,  
    "Rh_3day_mean": 64,  
    "Rain_3day_sum": 10.4,  
    "Pres_3day_mean": 973,  
    "Pressure_Drop": 0,  
  
    "Dist_Ranchi": 1,  
    "Season_Monsoon": 1  
}
```



```
In [197]: case_0_92 = {  
    "Month_No": 10,  
    "Day": 2,  
    "Latitude": 23.82,  
    "Longitude": 86.21,  
    "Pressure_hPa": 969,  
    "Rh": 69,  
    "Rain_mm": 5.0,  
    "Temp_C": 27.5,  
    "WindSpeed": 3.2,  
    "DayOfYear": 275,  
    "WeekOfYear": 40,  
    "IsMonsoon": 1,  
    "THI": 76,  
    "Temp_3day_mean": 27.0,  
    "Rh_3day_mean": 67,  
    "Rain_3day_sum": 13,  
    "Pres_3day_mean": 970,  
    "Pressure_Drop": 0,  
  
    "Dist_Ranchi": 1,  
    "Season_PostMonsoon": 1  
}
```

```
In [198]: predict_new_model(case_0_3)  
predict_new_model(case_0_5)  
predict_new_model(case_0_7)  
predict_new_model(case_0_85)  
predict_new_model(case_0_92)
```

```
Prediction (thr=0.5): 1  
Lightning probability: 0.7303751856709543  
Prediction (thr=0.5): 1  
Lightning probability: 0.9974525241367033  
Prediction (thr=0.5): 1  
Lightning probability: 0.9964308762298796  
Prediction (thr=0.5): 1  
Lightning probability: 0.989810278789396  
Prediction (thr=0.5): 1  
Lightning probability: 0.9899523321391506
```

```
Out[198]: (1, 0.9899523321391506)
```

```
In [201]: import pandas as pd
import numpy as np

# 1) Load your final FE dataset
df = pd.read_csv(r"X:\Lightning Prediction\lightning_final_FE_dataset.csv",

print(df.shape)
print(df.columns)

# 2) Columns we DO NOT want as features
drop_cols = ["Radiance", "Events", "Month", "Date", "State", "LightningOccu

# 3) Final feature columns
feature_cols = [c for c in df.columns if c not in drop_cols]

print("Number of features:", len(feature_cols))
print("Features:", feature_cols)

# 4) X, y
X = df[feature_cols]
y = df["LightningOccurred"]

print("Class balance:")
print(y.value_counts())
```

```
(53843, 51)
Index(['Month', 'Month_No', 'Day', 'Latitude', 'Longitude', 'Radiance',
      'Events', 'Date', 'Pressure_hPa', 'Rh', 'Rain_mm', 'State', 'Temp_
C',
      'WindSpeed', 'LightningOccurred', 'DayOfYear', 'WeekOfYear',
      'IsMonsoon', 'THI', 'Temp_3day_mean', 'Rh_3day_mean', 'Rain_3day_su
m',
      'Pres_3day_mean', 'Pressure_Drop', 'Dist_Bokaro', 'Dist_Chatra',
      'Dist_Deoghar', 'Dist_Dhanbad', 'Dist_Dumka', 'Dist_Garhwa',
      'Dist_Giridih', 'Dist_Godda', 'Dist_Gumla', 'Dist_Hazaribagh',
      'Dist_Jamtara', 'Dist_Khunti', 'Dist_Koderma', 'Dist_Latehar',
      'Dist_Lohardaga', 'Dist_Pakur', 'Dist_Palamu', 'Dist_Ramgarh',
      'Dist_Ranchi', 'Dist_Sahibganj', 'Dist_Simdega', 'Dist_West Singhbh
um',
      'Season_Monsoon', 'Season_PostMonsoon', 'Season_PreMonsoon',
      'Season_Summer', 'Season_Winter'],
      dtype='object')
Number of features: 45
Features: ['Month_No', 'Day', 'Latitude', 'Longitude', 'Pressure_hPa', 'R
h', 'Rain_mm', 'Temp_C', 'WindSpeed', 'DayOfYear', 'WeekOfYear', 'IsMonsoo
n', 'THI', 'Temp_3day_mean', 'Rh_3day_mean', 'Rain_3day_sum', 'Pres_3day_m
ean', 'Pressure_Drop', 'Dist_Bokaro', 'Dist_Chatra', 'Dist_Deoghar', 'Dist
_Dhanbad', 'Dist_Dumka', 'Dist_Garhwa', 'Dist_Giridih', 'Dist_Godda', 'Dis
t_Gumla', 'Dist_Hazaribagh', 'Dist_Jamtara', 'Dist_Khunti', 'Dist_Koderm
a', 'Dist_Latehar', 'Dist_Lohardaga', 'Dist_Pakur', 'Dist_Palamu', 'Dist_R
amgarh', 'Dist_Ranchi', 'Dist_Sahibganj', 'Dist_Simdega', 'Dist_West Singh
bhum', 'Season_Monsoon', 'Season_PostMonsoon', 'Season_PreMonsoon', 'Seaso
n_Summer', 'Season_Winter']
Class balance:
0    47130
1     6713
Name: LightningOccurred, dtype: int64
```

```
In [202]: from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    stratify=y,
    random_state=42
)

X_train.shape, X_test.shape
```

```
Out[202]: ((43074, 45), (10769, 45))
```

```
In [203]: from imblearn.pipeline import Pipeline as ImbPipeline
from imblearn.over_sampling import SMOTE
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.neural_network import MLPClassifier
from sklearn.ensemble import GradientBoostingClassifier

smote = SMOTE(random_state=42)

log_clf = LogisticRegression(
    max_iter=1000,
    n_jobs=-1,
    class_weight="balanced",
    solver="lbfgs"
)

mlp_clf = MLPClassifier(
    hidden_layer_sizes=(64, 32),
    activation="relu",
    alpha=1e-3,
    learning_rate_init=1e-3,
    max_iter=200,
    random_state=42
)

gb_clf = GradientBoostingClassifier(
    n_estimators=150,
    learning_rate=0.05,
    max_depth=3,
    subsample=0.8,
    random_state=42
)

log_pipe = ImbPipeline(steps=[
    ("scaler", StandardScaler()),
    ("smote", SMOTE(random_state=42)),
    ("clf", log_clf)
])

mlp_pipe = ImbPipeline(steps=[
    ("scaler", StandardScaler()),
    ("smote", SMOTE(random_state=42)),
    ("clf", mlp_clf)
])

gb_pipe = ImbPipeline(steps=[
    ("scaler", StandardScaler()),
    ("smote", SMOTE(random_state=42)),
    ("clf", gb_clf)
])
```

```
In [204]: print("Training LogisticRegression...")
log_pipe.fit(X_train, y_train)

print("Training MLP...")
mlp_pipe.fit(X_train, y_train)

print("Training GradientBoosting...")
gb_pipe.fit(X_train, y_train)
```

Training LogisticRegression...

C:\Users\SHUBHAM\anaconda3\lib\site-packages\sklearn\base.py:474: FutureWarning: `BaseEstimator.\_validate\_data` is deprecated in 1.6 and will be removed in 1.7. Use `sklearn.utils.validation.validate\_data` instead. This function becomes public and is part of the scikit-learn developer API.

warnings.warn(

Training MLP...

C:\Users\SHUBHAM\anaconda3\lib\site-packages\sklearn\base.py:474: FutureWarning: `BaseEstimator.\_validate\_data` is deprecated in 1.6 and will be removed in 1.7. Use `sklearn.utils.validation.validate\_data` instead. This function becomes public and is part of the scikit-learn developer API.

warnings.warn(

C:\Users\SHUBHAM\anaconda3\lib\site-packages\sklearn\neural_network_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (200) reached and the optimization hasn't converged yet.

warnings.warn(

C:\Users\SHUBHAM\anaconda3\lib\site-packages\sklearn\base.py:474: FutureWarning: `BaseEstimator.\_validate\_data` is deprecated in 1.6 and will be removed in 1.7. Use `sklearn.utils.validation.validate\_data` instead. This function becomes public and is part of the scikit-learn developer API.

warnings.warn(

Training GradientBoosting...

```
Out[204]: Pipeline(steps=[('scaler', StandardScaler()), ('smote', SMOTE(random_state
=42)),
                        ('clf',
                         GradientBoostingClassifier(learning_rate=0.05,
                                                         n_estimators=150, random_state
=42,
                                                         subsample=0.8))])
```

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.

On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

```
In [205]: from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score

def eval_model(name, model, X_tr, y_tr, X_te, y_te):
    y_tr_proba = model.predict_proba(X_tr)[:,-1]
    y_te_proba = model.predict_proba(X_te)[:,-1]
    y_te_pred = (y_te_proba >= 0.5).astype(int)

    print(f"\n===== {name} =====")
    print("Confusion matrix:\n", confusion_matrix(y_te, y_te_pred))
    print("\nClassification report:\n", classification_report(y_te, y_te_pred))
    print("ROC-AUC (test):", roc_auc_score(y_te, y_te_proba))

    # Look at probability spread
    pos_probs = y_te_proba[y_te == 1]
    neg_probs = y_te_proba[y_te == 0]
    print("\nPositives prob percentiles (0.1,0.5,0.9):", np.quantile(pos_probs, [0.1, 0.5, 0.9]))
    print("Negatives prob percentiles (0.1,0.5,0.9):", np.quantile(neg_probs, [0.1, 0.5, 0.9]))
    print("Max pos prob:", pos_probs.max(), "Max neg prob:", neg_probs.max())

    return {
        "name": name,
        "auc": roc_auc_score(y_te, y_te_proba)
    }

res_log = eval_model("LogisticRegression", log_pipe, X_train, y_train, X_test, y_test)
res_mlp = eval_model("MLP", mlp_pipe, X_train, y_train, X_test, y_test)
res_gb = eval_model("GradientBoosting", gb_pipe, X_train, y_train, X_test, y_test)
```

==== LogisticRegression =====

Confusion matrix:

```
[[6845 2581]
 [ 210 1133]]
```

Classification report:

	precision	recall	f1-score	support
0	0.97	0.73	0.83	9426
1	0.31	0.84	0.45	1343
accuracy			0.74	10769
macro avg	0.64	0.78	0.64	10769
weighted avg	0.89	0.74	0.78	10769

ROC-AUC (test): 0.8714524977174556

Positives prob percentiles (0.1,0.5,0.9): [0.40544585 0.77475527 0.91749125]

Negatives prob percentiles (0.1,0.5,0.9): [0.00083256 0.19185097 0.7347707]

Max pos prob: 0.9786743610614368 Max neg prob: 0.9734100764653875

==== MLP =====

Confusion matrix:

```
[[9027 399]
 [ 210 1133]]
```

Classification report:

	precision	recall	f1-score	support
0	0.98	0.96	0.97	9426
1	0.74	0.84	0.79	1343
accuracy			0.94	10769
macro avg	0.86	0.90	0.88	10769
weighted avg	0.95	0.94	0.95	10769

ROC-AUC (test): 0.968169188406333

Positives prob percentiles (0.1,0.5,0.9): [0.03351349 0.99956541 0.999999]

Negatives prob percentiles (0.1,0.5,0.9): [2.96662429e-36 1.80643117e-12 2.60925538e-02]

Max pos prob: 0.9999999999999993 Max neg prob: 0.9999989783529445

==== GradientBoosting =====

Confusion matrix:

```
[[8898 528]
 [ 240 1103]]
```

Classification report:

	precision	recall	f1-score	support
0	0.97	0.94	0.96	9426
1	0.68	0.82	0.74	1343
accuracy			0.93	10769
macro avg	0.83	0.88	0.85	10769
weighted avg	0.94	0.93	0.93	10769

ROC-AUC (test): 0.9614371238185788

Positives prob percentiles (0.1,0.5,0.9): [0.323884 0.98135879 0.98808195]

Negatives prob percentiles (0.1,0.5,0.9): [0.01060989 0.06629926 0.40145681]

Max pos prob: 0.990077279672876 Max neg prob: 0.919811041961741

```
In [206]: # Ensemble: average probabilities
proba_log = log_pipe.predict_proba(X_test)[: ,1]
proba_mlp = mlp_pipe.predict_proba(X_test)[: ,1]
proba_gb = gb_pipe.predict_proba(X_test)[: ,1]

proba_ens = (proba_log + proba_mlp + proba_gb) / 3.0
y_ens_pred = (proba_ens >= 0.5).astype(int)

print("\n===== ENSEMBLE (Log + MLP + GB avg) =====")
print("Confusion matrix:\n", confusion_matrix(y_test, y_ens_pred))
print("\nClassification report:\n", classification_report(y_test, y_ens_pred))
print("ROC-AUC (test):", roc_auc_score(y_test, proba_ens))

pos_probs = proba_ens[y_test == 1]
neg_probs = proba_ens[y_test == 0]
print("\nPositives prob percentiles (0.1,0.5,0.9):", np.quantile(pos_probs, [0.1, 0.5, 0.9]))
print("Negatives prob percentiles (0.1,0.5,0.9):", np.quantile(neg_probs, [0.1, 0.5, 0.9]))
print("Max pos prob:", pos_probs.max(), "Max neg prob:", neg_probs.max())
```

===== ENSEMBLE (Log + MLP + GB avg) =====

Confusion matrix:

```
[[8973 453]
 [ 199 1144]]
```

Classification report:

	precision	recall	f1-score	support
0	0.98	0.95	0.96	9426
1	0.72	0.85	0.78	1343
accuracy			0.94	10769
macro avg	0.85	0.90	0.87	10769
weighted avg	0.95	0.94	0.94	10769

ROC-AUC (test): 0.9694953471481979

Positives prob percentiles (0.1,0.5,0.9): [0.39714458 0.89343174 0.96733168]

Negatives prob percentiles (0.1,0.5,0.9): [0.00444134 0.09874335 0.3908136]

Max pos prob: 0.9890627381044519 Max neg prob: 0.9086705352623666


```
In [207]: import pickle

ensemble = {
    "log_pipe": log_pipe,
    "mlp_pipe": mlp_pipe,
    "gb_pipe": gb_pipe,
    "feature_list": feature_cols
}

with open(r"X:\Lighting Prediction\lightning_ensemble_model.pkl", "wb") as f:
    pickle.dump(ensemble, f)

print("Saved to lightning_ensemble_model.pkl")
```

Saved to lightning_ensemble_model.pkl

```
In [208]: import pickle
import pandas as pd
import numpy as np

with open(r"X:\Lighting Prediction\lightning_ensemble_model.pkl", "rb") as f:
    ens = pickle.load(f)

log_pipe = ens["log_pipe"]
mlp_pipe = ens["mlp_pipe"]
gb_pipe = ens["gb_pipe"]
feature_list = ens["feature_list"]

def predict_ensemble(input_dict, threshold=0.5):
    row = pd.DataFrame([input_dict]).reindex(columns=feature_list, fill_val=0)

    p_log = log_pipe.predict_proba(row)[:,1]
    p_mlp = mlp_pipe.predict_proba(row)[:,1]
    p_gb = gb_pipe.predict_proba(row)[:,1]

    prob = (p_log + p_mlp + p_gb) / 3.0
    pred = int(prob[0] >= threshold)

    print(f"Prediction (thr={threshold}):", pred)
    print("Lightning probability:", prob[0])
    return pred, prob[0]
```

```
In [209]: clear_winter = {
    "Month_No": 1,
    "Day": 10,
    "Latitude": 23.36,
    "Longitude": 85.33,
    "Pressure_hPa": 1008,
    "Rh": 38,
    "Rain_mm": 0.0,
    "Temp_C": 16.5,
    "WindSpeed": 0.8,
    "DayOfYear": 10,
    "WeekOfYear": 2,
    "IsMonsoon": 0,
    "THI": 52,                # cool + dry
    "Temp_3day_mean": 16.0,
    "Rh_3day_mean": 40,
    "Rain_3day_sum": 0.0,
    "Pres_3day_mean": 1009,
    "Pressure_Drop": 0.0,

    # District one-hot
    "Dist_Bokaro": 0, "Dist_Chatra": 0, "Dist_Deoghar": 0, "Dist_Dhanbad": 0,
    "Dist_Dumka": 0, "Dist_Garhwa": 0, "Dist_Giridih": 0, "Dist_Godda": 0,
    "Dist_Gumla": 0, "Dist_Hazaribagh": 0, "Dist_Jamtara": 0, "Dist_Khunti": 0,
    "Dist_Koderma": 0, "Dist_Latehar": 0, "Dist_Lohardaga": 0, "Dist_Pakur": 0,
    "Dist_Palamu": 0, "Dist_Ramgarh": 0, "Dist_Ranchi": 1, "Dist_Sahibganj": 0,
    "Dist_Simdega": 0, "Dist_West Singhbhum": 0,

    # Season one-hot
    "Season_Monsoon": 0,
    "Season_PostMonsoon": 0,
    "Season_PreMonsoon": 0,
    "Season_Summer": 0,
    "Season_Winter": 1
}
```

```
In [210]: weak_premonsoon = {
    "Month_No": 4,
    "Day": 5,
    "Latitude": 23.7,
    "Longitude": 86.1,
    "Pressure_hPa": 986,
    "Rh": 55,
    "Rain_mm": 0.8,
    "Temp_C": 26.0,
    "WindSpeed": 1.6,
    "DayOfYear": 96,
    "WeekOfYear": 14,
    "IsMonsoon": 0,
    "THI": 68,
    "Temp_3day_mean": 25.5,
    "Rh_3day_mean": 56,
    "Rain_3day_sum": 1.5,
    "Pres_3day_mean": 987,
    "Pressure_Drop": 0.0,

    "Dist_Bokaro": 0, "Dist_Chatra": 0, "Dist_Deoghar": 0, "Dist_Dhanbad": 0,
    "Dist_Dumka": 0, "Dist_Garhwa": 0, "Dist_Giridih": 0, "Dist_Godda": 0,
    "Dist_Gumla": 0, "Dist_Hazaribagh": 0, "Dist_Jamtara": 0, "Dist_Khunti": 0,
    "Dist_Koderma": 0, "Dist_Latehar": 0, "Dist_Lohardaga": 0, "Dist_Pakur": 0,
    "Dist_Palamu": 0, "Dist_Ramgarh": 0, "Dist_Ranchi": 1, "Dist_Sahibganj": 0,
    "Dist_Simdega": 0, "Dist_West Singhbhum": 0,

    "Season_Monsoon": 0,
    "Season_PostMonsoon": 0,
    "Season_PreMonsoon": 1,
    "Season_Summer": 0,
    "Season_Winter": 0
}
```

```
In [211]: moderate_monsoon = {  
    "Month_No": 7,  
    "Day": 15,  
    "Latitude": 23.4,  
    "Longitude": 85.4,  
    "Pressure_hPa": 972,  
    "Rh": 78,  
    "Rain_mm": 12.0,  
    "Temp_C": 26.5,  
    "WindSpeed": 2.5,  
    "DayOfYear": 196,  
    "WeekOfYear": 28,  
    "IsMonsoon": 1,  
    "THI": 75,  
    "Temp_3day_mean": 26.2,  
    "Rh_3day_mean": 76,  
    "Rain_3day_sum": 28.0,  
    "Pres_3day_mean": 973,  
    "Pressure_Drop": 0.0,  
  
    "Dist_Bokaro": 0, "Dist_Chatra": 0, "Dist_Deoghar": 0, "Dist_Dhanbad": 0,  
    "Dist_Dumka": 0, "Dist_Garhwa": 0, "Dist_Giridih": 0, "Dist_Godda": 0,  
    "Dist_Gumla": 0, "Dist_Hazaribagh": 0, "Dist_Jamtara": 0, "Dist_Khunti": 0,  
    "Dist_Koderma": 0, "Dist_Latehar": 0, "Dist_Lohardaga": 0, "Dist_Pakur": 0,  
    "Dist_Palamu": 0, "Dist_Ramgarh": 0, "Dist_Ranchi": 1, "Dist_Sahibganj": 0,  
    "Dist_Simdega": 0, "Dist_West Singhbhum": 0,  
  
    "Season_Monsoon": 1,  
    "Season_PostMonsoon": 0,  
    "Season_PreMonsoon": 0,  
    "Season_Summer": 0,  
    "Season_Winter": 0  
}
```

```
In [212]: strong_premonsoon = {
    "Month_No": 5,
    "Day": 29,
    "Latitude": 23.8,
    "Longitude": 86.2,
    "Pressure_hPa": 962,
    "Rh": 68,
    "Rain_mm": 7.5,
    "Temp_C": 28.5,
    "WindSpeed": 2.2,
    "DayOfYear": 149,
    "WeekOfYear": 22,
    "IsMonsoon": 0,
    "THI": 77,
    "Temp_3day_mean": 28.0,
    "Rh_3day_mean": 66,
    "Rain_3day_sum": 20.0,
    "Pres_3day_mean": 963,
    "Pressure_Drop": 0.0,

    "Dist_Bokaro": 0, "Dist_Chatra": 0, "Dist_Deoghar": 0, "Dist_Dhanbad": 0,
    "Dist_Dumka": 0, "Dist_Garhwa": 0, "Dist_Giridih": 0, "Dist_Godda": 0,
    "Dist_Gumla": 0, "Dist_Hazaribagh": 0, "Dist_Jamtara": 0, "Dist_Khunti": 0,
    "Dist_Koderma": 0, "Dist_Latehar": 0, "Dist_Lohardaga": 0, "Dist_Pakur": 0,
    "Dist_Palamu": 0, "Dist_Ramgarh": 0, "Dist_Ranchi": 1, "Dist_Sahibganj": 0,
    "Dist_Simdega": 0, "Dist_West Singhbhum": 0,

    "Season_Monsoon": 0,
    "Season_PostMonsoon": 0,
    "Season_PreMonsoon": 1,
    "Season_Summer": 0,
    "Season_Winter": 0
}
```

```
In [213]: strong_monsoon = {  
    "Month_No": 8,  
    "Day": 3,  
    "Latitude": 22.6,  
    "Longitude": 85.8,  
    "Pressure_hPa": 968,  
    "Rh": 82,  
    "Rain_mm": 25.0,  
    "Temp_C": 25.8,  
    "WindSpeed": 3.1,  
    "DayOfYear": 215,  
    "WeekOfYear": 31,  
    "IsMonsoon": 1,  
    "THI": 74,  
    "Temp_3day_mean": 26.0,  
    "Rh_3day_mean": 80,  
    "Rain_3day_sum": 60.0,  
    "Pres_3day_mean": 969,  
    "Pressure_Drop": 0.0,  
  
    "Dist_Bokaro": 0, "Dist_Chatra": 0, "Dist_Deoghar": 0, "Dist_Dhanbad": 0,  
    "Dist_Dumka": 0, "Dist_Garhwa": 0, "Dist_Giridih": 0, "Dist_Godda": 0,  
    "Dist_Gumla": 0, "Dist_Hazaribagh": 0, "Dist_Jamtara": 0, "Dist_Khunti": 0,  
    "Dist_Koderma": 0, "Dist_Latehar": 0, "Dist_Lohardaga": 0, "Dist_Pakur": 0,  
    "Dist_Palamu": 0, "Dist_Ramgarh": 0, "Dist_Ranchi": 0, "Dist_Sahibganj": 0,  
    "Dist_Simdega": 0, "Dist_West Singhbhum": 1,  
  
    "Season_Monsoon": 1,  
    "Season_PostMonsoon": 0,  
    "Season_PreMonsoon": 0,  
    "Season_Summer": 0,  
    "Season_Winter": 0  
}
```

```
In [214]: cases = {
    "clear_winter": clear_winter,
    "weak_premonsoon": weak_premonsoon,
    "moderate_monsoon": moderate_monsoon,
    "strong_premonsoon": strong_premonsoon,
    "strong_monsoon": strong_monsoon,
}

for name, case in cases.items():
    print("\n=== Scenario:", name, "===")
    predict_ensemble(case, threshold=0.5)
```

```
=== Scenario: clear_winter ===
Prediction (thr=0.5): 0
Lightning probability: 0.3614241392442392
```

```
=== Scenario: weak_premonsoon ===
Prediction (thr=0.5): 1
Lightning probability: 0.6431433258371116
```

```
=== Scenario: moderate_monsoon ===
Prediction (thr=0.5): 1
Lightning probability: 0.6483273967419975
```

```
=== Scenario: strong_premonsoon ===
Prediction (thr=0.5): 1
Lightning probability: 0.6593002995465925
```

```
=== Scenario: strong_monsoon ===
Prediction (thr=0.5): 1
Lightning probability: 0.6313299447092295
```

```

In [217]: better_clear_winter = {
    "Month_No": 8,
    "Day": 10,
    "Latitude": 23.36,
    "Longitude": 85.33,
    "Pressure_hPa": 850,           # higher pressure
    "Rh": 40,
    "Rain_mm": 25,
    "Temp_C": 25.5,
    "WindSpeed": 2.1,             # realistic winter wind
    "DayOfYear": 10,
    "WeekOfYear": 2,
    "IsMonsoon": 0,
    "THI": 50,
    "Temp_3day_mean": 25.8,
    "Rh_3day_mean": 42,
    "Rain_3day_sum": 0.0,
    "Pres_3day_mean": 853,
    "Pressure_Drop": 0.0,

    "Dist_Ranchi": 1,
    **{"f"Dist_{d}": 0 for d in [
        "Bokaro", "Chatra", "Deoghar", "Dhanbad", "Dumka", "Garhwa", "Giridih", "G
        "Gumla", "Hazaribagh", "Jamtara", "Khunti", "Koderma", "Latehar", "Lohard
        "Pakur", "Palamu", "Ramgarh", "Sahibganj", "Simdega", "West Singhbhum"
    ]},

    "Season_Monsoon": 0,
    "Season_PostMonsoon": 0,
    "Season_PreMonsoon": 0,
    "Season_Summer": 0,
    "Season_Winter": 1
}

```

```

In [218]: predict_ensemble(better_clear_winter)

```

Prediction (thr=0.5): 1
 Lightning probability: 0.6489173662523281

```

Out[218]: (1, 0.6489173662523281)

```



```

In [220]: from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.metrics import roc_auc_score
from sklearn.base import clone
import numpy as np

# X, y already defined earlier
# X = df[feature_cols]
# y = df["LightningOccurred"]

skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

print("Running 5-fold CV...\n")

# 1) Individual Models
log_scores = cross_val_score(log_pipe, X, y, cv=skf, scoring="roc_auc", n_j
mlp_scores = cross_val_score(mlp_pipe, X, y, cv=skf, scoring="roc_auc", n_j
gb_scores = cross_val_score(gb_pipe, X, y, cv=skf, scoring="roc_auc", n_j

print("==== 5-FOLD ROC-AUC (INDIVIDUAL MODELS) =====")
print(f"LogisticRegression: mean={log_scores.mean():.4f}, std={log_scores.s
print(f"MLP: mean={mlp_scores.mean():.4f}, std={mlp_scores.s
print(f"GradientBoosting: mean={gb_scores.mean():.4f}, std={gb_scores.std

# 2) Ensemble (average of the three models)
ens_scores = []

for fold, (train_idx, test_idx) in enumerate(skf.split(X, y), start=1):
    X_tr, X_te = X.iloc[train_idx], X.iloc[test_idx]
    y_tr, y_te = y.iloc[train_idx], y.iloc[test_idx]

    log_model = clone(log_pipe)
    mlp_model = clone(mlp_pipe)
    gb_model = clone(gb_pipe)

    log_model.fit(X_tr, y_tr)
    mlp_model.fit(X_tr, y_tr)
    gb_model.fit(X_tr, y_tr)

    p1 = log_model.predict_proba(X_te)[: , 1]
    p2 = mlp_model.predict_proba(X_te)[: , 1]
    p3 = gb_model.predict_proba(X_te)[: , 1]

    p_ens = (p1 + p2 + p3) / 3.0

    auc = roc_auc_score(y_te, p_ens)
    ens_scores.append(auc)

    print(f"Fold {fold} AUC (Ensemble): {auc:.4f}")

ens_scores = np.array(ens_scores)
print("\n==== 5-FOLD ROC-AUC (ENSEMBLE) =====")
print(f"Ensemble: mean={ens_scores.mean():.4f}, std={ens_scores.std():.4f}")

```

Running 5-fold CV...

```
===== 5-FOLD ROC-AUC (INDIVIDUAL MODELS) =====
LogisticRegression: mean=0.8717, std=0.0066
MLP:                  mean=0.9708, std=0.0017
GradientBoosting:    mean=0.9656, std=0.0027
```

C:\Users\SHUBHAM\anaconda3\lib\site-packages\sklearn\base.py:474: FutureWarning: `BaseEstimator.\_validate\_data` is deprecated in 1.6 and will be removed in 1.7. Use `sklearn.utils.validation.validate\_data` instead. This function becomes public and is part of the scikit-learn developer API.

warnings.warn(

C:\Users\SHUBHAM\anaconda3\lib\site-packages\sklearn\base.py:474: FutureWarning: `BaseEstimator.\_validate\_data` is deprecated in 1.6 and will be removed in 1.7. Use `sklearn.utils.validation.validate\_data` instead. This function becomes public and is part of the scikit-learn developer API.

warnings.warn(

C:\Users\SHUBHAM\anaconda3\lib\site-packages\sklearn\neural_network_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (200) reached and the optimization hasn't converged yet.

warnings.warn(

C:\Users\SHUBHAM\anaconda3\lib\site-packages\sklearn\base.py:474: FutureWarning: `BaseEstimator.\_validate\_data` is deprecated in 1.6 and will be removed in 1.7. Use `sklearn.utils.validation.validate\_data` instead. This function becomes public and is part of the scikit-learn developer API.

warnings.warn(

Fold 1 AUC (Ensemble): 0.9673

C:\Users\SHUBHAM\anaconda3\lib\site-packages\sklearn\base.py:474: FutureWarning: `BaseEstimator.\_validate\_data` is deprecated in 1.6 and will be removed in 1.7. Use `sklearn.utils.validation.validate\_data` instead. This function becomes public and is part of the scikit-learn developer API.

warnings.warn(

C:\Users\SHUBHAM\anaconda3\lib\site-packages\sklearn\base.py:474: FutureWarning: `BaseEstimator.\_validate\_data` is deprecated in 1.6 and will be removed in 1.7. Use `sklearn.utils.validation.validate\_data` instead. This function becomes public and is part of the scikit-learn developer API.

warnings.warn(

C:\Users\SHUBHAM\anaconda3\lib\site-packages\sklearn\neural_network_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (200) reached and the optimization hasn't converged yet.

warnings.warn(

C:\Users\SHUBHAM\anaconda3\lib\site-packages\sklearn\base.py:474: FutureWarning: `BaseEstimator.\_validate\_data` is deprecated in 1.6 and will be removed in 1.7. Use `sklearn.utils.validation.validate\_data` instead. This function becomes public and is part of the scikit-learn developer API.

warnings.warn(

Fold 2 AUC (Ensemble): 0.9743

```
C:\Users\SHUBHAM\anaconda3\lib\site-packages\sklearn\base.py:474: FutureWarning: `BaseEstimator._validate_data` is deprecated in 1.6 and will be removed in 1.7. Use `sklearn.utils.validation.validate_data` instead. This function becomes public and is part of the scikit-learn developer API.
```

```
warnings.warn(
```

```
C:\Users\SHUBHAM\anaconda3\lib\site-packages\sklearn\base.py:474: FutureWarning: `BaseEstimator._validate_data` is deprecated in 1.6 and will be removed in 1.7. Use `sklearn.utils.validation.validate_data` instead. This function becomes public and is part of the scikit-learn developer API.
```

```
warnings.warn(
```

```
C:\Users\SHUBHAM\anaconda3\lib\site-packages\sklearn\normalization\_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (200) reached and the optimization hasn't converged yet.
```

```
warnings.warn(
```

```
C:\Users\SHUBHAM\anaconda3\lib\site-packages\sklearn\base.py:474: FutureWarning: `BaseEstimator._validate_data` is deprecated in 1.6 and will be removed in 1.7. Use `sklearn.utils.validation.validate_data` instead. This function becomes public and is part of the scikit-learn developer API.
```

```
warnings.warn(
```

Fold 3 AUC (Ensemble): 0.9726

```
C:\Users\SHUBHAM\anaconda3\lib\site-packages\sklearn\base.py:474: FutureWarning: `BaseEstimator._validate_data` is deprecated in 1.6 and will be removed in 1.7. Use `sklearn.utils.validation.validate_data` instead. This function becomes public and is part of the scikit-learn developer API.
```

```
warnings.warn(
```

```
C:\Users\SHUBHAM\anaconda3\lib\site-packages\sklearn\base.py:474: FutureWarning: `BaseEstimator._validate_data` is deprecated in 1.6 and will be removed in 1.7. Use `sklearn.utils.validation.validate_data` instead. This function becomes public and is part of the scikit-learn developer API.
```

```
warnings.warn(
```

```
C:\Users\SHUBHAM\anaconda3\lib\site-packages\sklearn\normalization\_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (200) reached and the optimization hasn't converged yet.
```

```
warnings.warn(
```

```
C:\Users\SHUBHAM\anaconda3\lib\site-packages\sklearn\base.py:474: FutureWarning: `BaseEstimator._validate_data` is deprecated in 1.6 and will be removed in 1.7. Use `sklearn.utils.validation.validate_data` instead. This function becomes public and is part of the scikit-learn developer API.
```

```
warnings.warn(
```

Fold 4 AUC (Ensemble): 0.9741

```
C:\Users\SHUBHAM\anaconda3\lib\site-packages\sklearn\base.py:474: FutureWarning: `BaseEstimator._validate_data` is deprecated in 1.6 and will be removed in 1.7. Use `sklearn.utils.validation.validate_data` instead. This function becomes public and is part of the scikit-learn developer API.
```

```
warnings.warn(
```

```
C:\Users\SHUBHAM\anaconda3\lib\site-packages\sklearn\base.py:474: FutureWarning: `BaseEstimator._validate_data` is deprecated in 1.6 and will be removed in 1.7. Use `sklearn.utils.validation.validate_data` instead. This function becomes public and is part of the scikit-learn developer API.
```

```
warnings.warn(
```

```
C:\Users\SHUBHAM\anaconda3\lib\site-packages\sklearn\normalization\_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (200) reached and the optimization hasn't converged yet.
```

```
warnings.warn(
```

```
C:\Users\SHUBHAM\anaconda3\lib\site-packages\sklearn\base.py:474: FutureWarning: `BaseEstimator._validate_data` is deprecated in 1.6 and will be removed in 1.7. Use `sklearn.utils.validation.validate_data` instead. This function becomes public and is part of the scikit-learn developer API.
```

```
warnings.warn(
```

```
Fold 5 AUC (Ensemble): 0.9732
```

```
===== 5-FOLD ROC-AUC (ENSEMBLE) =====
```

```
Ensemble: mean=0.9723, std=0.0026
```

```
In [221]: pip install streamlit
```

Collecting streamlit

Downloading streamlit-1.12.0-py2.py3-none-any.whl (9.1 MB)

Requirement already satisfied: tornado>=5.0 in c:\users\shubham\anaconda3\lib\site-packages (from streamlit) (6.1)

Requirement already satisfied: python-dateutil in c:\users\shubham\anaconda3\lib\site-packages (from streamlit) (2.8.2)

Collecting pympler>=0.9

Downloading Pympler-1.1-py3-none-any.whl (165 kB)

Collecting altair>=3.2.0

Downloading altair-6.0.0-py3-none-any.whl (795 kB)

Requirement already satisfied: typing-extensions>=3.10.0.0 in c:\users\shubham\anaconda3\lib\site-packages (from streamlit) (3.10.0.2)

Requirement already satisfied: pillow>=6.2.0 in c:\users\shubham\anaconda3\lib\site-packages (from streamlit) (8.4.0)

Collecting tzlocal>=1.1

Downloading tzlocal-5.3.1-py3-none-any.whl (18 kB)

Requirement already satisfied: packaging>=14.1 in c:\users\shubham\anaconda3\lib\site-packages (from streamlit) (21.0)

Collecting pyarrow>=4.0

Downloading pyarrow-21.0.0-cp39-cp39-win_amd64.whl (26.2 MB)

Requirement already satisfied: pandas>=0.21.0 in c:\users\shubham\anaconda3\lib\site-packages (from streamlit) (1.3.4)

Requirement already satisfied: requests>=2.4 in c:\users\shubham\anaconda3\lib\site-packages (from streamlit) (2.26.0)

Collecting pydeck>=0.1.dev5

Downloading pydeck-0.9.1-py2.py3-none-any.whl (6.9 MB)

Requirement already satisfied: watchdog in c:\users\shubham\anaconda3\lib\site-packages (from streamlit) (2.1.3)

Requirement already satisfied: numpy in c:\users\shubham\anaconda3\lib\site-packages (from streamlit) (1.20.3)

Collecting rich>=10.11.0

Downloading rich-14.2.0-py3-none-any.whl (243 kB)

Requirement already satisfied: toml in c:\users\shubham\anaconda3\lib\site-packages (from streamlit) (0.10.2)

Collecting validators>=0.2

Downloading validators-0.35.0-py3-none-any.whl (44 kB)

Requirement already satisfied: importlib-metadata>=1.4 in c:\users\shubham\anaconda3\lib\site-packages (from streamlit) (4.8.1)

Collecting blinker>=1.0.0

Downloading blinker-1.9.0-py3-none-any.whl (8.5 kB)

Collecting gitpython!=3.1.19

Downloading gitpython-3.1.45-py3-none-any.whl (208 kB)

Requirement already satisfied: click>=7.0 in c:\users\shubham\anaconda3\lib\site-packages (from streamlit) (8.0.3)

Collecting cachetools>=4.0

Downloading cachetools-6.2.2-py3-none-any.whl (11 kB)

Collecting semver

Note: you may need to restart the kernel to use updated packages. Downloading semver-3.0.4-py3-none-any.whl (17 kB)

Collecting protobuf<4,>=3.12

Downloading protobuf-3.20.3-cp39-cp39-win_amd64.whl (904 kB)

Collecting narwhals>=1.27.1

Downloading narwhals-2.12.0-py3-none-any.whl (425 kB)

Requirement already satisfied: jsonschema>=3.0 in c:\users\shubham\anaconda3\lib\site-packages (from altair>=3.2.0->streamlit) (3.2.0)

Requirement already satisfied: jinja2 in c:\users\shubham\anaconda3\lib\site-packages (from altair>=3.2.0->streamlit) (2.11.3)

Collecting typing-extensions>=3.10.0.0

Downloading typing_extensions-4.15.0-py3-none-any.whl (44 kB)

Requirement already satisfied: colorama in c:\users\shubham\anaconda3\lib

```
\site-packages (from click>=7.0->streamlit) (0.4.4)  
Collecting gitdb<5,>=4.0.1
```

ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is the source of the following dependency conflicts.

spyder 5.1.5 requires PyQt5<5.13, which is not installed.

spyder 5.1.5 requires PyQtWebEngine<5.13, which is not installed.

```

    Downloading gitdb-4.0.12-py3-none-any.whl (62 kB)
Collecting smmap<6,>=3.0.1
    Downloading smmap-5.0.2-py3-none-any.whl (24 kB)
Requirement already satisfied: zipp>=0.5 in c:\users\shubham\anaconda3\lib\site-packages (from importlib-metadata>=1.4->streamlit) (3.6.0)
Requirement already satisfied: pyrsistent>=0.14.0 in c:\users\shubham\anaconda3\lib\site-packages (from jsonschema>=3.0->altair>=3.2.0->streamlit) (0.18.0)
Requirement already satisfied: setuptools in c:\users\shubham\anaconda3\lib\site-packages (from jsonschema>=3.0->altair>=3.2.0->streamlit) (58.0.4)
Requirement already satisfied: attrs>=17.4.0 in c:\users\shubham\anaconda3\lib\site-packages (from jsonschema>=3.0->altair>=3.2.0->streamlit) (21.2.0)
Requirement already satisfied: six>=1.11.0 in c:\users\shubham\anaconda3\lib\site-packages (from jsonschema>=3.0->altair>=3.2.0->streamlit) (1.16.0)
Requirement already satisfied: pyparsing>=2.0.2 in c:\users\shubham\anaconda3\lib\site-packages (from packaging>=14.1->streamlit) (3.0.4)
Requirement already satisfied: pytz>=2017.3 in c:\users\shubham\anaconda3\lib\site-packages (from pandas>=0.21.0->streamlit) (2021.3)
Requirement already satisfied: MarkupSafe>=0.23 in c:\users\shubham\anaconda3\lib\site-packages (from jinja2->altair>=3.2.0->streamlit) (1.1.1)
Requirement already satisfied: pywin32>=226 in c:\users\shubham\anaconda3\lib\site-packages (from pympler>=0.9->streamlit) (228)
Requirement already satisfied: idna<4,>=2.5 in c:\users\shubham\anaconda3\lib\site-packages (from requests>=2.4->streamlit) (3.2)
Requirement already satisfied: certifi>=2017.4.17 in c:\users\shubham\anaconda3\lib\site-packages (from requests>=2.4->streamlit) (2021.10.8)
Requirement already satisfied: charset-normalizer~=2.0.0 in c:\users\shubham\anaconda3\lib\site-packages (from requests>=2.4->streamlit) (2.0.4)
Requirement already satisfied: urllib3<1.27,>=1.21.1 in c:\users\shubham\anaconda3\lib\site-packages (from requests>=2.4->streamlit) (1.26.7)
Collecting markdown-it-py>=2.2.0
    Downloading markdown_it_py-3.0.0-py3-none-any.whl (87 kB)
Collecting pygments<3.0.0,>=2.13.0
    Downloading pygments-2.19.2-py3-none-any.whl (1.2 MB)
Collecting mdurl~=0.1
    Downloading mdurl-0.1.2-py3-none-any.whl (10.0 kB)
Collecting tzdata
    Downloading tzdata-2025.2-py2.py3-none-any.whl (347 kB)
Installing collected packages: smmap, mdurl, tzdata, typing-extensions, pygments, narwhals, markdown-it-py, gitdb, validators, tzlocal, semver, rich, pympler, pydeck, pyarrow, protobuf, gitpython, cachetools, blinker, altair, streamlit
Attempting uninstall: typing-extensions
    Found existing installation: typing-extensions 3.10.0.2
    Uninstalling typing-extensions-3.10.0.2:
        Successfully uninstalled typing-extensions-3.10.0.2
Attempting uninstall: pygments
    Found existing installation: Pygments 2.10.0
    Uninstalling Pygments-2.10.0:
        Successfully uninstalled Pygments-2.10.0
Successfully installed altair-6.0.0 blinker-1.9.0 cachetools-6.2.2 gitdb-4.0.12 gitpython-3.1.45 markdown-it-py-3.0.0 mdurl-0.1.2 narwhals-2.12.0 protobuf-3.20.3 pyarrow-21.0.0 pydeck-0.9.1 pygments-2.19.2 pympler-1.1 rich-14.2.0 semver-3.0.4 smmap-5.0.2 streamlit-1.12.0 typing-extensions-4.15.0 tzdata-2025.2 tzlocal-5.3.1 validators-0.35.0

```


In [227]: `pip install "altair<5"`

```
Collecting altair<5
  Downloading altair-4.2.2-py3-none-any.whl (813 kB)
Requirement already satisfied: pandas>=0.18 in c:\users\shubham\anaconda3\lib\site-packages (from altair<5) (1.3.4)
Requirement already satisfied: jsonschema>=3.0 in c:\users\shubham\anaconda3\lib\site-packages (from altair<5) (3.2.0)
Requirement already satisfied: toolz in c:\users\shubham\anaconda3\lib\site-packages (from altair<5) (0.11.1)
Requirement already satisfied: entrypoints in c:\users\shubham\anaconda3\lib\site-packages (from altair<5) (0.3)
Requirement already satisfied: numpy in c:\users\shubham\anaconda3\lib\site-packages (from altair<5) (1.20.3)
Requirement already satisfied: Jinja2 in c:\users\shubham\anaconda3\lib\site-packages (from altair<5) (2.11.3)
Requirement already satisfied: attrs>=17.4.0 in c:\users\shubham\anaconda3\lib\site-packages (from jsonschema>=3.0->altair<5) (21.2.0)
Requirement already satisfied: setuptools in c:\users\shubham\anaconda3\lib\site-packages (from jsonschema>=3.0->altair<5) (58.0.4)
Requirement already satisfied: pyrsistent>=0.14.0 in c:\users\shubham\anaconda3\lib\site-packages (from jsonschema>=3.0->altair<5) (0.18.0)
Requirement already satisfied: six>=1.11.0 in c:\users\shubham\anaconda3\lib\site-packages (from jsonschema>=3.0->altair<5) (1.16.0)
Requirement already satisfied: Pytz>=2017.3 in c:\users\shubham\anaconda3\lib\site-packages (from pandas>=0.18->altair<5) (2021.3)
Requirement already satisfied: python-dateutil>=2.7.3 in c:\users\shubham\anaconda3\lib\site-packages (from pandas>=0.18->altair<5) (2.8.2)
Requirement already satisfied: MarkupSafe>=0.23 in c:\users\shubham\anaconda3\lib\site-packages (from Jinja2->altair<5) (1.1.1)
Installing collected packages: altair
  Attempting uninstall: altair
    Found existing installation: altair 6.0.0
    Uninstalling altair-6.0.0:
      Successfully uninstalled altair-6.0.0
Successfully installed altair-4.2.2
Note: you may need to restart the kernel to use updated packages.
```

In [230]: `dir /s /b "C:\lightning_ensemble_model.pkl"`

File "C:\Users\SHUBHAM\AppData\Local\Temp\ipykernel_17164\510966884.py", line 1

`dir /s /b "C:\lightning_ensemble_model.pkl"`

^

SyntaxError: invalid syntax

```

In [231]: from sklearn.metrics import roc_curve, auc
import matplotlib.pyplot as plt

plt.figure(figsize=(8,6))

models = {
    "Logistic Regression": log_pipe,
    "MLP": mlp_pipe,
    "Gradient Boosting": gb_pipe,
}

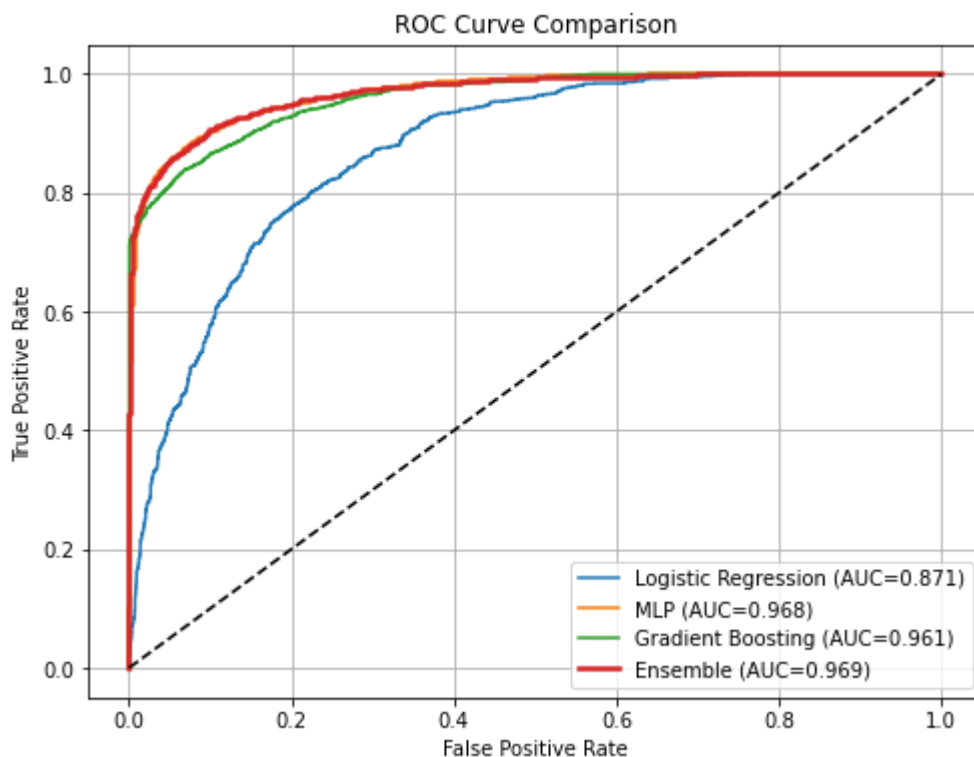
for name, model in models.items():
    prob = model.predict_proba(X_test)[: , 1]
    fpr, tpr, _ = roc_curve(y_test, prob)
    roc_auc = auc(fpr, tpr)
    plt.plot(fpr, tpr, label=f"{name} (AUC={roc_auc:.3f})")

# ensemble
ens_prob = (log_pipe.predict_proba(X_test)[: , 1] +
            mlp_pipe.predict_proba(X_test)[: , 1] +
            gb_pipe.predict_proba(X_test)[: , 1]) / 3

fpr, tpr, _ = roc_curve(y_test, ens_prob)
roc_auc = auc(fpr, tpr)
plt.plot(fpr, tpr, label=f"Ensemble (AUC={roc_auc:.3f})", linewidth=2.5)

plt.plot([0,1],[0,1], 'k--')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve Comparison")
plt.legend()
plt.grid(True)
plt.show()

```



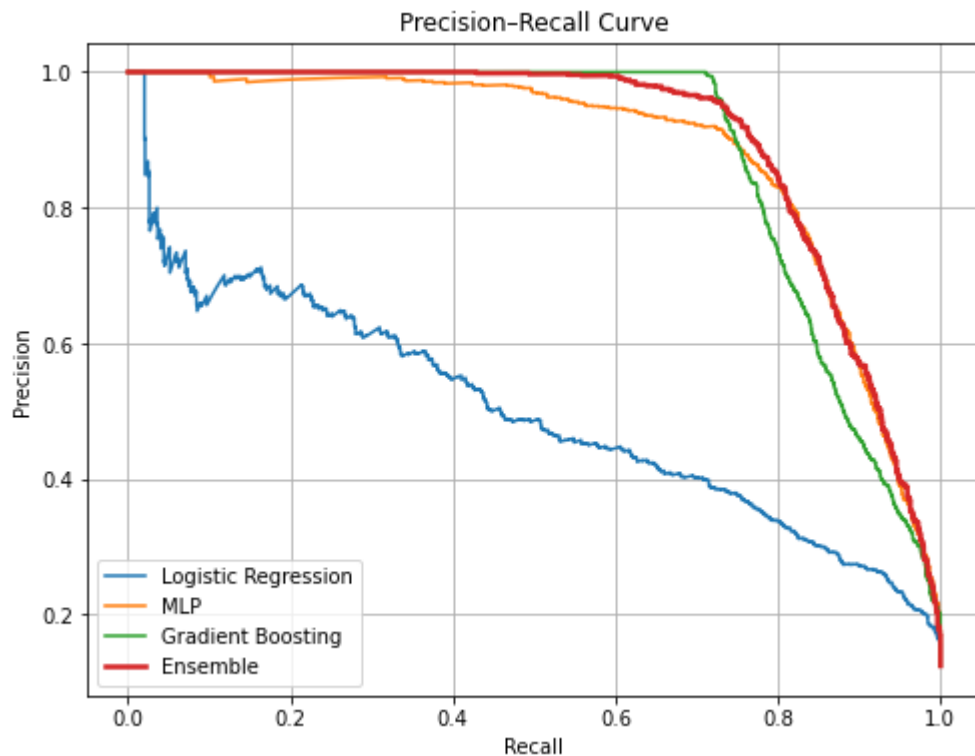
```
In [232]: from sklearn.metrics import precision_recall_curve

plt.figure(figsize=(8,6))

for name, model in models.items():
    prob = model.predict_proba(X_test)[: , 1]
    precision, recall, _ = precision_recall_curve(y_test, prob)
    plt.plot(recall, precision, label=name)

precision, recall, _ = precision_recall_curve(y_test, ens_prob)
plt.plot(recall, precision, label="Ensemble", linewidth=2.5)

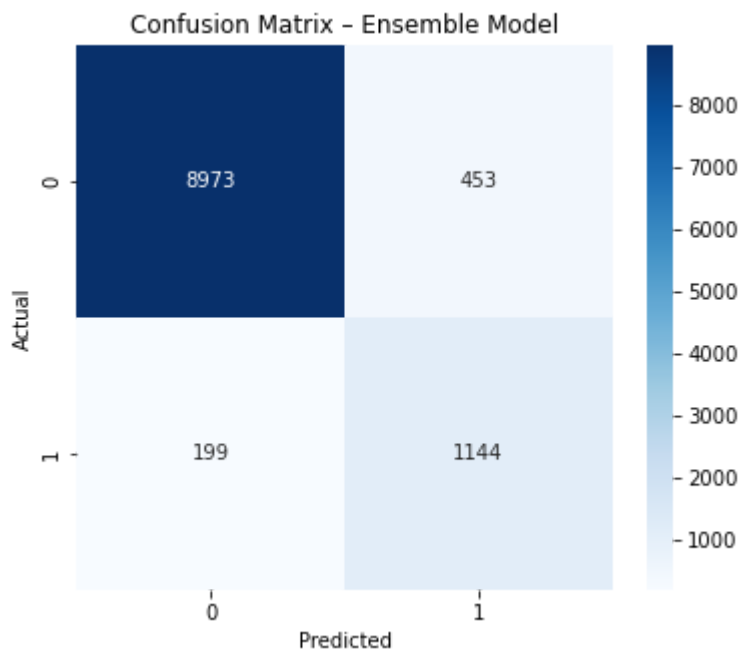
plt.xlabel("Recall")
plt.ylabel("Precision")
plt.title("Precision-Recall Curve")
plt.legend()
plt.grid(True)
plt.show()
```



```
In [233]: from sklearn.metrics import confusion_matrix
import seaborn as sns

cm = confusion_matrix(y_test, (ens_prob >= 0.5).astype(int))

plt.figure(figsize=(6,5))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix - Ensemble Model")
plt.show()
```



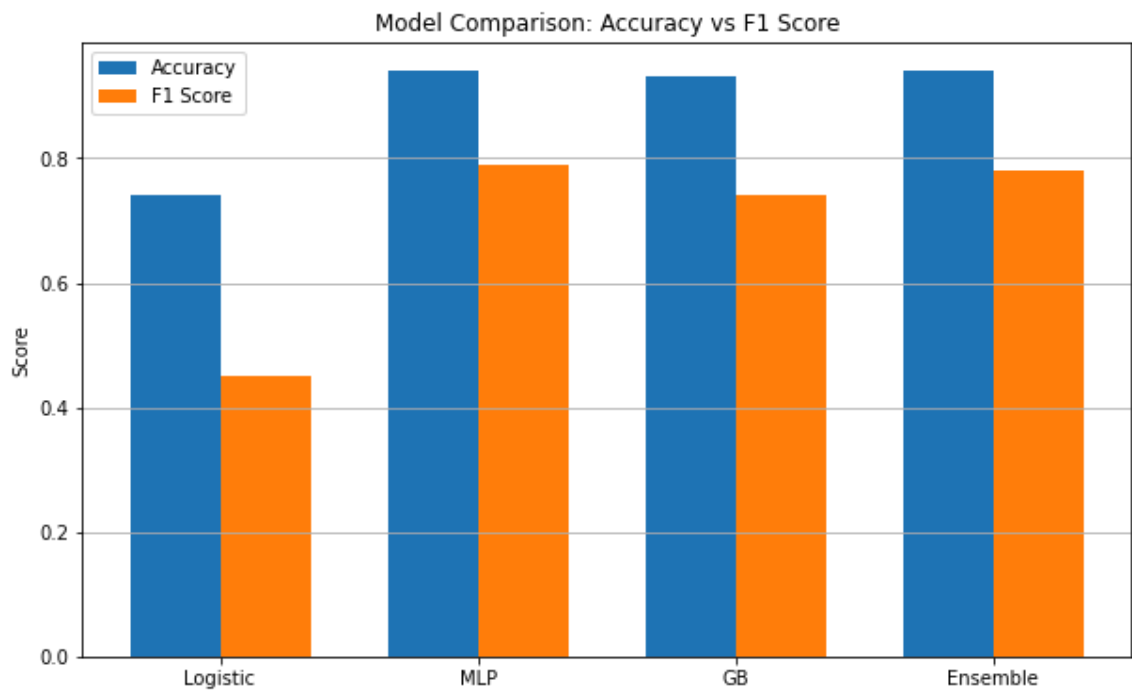
```
In [235]: import numpy as np

models_names = ["Logistic", "MLP", "GB", "Ensemble"]
accuracies = [0.74, 0.94, 0.93, 0.94]
f1_scores = [0.45, 0.79, 0.74, 0.78]

x = np.arange(len(models_names))
w = 0.35

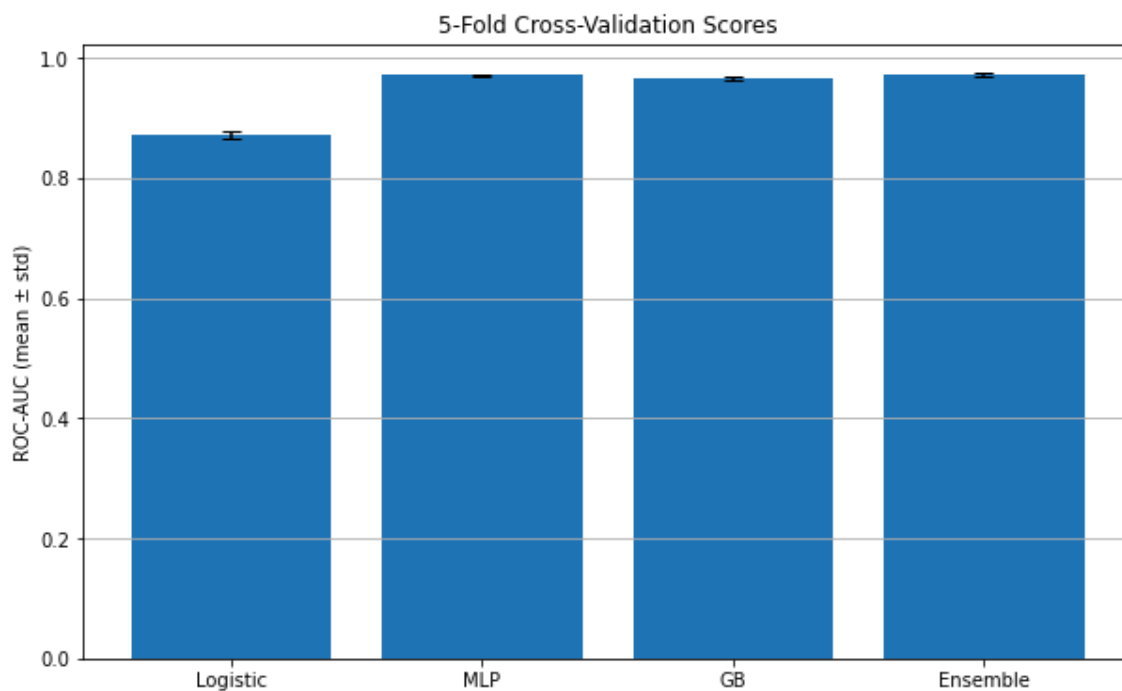
plt.figure(figsize=(10,6))
plt.bar(x - w/2, accuracies, width=w, label="Accuracy")
plt.bar(x + w/2, f1_scores, width=w, label="F1 Score")

plt.xticks(x, models_names)
plt.ylabel("Score")
plt.title("Model Comparison: Accuracy vs F1 Score")
plt.legend()
plt.grid(axis="y")
plt.show()
```



```
In [236]: cv_means = [0.8717, 0.9708, 0.9656, 0.9723]
cv_std    = [0.0066, 0.0017, 0.0027, 0.0026]

plt.figure(figsize=(10,6))
plt.bar(models_names, cv_means, yerr=cv_std, capsize=5)
plt.ylabel("ROC-AUC (mean  $\pm$  std)")
plt.title("5-Fold Cross-Validation Scores")
plt.grid(axis="y")
plt.show()
```



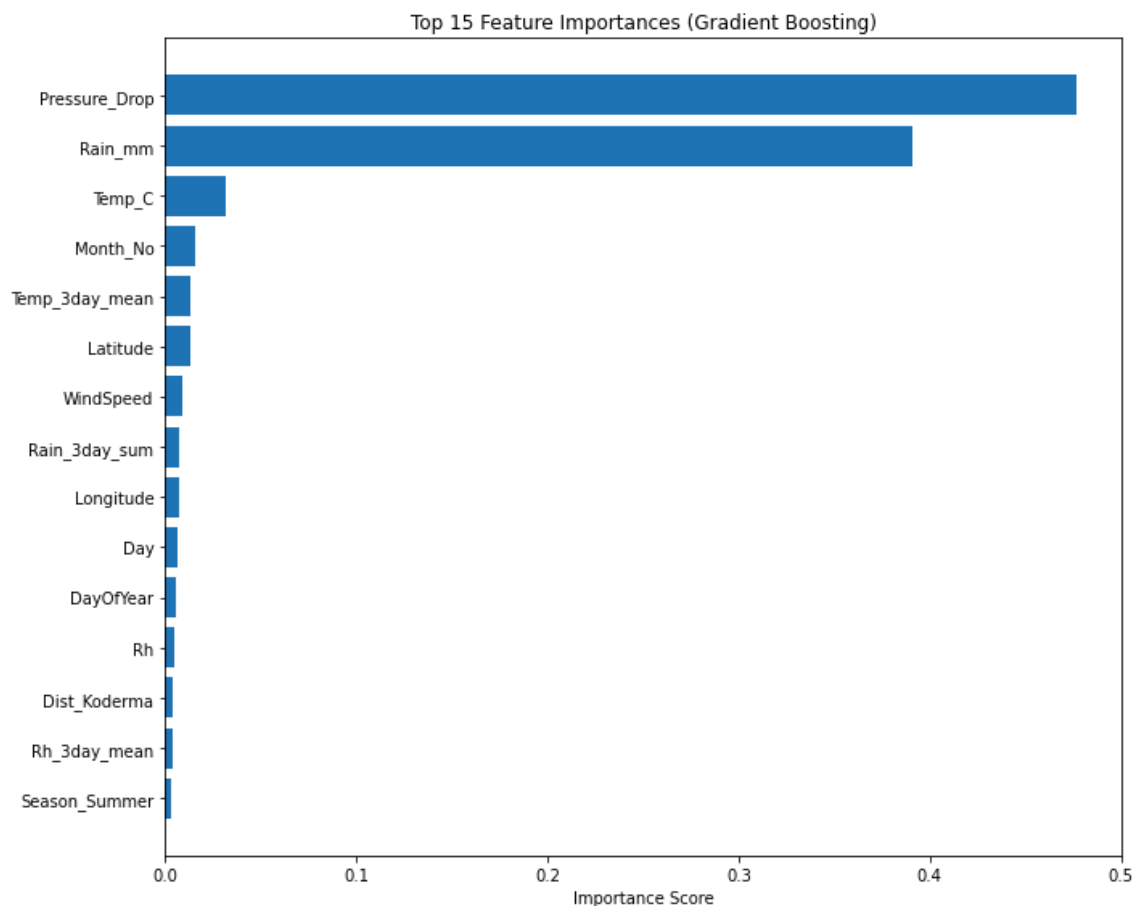
```
In [239]: import numpy as np
import matplotlib.pyplot as plt

# Get the final estimator from the pipeline (regardless of its step name)
gb_model = list(gb_pipe.named_steps.values())[-1]

# Now get feature importances
gb_importances = gb_model.feature_importances_

# Use your feature_list (must match training order)
sorted_idx = np.argsort(gb_importances)[::-1]

plt.figure(figsize=(10, 8))
plt.barh(np.array(feature_list)[sorted_idx][:15], gb_importances[sorted_idx][:15])
plt.gca().invert_yaxis()
plt.xlabel("Importance Score")
plt.title("Top 15 Feature Importances (Gradient Boosting)")
plt.tight_layout()
plt.show()
```



```
In [244]: import pickle

with open("X:/Lighting Prediction/lightning_ensemble_model.pkl", "rb") as f:
    ens = pickle.load(f)

log_pipe = ens["log_pipe"]
mlp_pipe = ens["mlp_pipe"]
gb_pipe = ens["gb_pipe"]
feature_list = ens["feature_list"]
```

```
In [245]: import pandas as pd

df = pd.read_csv("X:/Lighting Prediction/lightning_final_FE_dataset.csv")
df["Date"] = pd.to_datetime(df["Date"])

X = df[feature_list]
y = df["LightningOccurred"]
```

```
In [246]: from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, stratify=y, test_size=0.20, random_state=42
)
```

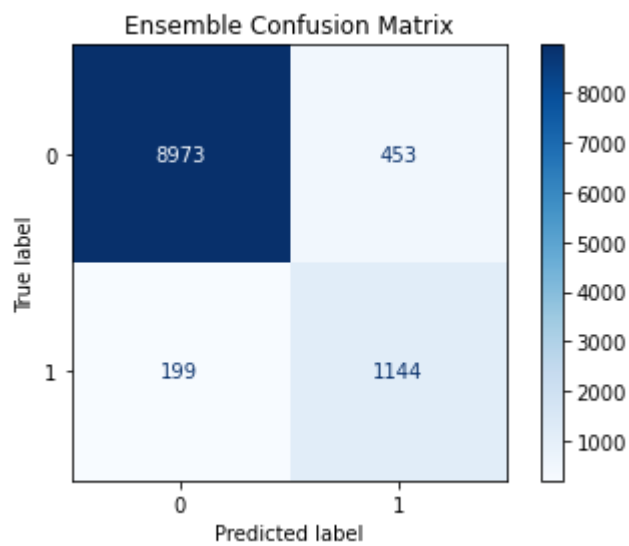


```
In [247]: from sklearn.metrics import ConfusionMatrixDisplay
import matplotlib.pyplot as plt

preds = []
preds.append(log_pipe.predict_proba(X_test)[: ,1])
preds.append(mlp_pipe.predict_proba(X_test)[: ,1])
preds.append(gb_pipe.predict_proba(X_test)[: ,1])

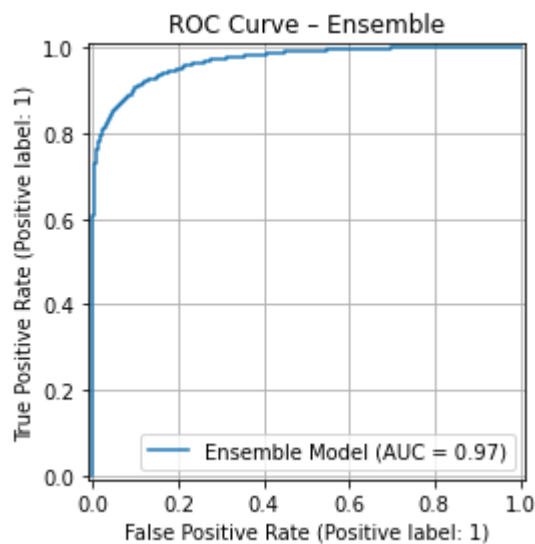
import numpy as np
ens_prob = np.mean(preds, axis=0)
ens_pred = (ens_prob >= 0.5).astype(int)

cm = confusion_matrix(y_test, ens_pred)
disp = ConfusionMatrixDisplay(cm)
disp.plot(cmap="Blues")
plt.title("Ensemble Confusion Matrix")
plt.show()
```

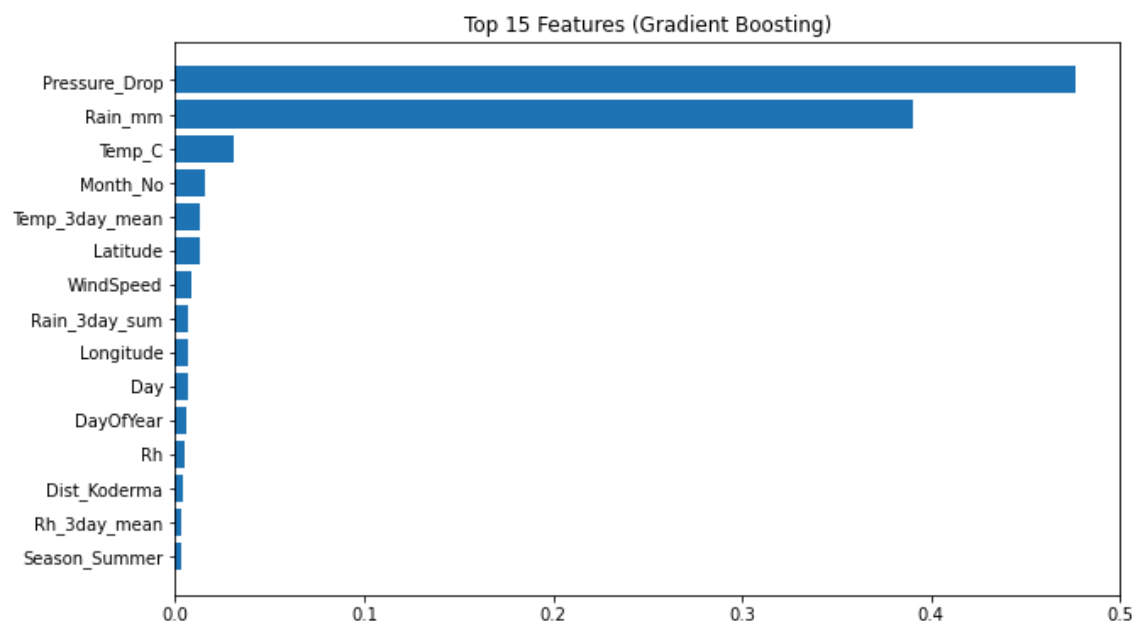


```
In [248]: from sklearn.metrics import RocCurveDisplay
```

```
RocCurveDisplay.from_predictions(  
    y_test,  
    ens_prob,  
    name="Ensemble Model"  
)  
plt.title("ROC Curve - Ensemble")  
plt.grid(True)  
plt.show()
```

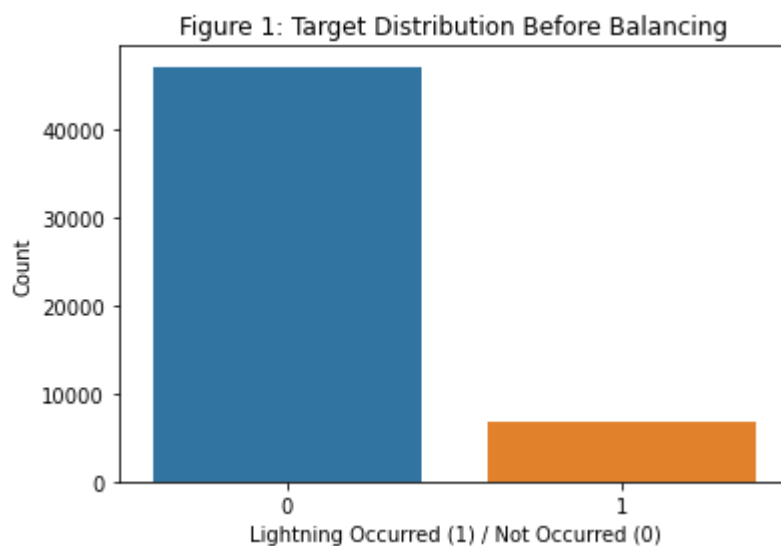


```
In [242]: gb_model = list(gb_pipe.named_steps.values())[-1]  
importances = gb_model.feature_importances_  
sorted_idx = np.argsort(importances)[::-1]  
  
plt.figure(figsize=(10,6))  
plt.barh(np.array(feature_list)[sorted_idx][:15], importances[sorted_idx][:15])  
plt.gca().invert_yaxis()  
plt.title("Top 15 Features (Gradient Boosting)")  
plt.show()
```

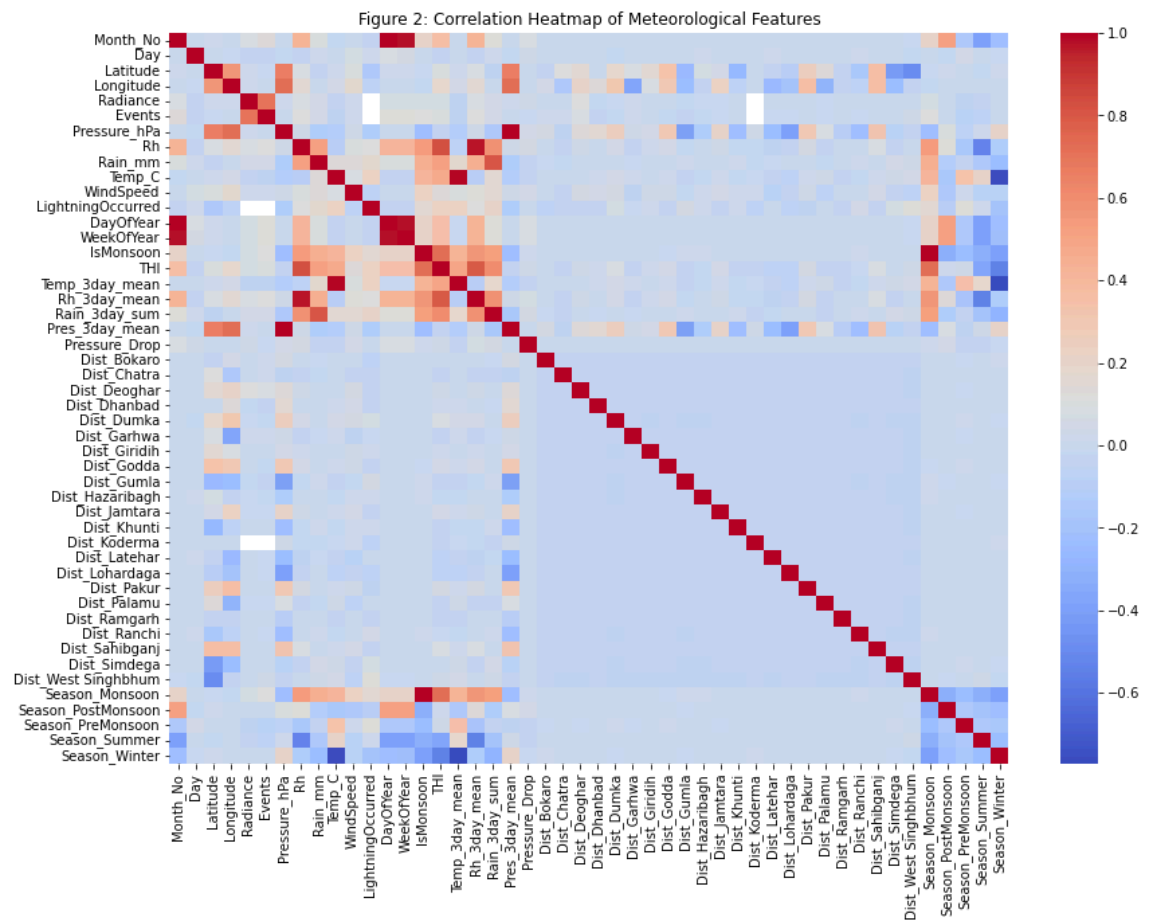


```
In [249]: import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(6,4))
sns.countplot(x=y)
plt.title("Figure 1: Target Distribution Before Balancing")
plt.xlabel("Lightning Occurred (1) / Not Occurred (0)")
plt.ylabel("Count")
plt.show()
```

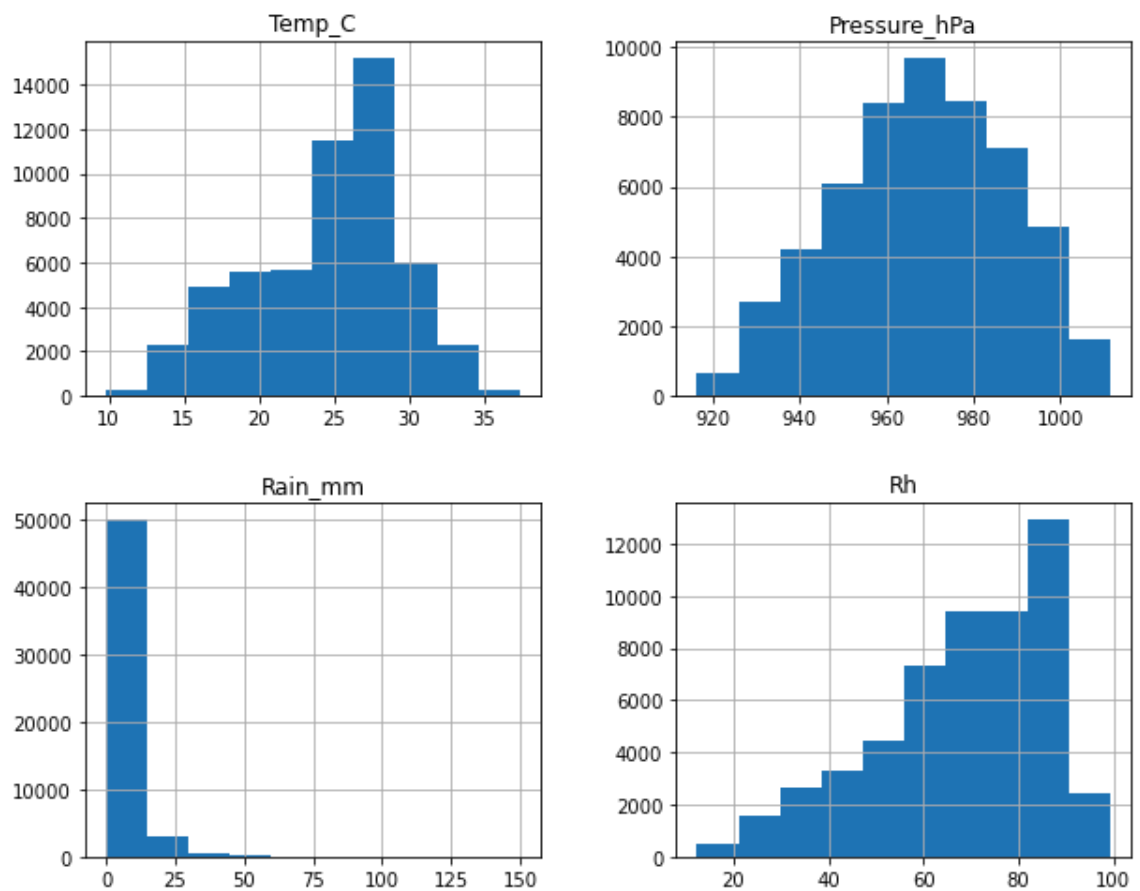


```
In [250]: plt.figure(figsize=(14,10))
sns.heatmap(df.corr(), cmap="coolwarm", annot=False)
plt.title("Figure 2: Correlation Heatmap of Meteorological Features")
plt.show()
```



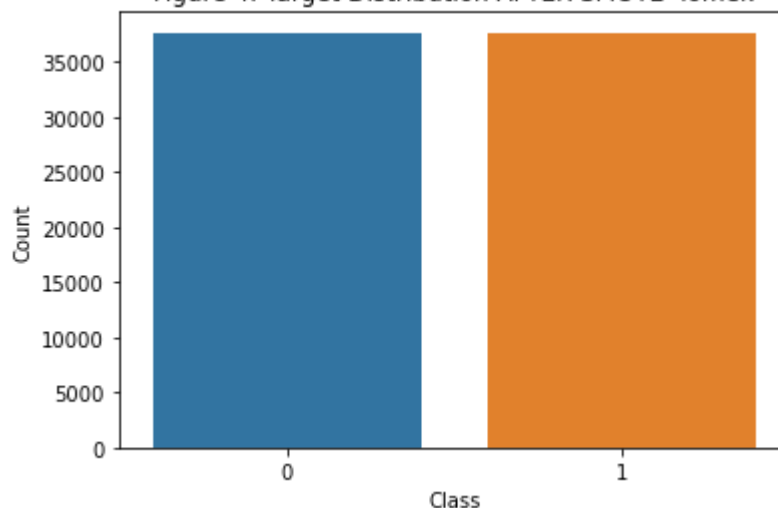
```
In [251]: df[["Temp_C", "Pressure_hPa", "Rain_mm", "Rh"]].hist(figsize=(10,8))  
plt.suptitle("Figure 3: Feature Distribution Plots")  
plt.show()
```

Figure 3: Feature Distribution Plots



```
In [252]: plt.figure(figsize=(6,4))  
sns.countplot(x=y_train_bal)  
plt.title("Figure 4: Target Distribution AFTER SMOTE-Tomek")  
plt.xlabel("Class")  
plt.ylabel("Count")  
plt.show()
```

Figure 4: Target Distribution AFTER SMOTE-Tomek



```
In [253]: print("Table 2 – Before Balancing")
print(y.value_counts())
print("\nAfter Balancing (SMOTE-Tomek)")
print(y_train_bal.value_counts())
```

Table 2 – Before Balancing

0 47130

1 6713

Name: LightningOccurred, dtype: int64

After Balancing (SMOTE-Tomek)

0 37643

1 37643

Name: LightningOccurred, dtype: int64

```

In [258]: import pickle

# Load ensemble (contains log, mlp, gb + feature list)
with open("lightning_ensemble_model.pkl", "rb") as f:
    ens = pickle.load(f)

log_reg      = ens["log_pipe"]
mlp          = ens["mlp_pipe"]
gb           = ens["gb_pipe"]
feature_list = ens["feature_list"] # List of 45 features

# Load RF model if needed
with open("lightning_rf_model.pkl", "rb") as f:
    rf_model = pickle.load(f)

# Load GB calibrated if needed
with open("lightning_gb_calibrated.pkl", "rb") as f:
    gb_cal = pickle.load(f)

# Load feature-engineered dataset to compute metrics
df = pd.read_csv("lightning_final_FE_dataset.csv")

# Extract X, y
X = df[feature_list]
y = df["LightningOccurred"]

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Scale for linear models (SVM, Logistic, SGD)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled  = scaler.transform(X_test)

# Train classic models
from sklearn.linear_model import LogisticRegression, SGDClassifier
from sklearn.svm import SVC

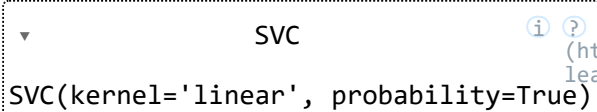
log_only = LogisticRegression(max_iter=500)
log_only.fit(X_train_scaled, y_train)

sgd = SGDClassifier(loss="log_loss")
sgd.fit(X_train_scaled, y_train)

svm = SVC(kernel="linear", probability=True)
svm.fit(X_train_scaled, y_train)

```

Out[258]:

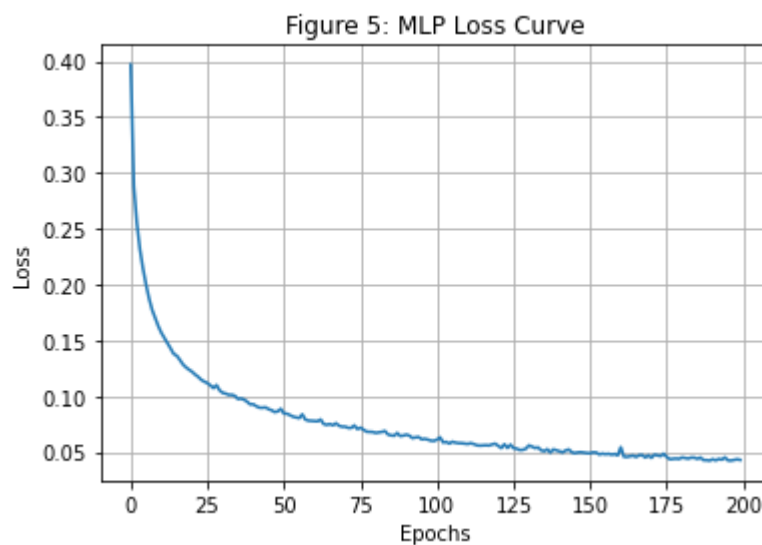

 SVC(kernel='linear', probability=True)
 <https://scikit-learn.org/1.6/modules/generated/sklearn.svm.SVC.html>

```
In [260]: import matplotlib.pyplot as plt

# get the final estimator of the pipeline
mlp_clf = list(mlp.named_steps.values())[-1]
print(mlp_clf)  # just to see what it is

plt.figure(figsize=(6,4))
plt.plot(mlp_clf.loss_curve_)
plt.title("Figure 5: MLP Loss Curve")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.grid(True)
plt.show()
```

MLPClassifier(alpha=0.001, hidden_layer_sizes=(64, 32), random_state=42)



```
In [261]: models = {
    "Logistic Regression": log_only,
    "Linear SVM": svm,
    "SGD Classifier": sgd,
    "Random Forest": rf_model,
    "MLP": mlp,
    "Gradient Boosting": gb,
    "Ensemble Model": ens
}

print("Models Loaded:", list(models.keys()))
```

Models Loaded: ['Logistic Regression', 'Linear SVM', 'SGD Classifier', 'Random Forest', 'MLP', 'Gradient Boosting', 'Ensemble Model']


```
In [262]: from sklearn.metrics import accuracy_score, f1_score, roc_auc_score

results = []

for name, m in models.items():

    if name == "Ensemble Model":
        p_log = log_reg.predict_proba(X_test)[:,-1]
        p_mlp = mlp.predict_proba(X_test)[:,-1]
        p_gb = gb.predict_proba(X_test)[:,-1]
        prob = (p_log + p_mlp + p_gb)/3
        pred = (prob >= 0.5).astype(int)
        auc = roc_auc_score(y_test, prob)

    else:
        pred = m.predict(X_test_scaled)
        prob = m.predict_proba(X_test_scaled)[:,-1]
        auc = roc_auc_score(y_test, prob)

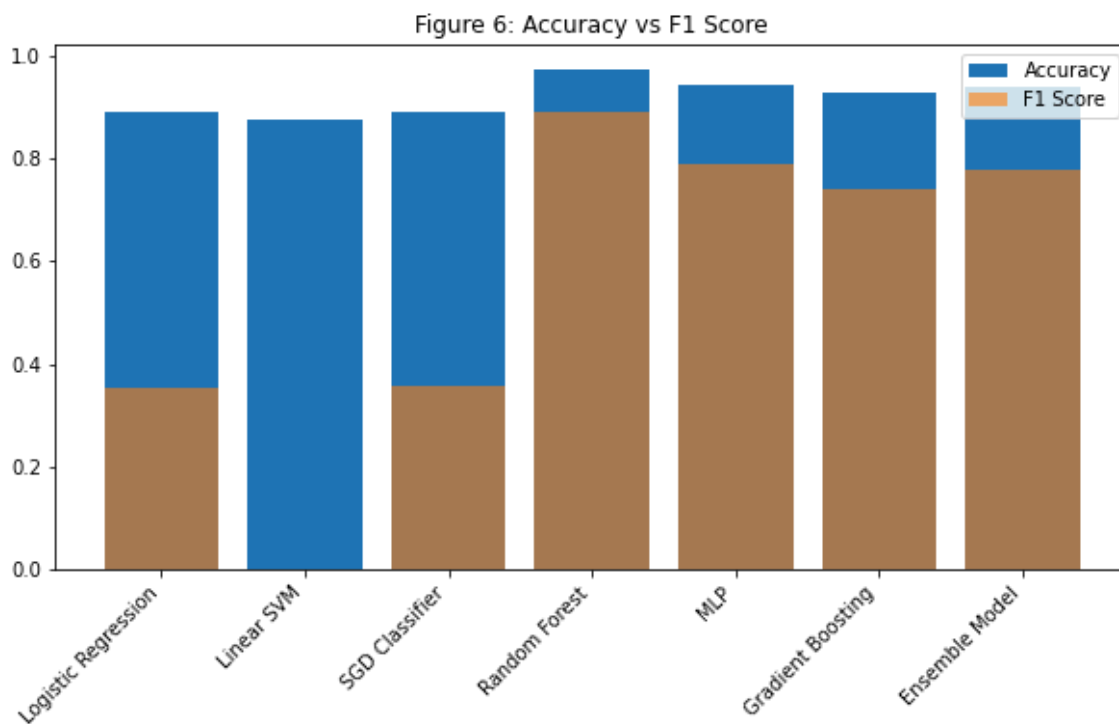
    results.append([name, accuracy_score(y_test, pred), f1_score(y_test, pred), auc])
```

```
In [263]: table3 = pd.DataFrame(results, columns=["Model", "Accuracy", "F1 Score", "ROC-AUC"])
table3
```

Out[263]:

	Model	Accuracy	F1 Score	ROC-AUC
0	Logistic Regression	0.890705	0.353652	0.872182
1	Linear SVM	0.875290	0.000000	0.852087
2	SGD Classifier	0.889405	0.357258	0.859976
3	Random Forest	0.973257	0.889571	0.988049
4	MLP	0.943449	0.788174	0.968169
5	Gradient Boosting	0.928684	0.741762	0.961437
6	Ensemble Model	0.939456	0.778231	0.969495

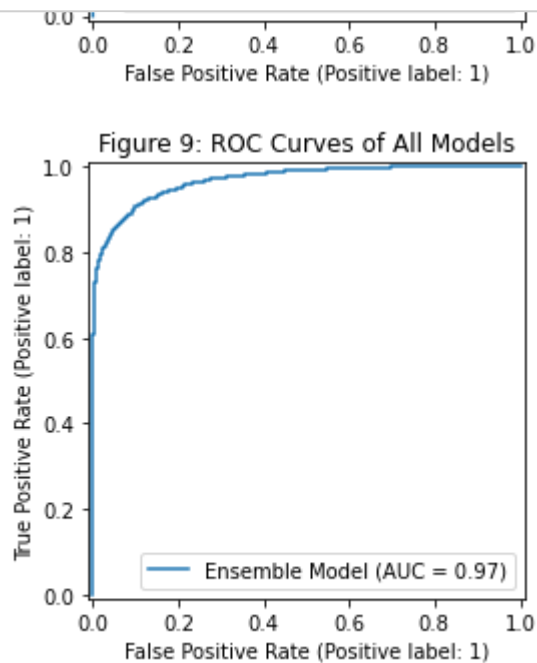
```
In [264]: plt.figure(figsize=(10,5))
plt.bar(table3["Model"], table3["Accuracy"], label="Accuracy")
plt.bar(table3["Model"], table3["F1 Score"], alpha=0.6, label="F1 Score")
plt.xticks(rotation=45, ha='right')
plt.title("Figure 6: Accuracy vs F1 Score")
plt.legend()
plt.show()
```



```
In [265]: plt.figure(figsize=(10,8))

for name, m in models.items():
    if name == "Ensemble Model":
        p_log = log_reg.predict_proba(X_test)[: ,1]
        p_mlp = mlp.predict_proba(X_test)[: ,1]
        p_gb = gb.predict_proba(X_test)[: ,1]
        prob = (p_log + p_mlp + p_gb)/3
        RocCurveDisplay.from_predictions(y_test, prob, name=name)
    else:
        RocCurveDisplay.from_estimator(
            m,
            X_test_scaled if name in ["Logistic Regression", "Linear SVM", "S
            y_test,
            name=name
        )

plt.title("Figure 9: ROC Curves of All Models")
plt.show()
```



In []: